HRM System Project Code

File: README.md

```
# HRM System

A Human Resource Management system built using:
- FastAPI backend
- Next.js frontend
- Separate AI service for CV parsing and scoring

## Project Structure

backend/      - HRM backend (FastAPI)
frontend/     - Web frontend (Next.js)
ai-service/   - Internal AI service (FastAPI)

## Architecture

- Modular monolithic backend
- Separate AI service
- PostgreSQL database
- JWT authentication with RBAC
```

File: backend\app\main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.database.database import engine
from app.database.base import Base
from app.leave import models as leave_models
from app.employees import models as employee_models  # if exists
from app.auth import models as auth_models  # if exists# ■ import router
from app.leave.router import router as leave_router
from fastapi.staticfiles import StaticFiles
from app.reports.router import router as reports_router


app = FastAPI(title="HRM Backend")

app.mount("/uploads", StaticFiles(directory="uploads"), name="uploads") # new added



app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000", "http://127.0.0.1:3000", "http://localhost:3001", "
http://127.0.0.1:3001"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.on_event("startup")
def startup():
    if engine:
        try:
            # ■ Create tables
            Base.metadata.create_all(bind=engine)

            # ■ Connection test
            with engine.connect() as connection:
                pass
            print("✓ Database connected successfully")
        except Exception as e:
```

```
            print(f"✗ Database connection failed: {e}")
    else:
        print("■ Database engine not initialized")

app.include_router(leave_router)
app.include_router(reports_router)

@app.get("/")
def root():
    return {"message": "HRM backend with DB connected"}

@app.get("/health")
def health():
    return {"status": "ok"}
```

File: backend\app__init__.py

File: backend\app\auth\models.py

File: backend\app\auth\router.py

File: backend\app\auth\schemas.py

File: backend\app\auth\service.py

File: backend\app\auth__init__.py

File: backend\app\core\config.py

File: backend\app\core\rbac.py

```
from fastapi import Header, HTTPException, Depends
from sqlalchemy.orm import Session
from app.database.database import get_db
from app.employees.models import Employee


def get_current_user(
    x_user_id: str = Header(...),
    x_user_roles: str = Header(...),
):
    return {
        "id": int(x_user_id),
        "role": x_user_roles.strip().lower(),
    }
```

```
def require_roles(allowed_roles: list[str]):
    def checker(current_user: dict = Depends(get_current_user)):
        if current_user["role"] not in [r.lower() for r in allowed_roles]:
            raise HTTPException(status_code=403, detail="Access denied")
        return current_user
    return checker
```

File: backend\app\core\security.py

File: backend\app\core__init__.py

File: backend\app\database\base.py

```
from sqlalchemy.orm import declarative_base
Base = declarative_base()
```

File: backend\app\database\database.py

```
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from dotenv import load_dotenv
from sqlalchemy.orm import Session


load_dotenv()

# Try to use DATABASE_URL directly from env, fallback to component-based construction
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    f"postgresql://{os.getenv('DB_USER', 'postgres')}:"
    f"{os.getenv('DB_PASSWORD', 'postgres')}"
    f"@{os.getenv('DB_HOST', 'localhost')}:"
    f"{os.getenv('DB_PORT', '5432')}/{os.getenv('DB_NAME', 'hrm_db')}"
)

print(f"Database URL: {DATABASE_URL}")

try:
    engine = create_engine(DATABASE_URL, echo=False)
except Exception as e:
    print(f"Error creating engine: {e}")
    engine = None

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
) if engine else None

def get_db():
    if SessionLocal is None:
        raise RuntimeError("Database session is not available. Engnie creation failed.")
    db: Session= SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

File: backend\app\database__init__.py

File: backend\app\documents\model.py

File: backend\app\documents\router.py

File: backend\app\documents\schemas.py

File: backend\app\documents\service.py

File: backend\app\documents__init__.py

File: backend\app\employees\models.py

```
from sqlalchemy import Column, String, Integer, Date, Enum, Text
from app.database.base import Base
import enum


class EmployeeStatus(enum.Enum):
    active = "active"
    inactive = "inactive"


class Employee(Base):
    __tablename__ = "employees"

    id = Column(Integer, primary_key=True, index=True)
    employee_id = Column(String(50), unique=True, index=True)
    first_name = Column(String(100), nullable=False)
    last_name = Column(String(100), nullable=False)
    email = Column(String(255), unique=True, index=True, nullable=False)
    phone = Column(String(20), nullable=False)
    address = Column(Text, nullable=True)
    department = Column(String(100), nullable=False)
    designation = Column(String(100), nullable=False)
    joined_date = Column(Date, nullable=True)
    status = Column(Enum(EmployeeStatus), default=EmployeeStatus.active)

    date_of_birth = Column(Date, nullable=True)
    gender = Column(String(20), nullable=True)
    marital_status = Column(String(20), nullable=True)
    nationality = Column(String(100), nullable=True)
```

File: backend\app\employees\router.py

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session

from app.database.database import get_db
from app.leave.schemas import LeaveRequestCreate, LeaveRequestOut, LeaveStatusUpdate
from app.leave.service import create_leave, my_requests, pending_requests, update_status

router = APIRouter(prefix="/leave", tags=["Leave"])



@router.post("/requests", response_model=LeaveRequestOut)
def submit_leave(payload: LeaveRequestCreate, db: Session = Depends(get_db)):
    try:
        return create_leave(db, get_current_employee_id(), payload)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))


@router.get("/requests/me", response_model=list[LeaveRequestOut])
def get_my_leave(db: Session = Depends(get_db)):
    return my_requests(db, get_current_employee_id())

@router.get("/requests/pending", response_model=list[LeaveRequestOut])
def get_pending(db: Session = Depends(get_db)):
    return pending_requests(db)

@router.patch("/requests/{request_id}/status", response_model=LeaveRequestOut)
def change_status(request_id: int, payload: LeaveStatusUpdate, db: Session = Depends(get_db)):

    allowed = {"APPROVED", "REJECTED", "REQ_INFO", "PENDING"}

    if payload.status not in allowed:
        raise HTTPException(status_code=400, detail="Invalid status")

    updated = update_status(
        db,
        request_id,
        payload.status,
        approved_by=get_current_manager_id(),
        rejection_reason=payload.rejection_reason,
    )

    if not updated:
        raise HTTPException(status_code=404, detail="Leave request not found")

    return updated
```

File: backend\app\employees\schemas.py

```
from pydantic import BaseModel, EmailStr
from datetime import date
from typing import Optional
from enum import Enum
from typing import List

class EmployeeStatus(str, Enum):
    active = "active"
    inactive = "inactive"

class EmployeeBase(BaseModel):

    employee_id: str
    first_name: str
    last_name: str
    email: EmailStr
    phone: str
    address: Optional[str] = None
    department: str
```

```
    designation: str
    joined_date: Optional[date] = None
    status: EmployeeStatus = EmployeeStatus.active
    date_of_birth: Optional[date] = None
    gender: Optional[str] = None
    marital_status: Optional[str] = None
    nationality: Optional[str] = None

class EmployeeCreate(EmployeeBase):
    pass

class EmployeeUpdate(BaseModel):
    first_name: Optional[str] = None
    last_name: Optional[str] = None
    email: Optional[EmailStr] = None
    phone: Optional[str] = None
    address: Optional[str] = None
    department: Optional[str] = None
    designation: Optional[str] = None
    joined_date: Optional[date] = None
    status: Optional[EmployeeStatus] = None
    date_of_birth: Optional[date] = None
    gender: Optional[str] = None
    marital_status: Optional[str] = None
    nationality: Optional[str] = None

class RoleInfo(BaseModel):
    id: int
    name: str
    class Config:
        from_attributes = True

class EmployeeOut(EmployeeBase):
    id: int
    roles: List[str] = []

    class Config:
        from_attributes = True
```

File: backend\app\employees\service.py

File: backend\app\employees__init__.py

File: backend\app\leave\models.py

```
from sqlalchemy import Column, Integer, String, Date, Float, Text, DateTime, Boolean, ForeignKey, JSON
from sqlalchemy.sql import func
from app.database.base import Base
from sqlalchemy import JSON

class LeaveType(Base):
    __tablename__ = "leave_types"
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(50), nullable=False, unique=True)
    description = Column(String(255), nullable=True)

class LeaveRequest(Base):
    __tablename__ = "leave_requests"

    leave_request_id = Column(Integer, primary_key=True, index=True)

    employee_id = Column(Integer, nullable=False)          # later FK to employees.id
```

```
    leave_type_id = Column(Integer, ForeignKey("leave_types.id"), nullable=False)

    approved_by = Column(Integer, nullable=True)           # later FK to employees.id

    start_date = Column(Date, nullable=False)
    end_date = Column(Date, nullable=False)

    total_days = Column(Float, nullable=False)
    half_day = Column(Boolean, default=False)

    status = Column(String(20), default="PENDING")

    reason = Column(Text, nullable=True)
    attachment_urls = Column(JSON, nullable=True)  #new added
    rejection_reason = Column(Text, nullable=True)

    manager_comment = Column(Text, nullable=True) #new added 29/03

    approved_date = Column(Date, nullable=True)

    created_at = Column(DateTime(timezone=True), server_default=func.now())
    updated_at = Column(DateTime(timezone=True), server_default=func.now(), onupdate=func.now())
```

File: backend\app\leave\router.py

```
from fastapi import APIRouter, Depends, HTTPException, UploadFile, File
import os
import shutil
from uuid import uuid4
from sqlalchemy.orm import Session

from app.core.rbac import require_roles, get_current_user
from app.database.database import get_db

from app.leave.schemas import (
    LeaveRequestCreate,
    LeaveRequestOut,
    LeaveStatusUpdate,
    LeaveTypeCreate,
    LeaveTypeOut,
    ApproveLeaveRequest,
    RejectLeaveRequest,
    RequestInfoLeaveRequest,
    ResubmitLeaveRequest,
)

from app.leave.service import (
    create_leave,
    my_requests,
    get_pending_requests,
    update_status,
    get_leave_types,
    create_leave_type,
    get_leave_history,
    get_leave_request_by_id,
    approve_leave_request,
    reject_leave_request,
    request_info_leave_request,
    resubmit_leave_request,
    delete_leave_request,
    update_leave_request,
)

router = APIRouter(prefix="/leave", tags=["Leave"])

UPLOAD_DIR = "uploads/medical_docs"
os.makedirs(UPLOAD_DIR, exist_ok=True)

# ----------------------------
```

```
# CREATE LEAVE (EMPLOYEE + HR)
# ----------------------------
@router.post("/requests", response_model=LeaveRequestOut)
def submit_leave(
    payload: LeaveRequestCreate,
    current_user: dict = Depends(require_roles(["employee", "hr"])),
    db: Session = Depends(get_db),
):
    return create_leave(db, current_user["id"], payload)


# ----------------------------
# UPDATE LEAVE (EMPLOYEE ONLY)
# ----------------------------
@router.put("/requests/{request_id}", response_model=LeaveRequestOut)
def update_leave(
    request_id: int,
    payload: LeaveRequestCreate,
    current_user: dict = Depends(require_roles(["employee", "hr"])),
    db: Session = Depends(get_db),
):
    try:
        return update_leave_request(db, request_id, current_user["id"], payload)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))


# ----------------------------
# DELETE LEAVE (EMPLOYEE ONLY)
# ----------------------------
@router.delete("/requests/{request_id}")
def delete_leave(
    request_id: int,
    current_user: dict = Depends(require_roles(["employee", "hr"])),
    db: Session = Depends(get_db),
):
    try:
        delete_leave_request(db, request_id, current_user["id"])
        return {"detail": "Leave request deleted"}
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))


# ----------------------------
# MY LEAVES
# ----------------------------
@router.get("/requests/me", response_model=list[LeaveRequestOut])
def get_my_leave(
    current_user: dict = Depends(require_roles(["employee", "hr"])),
    db: Session = Depends(get_db),
):
    return my_requests(db, current_user["id"])


# ----------------------------
# PENDING REQUESTS (HR ONLY)
# ----------------------------
@router.get("/requests/pending", response_model=list[LeaveRequestOut])
def get_pending(
    current_user: dict = Depends(require_roles(["hr"])),
    db: Session = Depends(get_db),
):
    return get_pending_requests(db, current_user)


# ----------------------------
# GET SINGLE REQUEST
# ----------------------------
@router.get("/requests/{request_id}", response_model=LeaveRequestOut)
def get_leave_request_details(
    request_id: int,
    current_user: dict = Depends(get_current_user),
```

```
    db: Session = Depends(get_db),
):
    req = get_leave_request_by_id(db, request_id)

    if not req:
        raise HTTPException(status_code=404, detail="Leave request not found")

    role = current_user["role"]

    # Employee can only see own requests
    if role == "employee":
        if req.employee_id != current_user["id"]:
            raise HTTPException(status_code=403, detail="Not authorized")

    # HR can see everything → no restriction

    return req


# ----------------------------
# APPROVE REQUEST (HR ONLY)
# ----------------------------
@router.patch("/requests/{request_id}/approve", response_model=LeaveRequestOut)
def approve_request(
    request_id: int,
    payload: ApproveLeaveRequest,
    current_user: dict = Depends(require_roles(["hr"])),
    db: Session = Depends(get_db),
):
    req = get_leave_request_by_id(db, request_id)

    if not req:
        raise HTTPException(status_code=404, detail="Leave request not found")
    if req.employee_id == current_user["id"]:
        raise HTTPException(status_code=403, detail="Cannot approve your own request")

    return approve_leave_request(
        db,
        request_id,
        approved_by=current_user["id"],
        manager_comment=payload.manager_comment,
    )


# ----------------------------
# REJECT REQUEST (HR ONLY)
# ----------------------------
@router.patch("/requests/{request_id}/reject", response_model=LeaveRequestOut)
def reject_request(
    request_id: int,
    payload: RejectLeaveRequest,
    current_user: dict = Depends(require_roles(["hr"])),
    db: Session = Depends(get_db),
):
    req = get_leave_request_by_id(db, request_id)

    if not req:
        raise HTTPException(status_code=404, detail="Leave request not found")

    if req.employee_id == current_user["id"]:
        raise HTTPException(status_code=403, detail="Cannot reject your own request")

    return reject_leave_request(
        db,
        request_id,
        approved_by=current_user["id"],
        rejection_reason=payload.rejection_reason,
    )


# ----------------------------
# REQUEST INFO (HR ONLY)
```

```
# ----------------------------
@router.patch("/requests/{request_id}/request-info", response_model=LeaveRequestOut)
def request_info(
    request_id: int,
    payload: RequestInfoLeaveRequest,
    current_user: dict = Depends(require_roles(["hr"])),
    db: Session = Depends(get_db),
):
    req = get_leave_request_by_id(db, request_id)

    if not req:
        raise HTTPException(status_code=404, detail="Leave request not found")

    if req.employee_id == current_user["id"]:
        raise HTTPException(status_code=403, detail="Cannot request info for own request")

    return request_info_leave_request(
        db,
        request_id,
        approved_by=current_user["id"],
        manager_comment=payload.manager_comment,
    )


# ----------------------------
# RESUBMIT REQUEST (EMPLOYEE ONLY)
# ----------------------------
@router.patch("/requests/{request_id}/resubmit", response_model=LeaveRequestOut)
def resubmit_request(
    request_id: int,
    payload: ResubmitLeaveRequest,
    current_user: dict = Depends(require_roles(["employee", "hr"])),
    db: Session = Depends(get_db),
):
    try:
        return resubmit_leave_request(
            db=db,
            request_id=request_id,
            employee_id=current_user["id"],
            attachment_urls=payload.attachment_urls,
            reason=payload.reason,
        )
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))



# ----------------------------
# UPDATE STATUS (HR ONLY)
# ----------------------------
@router.patch("/requests/{request_id}/status", response_model=LeaveRequestOut)
def change_status(
    request_id: int,
    payload: LeaveStatusUpdate,
    current_user: dict = Depends(require_roles(["hr"])),
    db: Session = Depends(get_db),
):
    allowed = {"APPROVED", "REJECTED", "REQ_INFO", "PENDING"}

    if payload.status not in allowed:
        raise HTTPException(status_code=400, detail="Invalid status")

    req = get_leave_request_by_id(db, request_id)

    if not req:
        raise HTTPException(status_code=404, detail="Leave request not found")

    if req.employee_id == current_user["id"]:
        raise HTTPException(status_code=403, detail="Cannot change own request")

    return update_status(
        db,
```

```
        request_id,
        payload.status,
        approved_by=current_user["id"],
        rejection_reason=payload.rejection_reason,
    )


# ----------------------------
# LEAVE TYPES
# ----------------------------
@router.get("/types", response_model=list[LeaveTypeOut])
def list_leave_types(db: Session = Depends(get_db)):
    return get_leave_types(db)


@router.post("/types", response_model=LeaveTypeOut)
def add_leave_type(payload: LeaveTypeCreate, db: Session = Depends(get_db)):
    return create_leave_type(db, payload.name, payload.description)


# ----------------------------
# UPLOAD FILE
# ----------------------------
@router.post("/upload")
def upload_leave_attachment(file: UploadFile = File(...)):
    allowed_types = ["image/jpeg", "image/png", "application/pdf"]

    if file.content_type not in allowed_types:
        raise HTTPException(status_code=400, detail="Invalid file type")

    filename = f"{uuid4()}_{file.filename}"
    file_path = os.path.join(UPLOAD_DIR, filename)

    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    return {"file_url": f"/uploads/medical_docs/{filename}"}

# ----------------------------
# LEAVE HISTORY
# ----------------------------
@router.get("/history/me", response_model=list[LeaveRequestOut])
def get_my_leave_history_api(
    current_user: dict = Depends(require_roles(["employee", "hr"])),
    search: str | None = None,
    leave_type_id: int | None = None,
    status: str | None = None,
    sort_by: str = "newest",
    db: Session = Depends(get_db),
):
    return get_leave_history(
        db=db,
        user=current_user,
        search=search,
        leave_type_id=leave_type_id,
        status=status,
        sort_by=sort_by,
    )
```

File: backend\app\leave\schemas.py

```
from pydantic import BaseModel
from datetime import date
from typing import Optional, List

class LeaveRequestCreate(BaseModel):
    leave_type_id: int
    start_date: date
    end_date: date
```

```
    half_day: bool = False
    reason: Optional[str] = None
    attachment_urls: Optional[List[str]] = [] #if you have file upload later

class LeaveRequestOut(BaseModel):
    leave_request_id: int
    employee_id: int
    leave_type_id: int
    leave_type_name: Optional[str]= None
    start_date: date
    end_date: date
    total_days: float
    half_day: bool
    status: str
    reason: Optional[str]
    attachment_urls: Optional[List[str]] = []
    rejection_reason: Optional[str]
    manager_comment: Optional[str]= None #new added 29/03
    approved_by: Optional[int]
    approved_date: Optional[date]

    class Config:
        from_attributes = True

class LeaveStatusUpdate(BaseModel):
    status: str  # APPROVED / REJECTED / REQ_INFO / PENDING
    rejection_reason: Optional[str] = None  # only needed if REJECTED

class ApproveLeaveRequest(BaseModel):
    manager_comment: Optional[str] = None


class RejectLeaveRequest(BaseModel):
    rejection_reason: str


class RequestInfoLeaveRequest(BaseModel):
    manager_comment: str

class ResubmitLeaveRequest(BaseModel):
    attachment_urls: Optional[List[str]] = []
    reason: Optional[str] = None

class LeaveTypeCreate(BaseModel):
    name: str
    description: Optional[str] = None

class LeaveTypeOut(BaseModel):
    id: int
    name: str
    description: Optional[str] = None

    class Config:
        from_attributes = True
```

File: backend\app\leave\service.py

```
from sqlalchemy.orm import Session
from sqlalchemy.exc import IntegrityError
from datetime import date
from sqlalchemy import or_, cast, String

from app.leave.models import LeaveRequest, LeaveType
from app.leave.schemas import LeaveRequestCreate

from fastapi import HTTPException

TEAM_MEMBERS = {
    2: [1, 3],
```

```
    4: [1, 2, 3],
}


def get_team_members(manager_id: int) -> list[int]:
    return TEAM_MEMBERS.get(manager_id, [])


def calculate_total_days(start_date: date, end_date: date, half_day: bool) -> float:
    days = (end_date - start_date).days + 1

    if days < 1:
        raise ValueError("Invalid date range")

    if half_day:
        if start_date != end_date:
            raise ValueError("Half day leave must be for a single day only")
        return 0.5

    return float(days)


def leave_type_exists(db: Session, leave_type_id: int) -> bool:
    leave_type = db.query(LeaveType).filter(LeaveType.id == leave_type_id).first()
    return leave_type is not None


def has_overlapping_leave(db: Session, employee_id: int, start_date: date, end_date: date) -> bool:
    overlap = (
        db.query(LeaveRequest)
        .filter(
            LeaveRequest.employee_id == employee_id,
            LeaveRequest.start_date <= end_date,
            LeaveRequest.end_date >= start_date,
            LeaveRequest.status.in_(["PENDING", "APPROVED"])
        )
        .first()
    )
    return overlap is not None


def get_pending_requests(db: Session, user: dict):
    role = user.get("role", "").lower()

    query = (
        db.query(LeaveRequest, LeaveType.name.label("leave_type_name"))
        .join(LeaveType, LeaveRequest.leave_type_id == LeaveType.id)
        .filter(LeaveRequest.status == "PENDING")
    )

    # HR → see all except their own
    if role == "hr":
        query = query.filter(LeaveRequest.employee_id != user["id"])
    else:
        return []

    results = query.order_by(LeaveRequest.leave_request_id.desc()).all()

    output = []
    for leave, leave_type_name in results:
        leave.leave_type_name = leave_type_name
        output.append(leave)

    return output
def get_leave_history(
    db: Session,
    user: dict,
    search: str | None = None,
    leave_type_id: int | None = None,
    status: str | None = None,
    sort_by: str = "newest",
):
```

```
    query = db.query(LeaveRequest, LeaveType.name.label("leave_type_name")).join(
        LeaveType, LeaveRequest.leave_type_id == LeaveType.id
    )

    if user["role"] == "employee":
        query = query.filter(LeaveRequest.employee_id == user["id"])
    elif user["role"] == "hr":
        query = query.filter(LeaveRequest.employee_id == user["id"])
    elif user["role"] == "manager":
        team = get_team_members(user["id"])
        query = query.filter(
            or_(
                LeaveRequest.employee_id == user["id"],
                LeaveRequest.employee_id.in_(team),
            )
        )

    if search:
        search = search.strip()
        if search:
            query = query.filter(
                or_(
                    LeaveType.name.ilike(f"%{search}%"),
                    LeaveRequest.reason.ilike(f"%{search}%"),
                    LeaveRequest.status.ilike(f"%{search}%"),
                    cast(LeaveRequest.leave_request_id, String).ilike(f"%{search}%")
                )
            )

    if leave_type_id is not None:
        query = query.filter(LeaveRequest.leave_type_id == leave_type_id)

    if status:
        query = query.filter(LeaveRequest.status == status)

    if sort_by == "oldest":
        query = query.order_by(LeaveRequest.start_date.asc())
    else:
        query = query.order_by(LeaveRequest.start_date.desc())

    results = query.all()

    output = []
    for leave, leave_type_name in results:
        leave.leave_type_name = leave_type_name
        output.append(leave)

    return output


def create_leave(db: Session, employee_id: int, data: LeaveRequestCreate):
    if data.start_date > data.end_date:
        raise ValueError("start_date cannot be after end_date")

    if not leave_type_exists(db, data.leave_type_id):
        raise ValueError("Invalid leave_type_id")

    leave_type = db.query(LeaveType).filter(LeaveType.id == data.leave_type_id).first()

    if leave_type and "medical" in leave_type.name.lower() and not data.attachment_urls:
        raise ValueError("Medical leave requires at least one supporting document")

    if has_overlapping_leave(db, employee_id, data.start_date, data.end_date):
        raise HTTPException(status_code=400,detail="You already have a leave request for these dates")

    total_days = calculate_total_days(data.start_date, data.end_date, data.half_day)

    req = LeaveRequest(
        employee_id=employee_id,
        leave_type_id=data.leave_type_id,
        start_date=data.start_date,
        end_date=data.end_date,
```

```
        total_days=total_days,
        half_day=data.half_day,
        status="PENDING",
        reason=data.reason,
        attachment_urls=data.attachment_urls,
    )

    try:
        db.add(req)
        db.commit()
        db.refresh(req)
        return req
    except IntegrityError:
        db.rollback()
        raise ValueError("Could not create leave request")


def my_requests(db: Session, employee_id: int):
    return (
        db.query(LeaveRequest)
        .filter(LeaveRequest.employee_id == employee_id)
        .order_by(LeaveRequest.leave_request_id.desc())
        .all()
    )


def pending_requests(db: Session):
    return (
        db.query(LeaveRequest)
        .filter(LeaveRequest.status == "PENDING")
        .order_by(LeaveRequest.leave_request_id.desc())
        .all()
    )


def get_leave_request_by_id(db: Session, request_id: int):
    result = (
        db.query(LeaveRequest, LeaveType.name.label("leave_type_name"))
        .join(LeaveType, LeaveRequest.leave_type_id == LeaveType.id)
        .filter(LeaveRequest.leave_request_id == request_id)
        .first()
    )

    if not result:
        return None

    leave, leave_type_name = result
    leave.leave_type_name = leave_type_name
    return leave


def approve_leave_request(
    db: Session,
    request_id: int,
    approved_by: int,
    manager_comment: str | None = None,
):
    req = db.query(LeaveRequest).filter(LeaveRequest.leave_request_id == request_id).first()

    if not req:
        return None

    if req.status != "PENDING":
        raise ValueError("Only pending leave requests can be approved")

    req.status = "APPROVED"
    req.approved_by = approved_by
    req.approved_date = date.today()
    req.rejection_reason = None
    req.manager_comment = manager_comment

    db.commit()
```

```
    db.refresh(req)
    return req


def reject_leave_request(
    db: Session,
    request_id: int,
    approved_by: int,
    rejection_reason: str,
):
    req = db.query(LeaveRequest).filter(LeaveRequest.leave_request_id == request_id).first()

    if not req:
        return None

    if req.status != "PENDING":
        raise ValueError("Only pending leave requests can be rejected")

    if not rejection_reason or not rejection_reason.strip():
        raise ValueError("rejection_reason is required")

    req.status = "REJECTED"
    req.approved_by = approved_by
    req.approved_date = date.today()
    req.rejection_reason = rejection_reason.strip()
    req.manager_comment = None

    db.commit()
    db.refresh(req)
    return req


def request_info_leave_request(
    db: Session,
    request_id: int,
    approved_by: int,
    manager_comment: str,
):
    req = db.query(LeaveRequest).filter(LeaveRequest.leave_request_id == request_id).first()

    if not req:
        return None

    if req.status != "PENDING":
        raise ValueError("Only pending leave requests can be marked as request info")

    if not manager_comment or not manager_comment.strip():
        raise ValueError("manager_comment is required")

    req.status = "REQ_INFO"
    req.approved_by = approved_by
    req.manager_comment = manager_comment.strip()

    db.commit()
    db.refresh(req)
    return req


def resubmit_leave_request(
    db: Session,
    request_id: int,
    employee_id: int,
    attachment_urls: list[str] | None,
    reason: str | None,
):
    req = db.query(LeaveRequest).filter(
        LeaveRequest.leave_request_id == request_id,
        LeaveRequest.employee_id == employee_id
    ).first()

    if not req:
        raise ValueError("Leave request not found or not authorized")
```

```
    if req.status != "REQ_INFO":
        raise ValueError("Only requests needing info can be resubmitted")

    # Append new attachments to existing ones
    if attachment_urls:
        existing_urls = req.attachment_urls if isinstance(req.attachment_urls, list) else []
        req.attachment_urls = existing_urls + attachment_urls

    if reason is not None and reason.strip():
        req.reason = reason.strip()

    req.status = "PENDING"
    req.manager_comment = None  # Clear HR comment since they are replying

    db.commit()
    db.refresh(req)
    return req


def update_status(
    db: Session,
    request_id: int,
    status: str,
    approved_by: int | None = None,
    rejection_reason: str | None = None,
):
    req = db.query(LeaveRequest).filter(LeaveRequest.leave_request_id == request_id).first()
    if not req:
        return None

    if status == "REJECTED" and (rejection_reason is None or rejection_reason.strip() == ""):
        raise ValueError("rejection_reason is required when rejecting a leave request")

    req.status = status

    if status == "APPROVED":
        req.approved_by = approved_by
        req.approved_date = date.today()
        req.rejection_reason = None

    elif status == "REJECTED":
        req.approved_by = approved_by
        req.approved_date = date.today()
        req.rejection_reason = rejection_reason

    db.commit()
    db.refresh(req)
    return req
def delete_leave_request(db: Session, request_id: int, employee_id: int):
    req = db.query(LeaveRequest).filter(
        LeaveRequest.leave_request_id == request_id,
        LeaveRequest.employee_id == employee_id
    ).first()
    if not req:
        raise ValueError("Leave request not found or not authorized")
    if req.status != "PENDING":
        raise ValueError("Only PENDING leave requests can be deleted")
    db.delete(req)
    db.commit()
    return True


def update_leave_request(db: Session, request_id: int, employee_id: int, payload: LeaveRequestCreate):
    req = db.query(LeaveRequest).filter(
        LeaveRequest.leave_request_id == request_id,
        LeaveRequest.employee_id == employee_id
    ).first()

    if not req:
        raise ValueError("Leave request not found or not authorized")
    if req.status != "PENDING":
```

```
        raise ValueError("Only PENDING leave requests can be updated")

    if not leave_type_exists(db, payload.leave_type_id):
        raise ValueError("Invalid leave type")

    # Check for overlapping leave excluding this current request
    overlap = (
        db.query(LeaveRequest)
        .filter(
            LeaveRequest.employee_id == employee_id,
            LeaveRequest.leave_request_id != request_id,
            LeaveRequest.start_date <= payload.end_date,
            LeaveRequest.end_date >= payload.start_date,
            LeaveRequest.status.in_(["PENDING", "APPROVED"])
        )
        .first()
    )
    if overlap:
        raise ValueError("You already have an overlapping leave request for these dates")

    total_days = calculate_total_days(payload.start_date, payload.end_date, payload.half_day)

    req.leave_type_id = payload.leave_type_id
    req.start_date = payload.start_date
    req.end_date = payload.end_date
    req.half_day = payload.half_day
    req.reason = payload.reason
    if payload.attachment_urls is not None:
        req.attachment_urls = payload.attachment_urls
    req.total_days = total_days

    db.commit()
    db.refresh(req)
    return req


def get_leave_types(db: Session):
    return db.query(LeaveType).order_by(LeaveType.id.asc()).all()


def create_leave_type(db: Session, name: str, description: str | None = None):
    existing = db.query(LeaveType).filter(LeaveType.name == name).first()
    if existing:
        raise ValueError("Leave type already exists")

    leave_type = LeaveType(name=name, description=description)
    db.add(leave_type)
    db.commit()
    db.refresh(leave_type)
    return leave_type


def get_my_leave_history(
    db: Session,
    employee_id: int,
    search: str | None = None,
    leave_type_id: int | None = None,
    status: str | None = None,
    sort_by: str = "newest",
):
    query = db.query(LeaveRequest, LeaveType.name.label("leave_type_name")).join(
        LeaveType, LeaveRequest.leave_type_id == LeaveType.id
    ).filter(
        LeaveRequest.employee_id == employee_id
    )

    if search:
        search = search.strip()
        if search:
            query = query.filter(
                or_(
                    LeaveType.name.ilike(f"%{search}%"),
```

```
                    LeaveRequest.reason.ilike(f"%{search}%"),
                    LeaveRequest.status.ilike(f"%{search}%"),
                    cast(LeaveRequest.leave_request_id, String).ilike(f"%{search}%")
                )
            )

    if leave_type_id is not None:
        query = query.filter(LeaveRequest.leave_type_id == leave_type_id)

    if status:
        query = query.filter(LeaveRequest.status == status)

    if sort_by == "oldest":
        query = query.order_by(LeaveRequest.start_date.asc())
    else:
        query = query.order_by(LeaveRequest.start_date.desc())

    results = query.all()

    output = []
    for leave, leave_type_name in results:
        leave.leave_type_name = leave_type_name
        output.append(leave)

    return output
```

File: backend\app\leave__init__.py

File: backend\app\recruitment\models.py

File: backend\app\recruitment\router.py

File: backend\app\recruitment\schemas.py

File: backend\app\recruitment\service.py

File: backend\app\recruitment__init__.py

File: backend\app\reports\router.py

```
from datetime import date
from fastapi import APIRouter, Depends
from fastapi.responses import StreamingResponse
from sqlalchemy.orm import Session
from io import BytesIO

from app.database.database import get_db
from app.reports.schemas import LeaveReportResponse
```

```
from app.reports.service import get_leave_report, generate_leave_report_csv

from fastapi.responses import StreamingResponse
from app.reports.service import generate_leave_report_pdf

router = APIRouter(prefix="/reports", tags=["Reports"])


@router.get("/leave", response_model=LeaveReportResponse)
def preview_leave_report(
    employee_id: int | None = None,
    leave_type_id: int | None = None,
    status: str | None = None,
    start_date: date | None = None,
    end_date: date | None = None,
    db: Session = Depends(get_db),
):
    return get_leave_report(
        db=db,
        employee_id=employee_id,
        leave_type_id=leave_type_id,
        status=status,
        start_date=start_date,
        end_date=end_date,
    )


@router.get("/leave/export/csv")
def export_leave_report_csv(
    employee_id: int | None = None,
    leave_type_id: int | None = None,
    status: str | None = None,
    start_date: date | None = None,
    end_date: date | None = None,
    db: Session = Depends(get_db),
):
    report_data = get_leave_report(
        db=db,
        employee_id=employee_id,
        leave_type_id=leave_type_id,
        status=status,
        start_date=start_date,
        end_date=end_date,
    )

    csv_content = generate_leave_report_csv(report_data)

    return StreamingResponse(
        iter([csv_content]),
        media_type="text/csv",
        headers={
            "Content-Disposition": "attachment; filename=leave_report.csv"
        },
    )

@router.get("/leave/pdf")
def download_leave_pdf(
    employee_id: int | None = None,
    leave_type_id: int | None = None,
    status: str | None = None,
    start_date: date | None = None,
    end_date: date | None = None,
    db: Session = Depends(get_db),
):
    data = get_leave_report(
        db=db,
        employee_id=employee_id,
        leave_type_id=leave_type_id,
        status=status,
        start_date=start_date,
        end_date=end_date,
    )
```

```
    pdf_buffer = generate_leave_report_pdf(data)

    return StreamingResponse(
        pdf_buffer,
        media_type="application/pdf",
        headers={"Content-Disposition": "attachment; filename=leave_report.pdf"},
    )
```

File: backend\app\reports\schemas.py

```
from pydantic import BaseModel
from datetime import date
from typing import Optional, List


class LeaveReportRow(BaseModel):
    leave_request_id: int
    employee_id: int
    employee_code: Optional[str] = None
    employee_name: Optional[str] = None
    department: Optional[str] = None
    designation: Optional[str] = None
    joined_date: Optional[date] = None
    employee_status: Optional[str] = None
    leave_type_id: int
    leave_type_name: Optional[str] = None
    start_date: date
    end_date: date
    total_days: float
    half_day: bool
    status: str
    reason: Optional[str] = None
    approved_by: Optional[int] = None
    approved_date: Optional[date] = None
    rejection_reason: Optional[str] = None
    manager_comment: Optional[str] = None


class LeaveReportSummary(BaseModel):
    total_requests: int
    total_leave_days: float
    approved_requests: int
    pending_requests: int
    rejected_requests: int
    req_info_requests: int


class LeaveReportResponse(BaseModel):
    summary: LeaveReportSummary
    records: List[LeaveReportRow]
```

File: backend\app\reports\service.py

```
from io import StringIO
import csv
from datetime import date
from sqlalchemy.orm import Session
#from sqlalchemy import func
from reportlab.lib.pagesizes import letter
from reportlab.platypus import SimpleDocTemplate, Table, TableStyle, Paragraph
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet
from io import BytesIO

from app.leave.models import LeaveRequest, LeaveType
from app.employees.models import Employee
```

```
def build_leave_report_query(
    db: Session,
    employee_id: int | None = None,
    leave_type_id: int | None = None,
    status: str | None = None,
    start_date: date | None = None,
    end_date: date | None = None,
):
    query = (
        db.query(
            LeaveRequest,
            LeaveType.name.label("leave_type_name"),
            Employee.employee_id.label("employee_code"),
            Employee.first_name,
            Employee.last_name,
            Employee.department,
            Employee.designation,
            Employee.joined_date,
            Employee.status.label("employee_status"),
            )
        .join(LeaveType, LeaveRequest.leave_type_id == LeaveType.id)
        .outerjoin(Employee, LeaveRequest.employee_id == Employee.id)
    )

    if employee_id is not None:
        query = query.filter(LeaveRequest.employee_id == employee_id)

    if leave_type_id is not None:
        query = query.filter(LeaveRequest.leave_type_id == leave_type_id)

    if status:
        query = query.filter(LeaveRequest.status == status)

    if start_date is not None and end_date is not None:
        query = query.filter(
            LeaveRequest.start_date <= end_date,
            LeaveRequest.end_date >= start_date,)

    elif start_date is not None:
        query = query.filter(LeaveRequest.end_date >= start_date)
    elif end_date is not None:
        query = query.filter(LeaveRequest.start_date <= end_date)


    return query.order_by(LeaveRequest.start_date.desc())


def get_leave_report(
    db: Session,
    employee_id: int | None = None,
    leave_type_id: int | None = None,
    status: str | None = None,
    start_date: date | None = None,
    end_date: date | None = None,
):
    results = build_leave_report_query(
        db=db,
        employee_id=employee_id,
        leave_type_id=leave_type_id,
        status=status,
        start_date=start_date,
        end_date=end_date,
    ).all()

    records = []
    total_leave_days = 0.0
    approved_requests = 0
    pending_requests = 0
    rejected_requests = 0
    req_info_requests = 0
```

```
    for (leave,
         leave_type_name,
           employee_code,
           first_name,
           last_name,
           department,
           designation,
           joined_date,
           employee_status,
    ) in results:
        total_leave_days += float(leave.total_days or 0)

        if leave.status == "APPROVED":
            approved_requests += 1
        elif leave.status == "PENDING":
            pending_requests += 1
        elif leave.status == "REJECTED":
            rejected_requests += 1
        elif leave.status == "REQ_INFO":
            req_info_requests += 1

        records.append({
            "leave_request_id": leave.leave_request_id,
            "employee_id": leave.employee_id,
            "employee_code": employee_code,
            "employee_name": f"{first_name} {last_name}".strip() or None,
            "department": department,
            "designation": designation,
            "joined_date": joined_date,
            "employee_status": employee_status.value if employee_status else None,
            "leave_type_id": leave.leave_type_id,
            "leave_type_name": leave_type_name,
            "start_date": leave.start_date,
            "end_date": leave.end_date,
            "total_days": leave.total_days,
            "half_day": leave.half_day,
            "status": leave.status,
            "reason": leave.reason,
            "approved_by": leave.approved_by,
            "approved_date": leave.approved_date,
            "rejection_reason": leave.rejection_reason,
            "manager_comment": leave.manager_comment,
        })

    summary = {
        "total_requests": len(records),
        "total_leave_days": total_leave_days,
        "approved_requests": approved_requests,
        "pending_requests": pending_requests,
        "rejected_requests": rejected_requests,
        "req_info_requests": req_info_requests,
    }

    return {
        "summary": summary,
        "records": records,
    }


def generate_leave_report_csv(report_data: dict) -> str:
    output = StringIO()
    writer = csv.writer(output)

    summary = report_data["summary"]
    records = report_data["records"]

    writer.writerow(["LEAVE REPORT SUMMARY"])
    writer.writerow(["Total Requests", summary["total_requests"]])
    writer.writerow(["Total Leave Days", summary["total_leave_days"]])
    writer.writerow(["Approved Requests", summary["approved_requests"]])
    writer.writerow(["Pending Requests", summary["pending_requests"]])
```

```
    writer.writerow(["Rejected Requests", summary["rejected_requests"]])
    writer.writerow(["Request Info Requests", summary["req_info_requests"]])
    writer.writerow([])

    writer.writerow([
        "Request ID",
        "Employee DB ID",
        "Employee Code",
        "Employee Name",
        "Department",
        "Designation",
        "Joined Date",
        "Employee Status",
        "Leave Type ID",
        "Leave Type Name",
        "Start Date",
        "End Date",
        "Total Days",
        "Half Day",
        "Status",
        "Reason",
        "Approved By",
        "Approved Date",
        "Rejection Reason",
        "Manager Comment",
    ])

    for row in records:
        writer.writerow([
            row["leave_request_id"],
            row["employee_id"],
            row.get("employee_code"),
            row.get("employee_name"),
            row.get("department"),
            row.get("designation"),
            row.get("joined_date"),
            row.get("employee_status"),
            row["leave_type_id"],
            row["leave_type_name"],
            row["start_date"],
            row["end_date"],
            row["total_days"],
            row["half_day"],
            row["status"],
            row["reason"],
            row["approved_by"],
            row["approved_date"],
            row["rejection_reason"],
            row["manager_comment"],
        ])

    return output.getvalue()

def generate_leave_report_pdf(data):
    buffer = BytesIO()

    doc = SimpleDocTemplate(buffer, pagesize=letter)
    elements = []

    styles = getSampleStyleSheet()

    records = data.get("records", [])
    has_employee = len(records) > 0
    employee_name = records[0].get("employee_name", "N/A") if has_employee else "N/A"
    employee_dept = records[0].get("department", "N/A") if has_employee else "N/A"
    employee_code = records[0].get("employee_code", "N/A") if has_employee else "N/A"

    # Title
    title = f"Employee Detailed Report - {employee_name}" if has_employee else "Leave Report Summary"
    elements.append(Paragraph(title, styles["Title"]))

    if has_employee:
```

```
        elements.append(Paragraph(f"<b>Employee Name:</b> {employee_name}", styles["Normal"]))
        elements.append(Paragraph(f"<b>Employee Code:</b> {employee_code}", styles["Normal"]))
        elements.append(Paragraph(f"<b>Department:</b> {employee_dept}", styles["Normal"]))
        elements.append(Paragraph("<br/>", styles["Normal"]))

    # Summary
    summary = data["summary"]
    elements.append(Paragraph(f"Total Requests: {summary['total_requests']}", styles["Normal"]))
    elements.append(Paragraph(f"Total Leave Days: {summary['total_leave_days']}", styles["Normal"]))
    elements.append(Paragraph(f"Approved: {summary['approved_requests']}", styles["Normal"]))
    elements.append(Paragraph(f"Pending: {summary['pending_requests']}", styles["Normal"]))
    elements.append(Paragraph("<br/>", styles["Normal"]))

    # Table
    table_data = [
        ["Employee", "Department", "Leave Type", "Days", "Status"]
    ]

    for r in data["records"]:
        table_data.append([
            r["employee_name"],
            r["department"],
            r["leave_type_name"],
            str(r["total_days"]),
            r["status"],
        ])

    table = Table(table_data)

    table.setStyle(TableStyle([
        ("BACKGROUND", (0, 0), (-1, 0), colors.grey),
        ("TEXTCOLOR", (0, 0), (-1, 0), colors.whitesmoke),
        ("GRID", (0, 0), (-1, -1), 1, colors.black),
    ]))

    elements.append(table)

    doc.build(elements)
    buffer.seek(0)
    return buffer
```

File: backend\app\reports__init__..py

File: frontend\next-env.d.ts

```
/// <reference types="next" />
/// <reference types="next/image-types/global" />
import "./.next/dev/types/routes.d.ts";

// NOTE: This file should not be edited
// see https://nextjs.org/docs/app/api-reference/config/typescript for more information.
```

File: frontend\next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
};

export default nextConfig;
```

File: frontend\package.json

```
{
  "name": "frontend",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "set NEXT_DISABLE_TURBOPACK=1 && next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "framer-motion": "^12.38.0",
    "lucide-react": "^0.564.0",
    "next": "^16.2.4",
    "react": "19.2.3",
    "react-dom": "19.2.3",
    "recharts": "^3.8.1"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.1.1",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

File: frontend\README.md

````
This is a [Next.js](https://nextjs.org) project bootstrapped with
[`create-next-app`](https://nextjs.org/docs/app/api-reference/cli/create-next-app).

## Getting Started

First, run the development server:

```bash
npm run dev
# or
yarn dev
# or
pnpm dev
# or
bun dev
```

Open [http://localhost:3000](http://localhost:3000) with your browser to see the result.

You can start editing the page by modifying `app/page.tsx`. The page auto-updates as you edit the file.

This project uses [`next/font`](https://nextjs.org/docs/app/building-your-application/optimizing/fonts) to
automatically optimize and load [Geist](https://vercel.com/font), a new font family for Vercel.

## Learn More

To learn more about Next.js, take a look at the following resources:

- [Next.js Documentation](https://nextjs.org/docs) - learn about Next.js features and API.
- [Learn Next.js](https://nextjs.org/learn) - an interactive Next.js tutorial.

You can check out [the Next.js GitHub repository](https://github.com/vercel/next.js) - your feedback and
contributions are welcome!
````

```
## Deploy on Vercel

The easiest way to deploy your Next.js app is to use the [Vercel
Platform](https://vercel.com/new?utm_medium=default-template&filter=next.js&utm_source=create-next-app&utm_campa
 ign=create-next-app-readme) from the creators of Next.js.

Check out our [Next.js deployment documentation](https://nextjs.org/docs/app/building-your-application/deploying)
  for more details.
```

File: frontend\tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./*"],
      "@/components/*": ["./components/*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

File: frontend\app\globals.css

```
@import "tailwindcss";

:root {
  --background: #f8f9fb;
  --foreground: #171717;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}

html,
body {
  margin: 0;
  padding: 0;
```

```
  background: var(--background);
  color: var(--foreground);
  font-family: Arial, Helvetica, sans-serif;
}

* {
  box-sizing: border-box;
}
```

File: frontend\app\layout.tsx

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "Create Next App",
  description: "Generated by create next app",
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="en">
      <body
        className={`${geistSans.variable} ${geistMono.variable} antialiased`}
      >
        {/* layout header removed to avoid duplicate top bars; page components render TopBar instead */}
        {children}
      </body>
    </html>
  );
}
```

File: frontend\app\page.tsx

```
"use client";

import Link from "next/link";
import { useEffect, useState } from "react";

export default function Home() {
  const [role, setRole] = useState("employee");
  const [userId, setUserId] = useState("1");

  // persist role
  useEffect(() => {
    const savedRole = localStorage.getItem("role");
    const savedUserId = localStorage.getItem("userId");

    if (savedRole) setRole(savedRole);
    if (savedUserId) setUserId(savedUserId);
  }, []);
```

```
  useEffect(() => {
    localStorage.setItem("role", role);
    localStorage.setItem("userId", userId);
  }, [role, userId]);


  return (
    <div className="min-h-screen bg-gray-50 flex flex-col items-center justify-center gap-6">

      {/* ================= ROLE CONTROL PANEL ================= */}
      <div className="bg-white border rounded-lg shadow-sm p-6 w-full max-w-xl">
        <h2 className="text-xl font-semibold mb-4">HRMS Role Control (Dev Mode)</h2>

        {/* Role Selector */}
        <div className="mb-4">
          <label className="text-sm text-gray-600">Select Role</label>
          <select
            className="w-full border p-2 rounded mt-1"
            value={role}
            onChange={(e) => setRole(e.target.value)}
          >
            <option value="employee">Employee</option>
            <option value="hr">HR</option>
          </select>
        </div>

        {/* User ID */}
        <div className="mb-4">
          <label className="text-sm text-gray-600">User ID</label>
          <input
            className="w-full border p-2 rounded mt-1"
            value={userId}
            onChange={(e) => setUserId(e.target.value)}
          />
        </div>

        {/* Debug Info */}
        <div className="text-sm text-gray-500">
          Role: <b>{role}</b> | User ID: <b>{userId}</b>
        </div>
      </div>

      {/* ================= MAIN DASHBOARD ================= */}
      <div className="max-w-2xl w-full p-8">
        <div className="bg-white border border-gray-100 rounded-lg shadow-sm p-8 text-center">

          <h1 className="text-2xl font-semibold mb-2">
            Welcome to HRMS ({role.toUpperCase()})
          </h1>

          <p className="text-gray-600 mb-6">
            Quick access to common tasks.
          </p>

          <div className="flex items-center justify-center gap-4">

            <Link
              href="/apply-leave"
              className="inline-block bg-orange-500 text-white px-6 py-3 rounded-lg hover:bg-orange-600
transition"
              onClick={() => {
                localStorage.setItem("role", role);
                localStorage.setItem("userId", userId);
              }}
            >
              Request Leave
            </Link>

            <Link
              href="/leave-history"
              className="inline-block border border-gray-200 px-6 py-3 rounded-lg text-gray-700
hover:bg-gray-100 transition"
```

```
              onClick={() => {
                localStorage.setItem("role", role);
                localStorage.setItem("userId", userId);
              }}
            >
              Leave History
            </Link>

            {
              role !== "employee" && (
                <Link href="/approval">Approval Panel</Link>
              )
            }

          </div>

          {/* ROLE INFO PANEL */}
          <div className="mt-6 text-sm text-gray-500">
            <p>Employee → can apply leave + view own history</p>
            <p>Manager → can approve team leave</p>
            <p>HR → full access</p>
          </div>

        </div>
      </div>
    </div>
  );
}
```

File: frontend\app\apply-leave\page.tsx

```
'use client';

import React, { useState } from 'react';

import Sidebar from '@/components/Sidebar';
import TopBar from '@/components/TopBar';
import LeaveTabs from '@/components/LeaveTabs';
import LeaveBalanceCards from '@/components/LeaveBalanceCards';
import ApplyLeaveForm from '@/components/ApplyLeaveForm';

export default function ApplyLeavePage() {
  // mock balance data
  const [balances] = useState({
    Annual: 8,
    Casual: 2,
    Medical: 5,
  });

  return (
    <div className="min-h-screen bg-gray-50">
      {/* fixed sidebar and header live outside the flow; main content gets offsets */}
      <Sidebar />
      <TopBar />

      <div className="ml-64 pt-16">
        <main className="px-10 pt-4 pb-8 min-h-[calc(100vh-4rem)] overflow-auto">
          <LeaveTabs />
          <LeaveBalanceCards balances={balances} />

          <ApplyLeaveForm balances={balances} />
        </main>
      </div>
    </div>
  );
}
```

File: frontend\app\approval\page.tsx

```
'use client';

import React, { useEffect, useMemo, useState, useCallback } from 'react';
import { useRouter } from 'next/navigation';
import Sidebar from '../../components/Sidebar';
import TopBar from '../../components/TopBar';
import LeaveTabs from '../../components/LeaveTabs';
import ApprovalRequestCard, {
  ApprovalRequest,
} from '../../components/ApprovalRequestCard';
import ApprovalReviewModal, {
  ModalMode,
} from '../../components/ApprovalReviewModal';
import { Search, ChevronDown, CheckCircle2, XCircle } from 'lucide-react';
import { API_BASE_URL, getAuthHeaders } from '../lib/api';

type BackendLeaveRequest = {
  leave_request_id: number;
  employee_id: number;
  leave_type_id: number;
  leave_type_name?: string | null;
  start_date: string;
  end_date: string;
  total_days: number;
  half_day: boolean;
  status: string;
  reason?: string | null;
  attachment_urls?: string[] | null;
  rejection_reason?: string | null;
  manager_comment?: string | null;
  approved_by?: number | null;
  approved_date?: string | null;
};

type PendingRequestsResponse =
  | BackendLeaveRequest[]
  | {
      items?: BackendLeaveRequest[];
      data?: BackendLeaveRequest[];
      results?: BackendLeaveRequest[];
    };

function formatDate(dateString: string) {
  if (!dateString) return '--';

  const normalized = /^\d{4}-\d{2}-\d{2}$/.test(dateString)
    ? `${dateString}T00:00:00`
    : dateString;

  const date = new Date(normalized);

  if (Number.isNaN(date.getTime())) return '--';

  return date.toLocaleDateString('en-US', {
    month: 'short',
    day: '2-digit',
  });
}

function mapBackendToFrontend(item: BackendLeaveRequest): ApprovalRequest {
  const urls = Array.isArray(item.attachment_urls) ? item.attachment_urls : [];
  return {
    id: item.leave_request_id,
    employeeName: `Employee ${item.employee_id}`,
    employeeCode: `EMP_${String(item.employee_id).padStart(2, '0')}`,
    department: 'Department',
    role: 'Employee',
    leaveType: item.leave_type_name || `Leave Type ${item.leave_type_id}`,
    startDate: formatDate(item.start_date),
```

```
    endDate: formatDate(item.end_date),
    durationText: item.half_day ? '0.5 Days' : `${item.total_days} Days`,
    hasAttachment: urls.length > 0,
    attachmentUrls: urls,
    appliedOn: item.start_date,
    reason: item.reason || 'No reason provided',
    balances: {
      annual: '--',
      casual: '--',
    },
  };
}

function extractRequestList(payload: PendingRequestsResponse): BackendLeaveRequest[] {
  if (Array.isArray(payload)) return payload;
  if (Array.isArray(payload.items)) return payload.items;
  if (Array.isArray(payload.data)) return payload.data;
  if (Array.isArray(payload.results)) return payload.results;
  return [];
}

async function parseErrorResponse(response: Response): Promise<string> {
  const fallback = `Request failed (${response.status} ${response.statusText})`;

  try {
    const text = await response.text();
    if (!text) return fallback;

    try {
      const json = JSON.parse(text);
      if (typeof json?.detail === 'string') return json.detail;
      if (typeof json?.message === 'string') return json.message;
      return text;
    } catch {
      return text;
    }
  } catch {
    return fallback;
  }
}

export default function ApprovalPage() {
  const [requests, setRequests] = useState<ApprovalRequest[]>([]);
  const [searchTerm, setSearchTerm] = useState('');
  const [leaveTypeFilter, setLeaveTypeFilter] = useState('All type');
  const [departmentFilter, setDepartmentFilter] = useState('All Departments');
  const [sortBy, setSortBy] = useState('sort by');

  const [selectedRequest, setSelectedRequest] = useState<ApprovalRequest | null>(null);
  const [modalMode, setModalMode] = useState<ModalMode>('review');

  const [actionMessage, setActionMessage] = useState('');
  const [messageType, setMessageType] = useState<'success' | 'error' | ''>('');
  const [loading, setLoading] = useState(true);
  const [actionLoading, setActionLoading] = useState(false);
  const [role, setRole] = useState<string | null | undefined>(undefined);

  const router = useRouter();

  useEffect(() => {
    const storedRole = localStorage.getItem('role');
    setRole(storedRole);
  }, []);

  useEffect(() => {
    if (role === undefined) return;

    if (role !== 'hr') {
      router.replace('/apply-leave');
    }
  }, [role, router]);
```

```
const loadPendingRequests = useCallback(async () => {
  try {
    setLoading(true);
    setActionMessage('');
    setMessageType('');

    const response = await fetch(`${API_BASE_URL}/leave/requests/pending`, {
      method: 'GET',
      headers: getAuthHeaders(),
      cache: 'no-store',
    });

    if (!response.ok) {
      const errorMessage = await parseErrorResponse(response);
      throw new Error(errorMessage || 'Failed to load pending leave requests');
    }

    const payload: PendingRequestsResponse = await response.json();
    const items = extractRequestList(payload);
    setRequests(items.map(mapBackendToFrontend));
  } catch (error) {
    console.error('loadPendingRequests error:', error);
    setRequests([]);
    setActionMessage(
      error instanceof Error
        ? error.message
        : 'Failed to load pending leave requests.'
    );
    setMessageType('error');
  } finally {
    setLoading(false);
  }
}, []);

useEffect(() => {
  if (role === 'hr') {
    loadPendingRequests();
  }
}, [role, loadPendingRequests]);

useEffect(() => {
  if (!actionMessage) return;

  const timer = setTimeout(() => {
    setActionMessage('');
    setMessageType('');
  }, 3000);

  return () => clearTimeout(timer);
}, [actionMessage]);

const filteredRequests = useMemo(() => {
  let data = [...requests];

  if (searchTerm.trim()) {
    const q = searchTerm.toLowerCase();
    data = data.filter(
      (item) =>
        item.employeeName.toLowerCase().includes(q) ||
        item.employeeCode.toLowerCase().includes(q) ||
        item.department.toLowerCase().includes(q) ||
        item.role.toLowerCase().includes(q) ||
        item.leaveType.toLowerCase().includes(q)
    );
  }

  if (leaveTypeFilter !== 'All type') {
    data = data.filter((item) => item.leaveType === leaveTypeFilter);
  }

  if (departmentFilter !== 'All Departments') {
    data = data.filter((item) => item.department === departmentFilter);
```

```
  }

  if (sortBy === 'Name') {
    data.sort((a, b) => a.employeeName.localeCompare(b.employeeName));
  }

  return data;
}, [requests, searchTerm, leaveTypeFilter, departmentFilter, sortBy]);

const openReview = async (request: ApprovalRequest) => {
  try {
    const response = await fetch(`${API_BASE_URL}/leave/requests/${request.id}`, {
      method: 'GET',
      headers: getAuthHeaders(),
      cache: 'no-store',
    });

    if (!response.ok) {
      const errorMessage = await parseErrorResponse(response);
      throw new Error(errorMessage || 'Failed to load leave request details');
    }

    const data: BackendLeaveRequest = await response.json();
    setSelectedRequest(mapBackendToFrontend(data));
    setModalMode('review');
  } catch (error) {
    console.error('openReview error:', error);
    setActionMessage(
      error instanceof Error
        ? error.message
        : 'Failed to load leave request details.'
    );
    setMessageType('error');
  }
};

const closeModal = () => {
  setSelectedRequest(null);
  setModalMode('review');
};

const handleApprove = async (comment: string) => {
  if (!selectedRequest) return;

  try {
    setActionLoading(true);

    const response = await fetch(
      `${API_BASE_URL}/leave/requests/${selectedRequest.id}/approve`,
      {
        method: 'PATCH',
        headers: {
          ...getAuthHeaders(),
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          manager_comment: comment || null,
        }),
      }
    );

    if (!response.ok) {
      const errorMessage = await parseErrorResponse(response);
      throw new Error(errorMessage || 'Failed to approve leave request');
    }

    setActionMessage('Leave request approved successfully.');
    setMessageType('success');
    closeModal();
    await loadPendingRequests();
  } catch (error) {
    console.error('handleApprove error:', error);
```

```
    setActionMessage(
      error instanceof Error ? error.message : 'Failed to approve leave request.'
    );
    setMessageType('error');
  } finally {
    setActionLoading(false);
  }
};

const handleReject = async (reason: string) => {
  if (!selectedRequest) return;

  try {
    setActionLoading(true);
    const response = await fetch(
      `${API_BASE_URL}/leave/requests/${selectedRequest.id}/reject`,
      {
        method: 'PATCH',
        headers: {
          ...getAuthHeaders(),
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          rejection_reason: reason,
        }),
      }
    );

    if (!response.ok) {
      const errorMessage = await parseErrorResponse(response);
      throw new Error(errorMessage || 'Failed to reject leave request');
    }

    setActionMessage('Leave request rejected successfully.');
    setMessageType('success');
    closeModal();
    await loadPendingRequests();
  } catch (error) {
    console.error('handleReject error:', error);
    setActionMessage(
      error instanceof Error ? error.message : 'Failed to reject leave request.'
    );
    setMessageType('error');
  } finally {
    setActionLoading(false);
  }
};

const handleRequestInfo = async (message: string) => {
  if (!selectedRequest) return;

  try {
    setActionLoading(true);

    const response = await fetch(
      `${API_BASE_URL}/leave/requests/${selectedRequest.id}/request-info`,
      {
        method: 'PATCH',
        headers: {
          ...getAuthHeaders(),
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({
          manager_comment: message,
        }),
      }
    );

    if (!response.ok) {
      const errorMessage = await parseErrorResponse(response);
      throw new Error(errorMessage || 'Failed to send info request');
    }
```

```
      setActionMessage('Additional information request sent successfully.');
      setMessageType('success');
      closeModal();
      await loadPendingRequests();
    } catch (error) {
      console.error('handleRequestInfo error:', error);
      setActionMessage(
        error instanceof Error ? error.message : 'Failed to send info request.'
      );
      setMessageType('error');
    } finally {
      setActionLoading(false);
    }
  };

  if (role === undefined) {
    return null;
  }

  if (role !== 'hr') {
    return <div className="p-10 text-red-500 font-semibold">Access Denied</div>;
  }

  return (
    <div className="min-h-screen bg-gray-50">
      <Sidebar />
      <TopBar />

      <div className="ml-64 pt-16">
        <main className="px-10 pt-4 pb-8 min-h-[calc(100vh-4rem)] overflow-auto">
          <LeaveTabs />

          <div className="mt-8 flex flex-col gap-4 lg:flex-row lg:items-start lg:justify-between">
            <div>
              <h1 className="text-[24px] font-bold leading-tight text-[#1F2937]">
                Approval Panel
              </h1>
              <p className="mt-1 text-[16px] text-[#667085]">
                Review and manage pending leave requests
              </p>
            </div>

            <div className="flex h-[46px] items-center rounded-[14px] bg-[#FFF2E3] px-5">
              <span className="mr-2 text-[18px] font-bold text-[#F2924E]">
                {filteredRequests.length}
              </span>
              <span className="text-[16px] text-[#F2924E]">Pending Requests</span>
            </div>
          </div>

          <div className="mt-8 flex flex-wrap items-center gap-4">
            <div className="relative w-full max-w-[360px]">
              <Search
                size={18}
                className="absolute left-4 top-1/2 -translate-y-1/2 text-[#98A2B3]"
              />
              <input
                type="text"
                placeholder="Search leave requests"
                value={searchTerm}
                onChange={(e) => setSearchTerm(e.target.value)}
                className="h-[48px] w-full rounded-[14px] border border-[#E4E7EC] bg-white pl-11 pr-4
text-[15px] text-[#344054] placeholder:text-[#98A2B3] focus:outline-none"
              />
            </div>

            <div className="relative w-[150px]">
              <select
                value={leaveTypeFilter}
                onChange={(e) => setLeaveTypeFilter(e.target.value)}
                className="h-[48px] w-full appearance-none rounded-[14px] border border-[#E4E7EC] bg-white px-4
```

```
pr-10 text-[15px] text-[#344054] focus:outline-none"
              >
                <option>All type</option>
                {[...new Set(requests.map((r) => r.leaveType))].map((type) => (
                  <option key={type}>{type}</option>
                ))}
              </select>
              <ChevronDown
                size={18}
                className="pointer-events-none absolute right-4 top-1/2 -translate-y-1/2 text-[#667085]"
              />
            </div>

            <div className="relative w-[180px]">
              <select
                value={departmentFilter}
                onChange={(e) => setDepartmentFilter(e.target.value)}
                className="h-[48px] w-full appearance-none rounded-[14px] border border-[#E4E7EC] bg-white px-4
pr-10 text-[15px] text-[#344054] focus:outline-none"
              >
                <option>All Departments</option>
                {[...new Set(requests.map((r) => r.department))].map((dept) => (
                  <option key={dept}>{dept}</option>
                ))}
              </select>
              <ChevronDown
                size={18}
                className="pointer-events-none absolute right-4 top-1/2 -translate-y-1/2 text-[#667085]"
              />
            </div>

            <div className="relative ml-auto w-[140px]">
              <select
                value={sortBy}
                onChange={(e) => setSortBy(e.target.value)}
                className="h-[48px] w-full appearance-none rounded-[14px] border border-[#E4E7EC] bg-white px-4
pr-10 text-[15px] text-[#344054] focus:outline-none"
              >
                <option>sort by</option>
                <option>Name</option>
              </select>
              <ChevronDown
                size={18}
                className="pointer-events-none absolute right-4 top-1/2 -translate-y-1/2 text-[#667085]"
              />
            </div>
          </div>

          {actionMessage && (
            <div
              className={`mt-5 flex items-center gap-3 rounded-[14px] border px-4 py-3 text-[15px] font-medium
${
                messageType === 'success'
                  ? 'border-green-200 bg-green-50 text-green-700'
                  : 'border-red-200 bg-red-50 text-red-700'
              }`}
            >
              {messageType === 'success' ? <CheckCircle2 size={18} /> : <XCircle size={18} />}
              <span>{actionMessage}</span>
            </div>
          )}

          {loading ? (
            <div className="mt-8 rounded-[14px] bg-white p-6 text-[16px] text-[#667085] shadow-sm">
              Loading pending leave requests...
            </div>
          ) : filteredRequests.length === 0 ? (
            <div className="mt-8 rounded-[14px] bg-white p-6 text-[16px] text-[#667085] shadow-sm">
              No pending leave requests found.
            </div>
          ) : (
            <div className="mt-6 grid grid-cols-1 gap-5 md:grid-cols-2 xl:grid-cols-4">
```

```
              {filteredRequests.map((request) => (
                <ApprovalRequestCard
                  key={request.id}
                  request={request}
                  onReview={() => openReview(request)}
                />
              ))}
            </div>
          )}
        </main>
      </div>

      {selectedRequest && (
        <ApprovalReviewModal
          request={selectedRequest}
          mode={modalMode}
          onClose={closeModal}
          onApprove={() => setModalMode('approve')}
          onReject={() => setModalMode('reject')}
          onReqInfo={() => setModalMode('info')}
          onCancelAction={() => setModalMode('review')}
          onConfirmApprove={handleApprove}
          onConfirmReject={handleReject}
          onSendInfoRequest={handleRequestInfo}
        />
      )}

      {actionLoading && (
        <div className="fixed inset-0 z-[110] flex items-center justify-center bg-black/20">
          <div className="rounded-[14px] bg-white px-6 py-4 text-[16px] font-medium text-[#344054] shadow-lg">
            Processing...
          </div>
        </div>
      )}
    </div>
  );
}
```

File: frontend\app\components\LeaveBalanceCards.tsx

```
import React from 'react';

interface Props {
  balances: Record<string, number>;
}

export default function LeaveBalanceCards({ balances }: Props) {
  return (
    <div className="grid grid-cols-2 gap-6 mb-6">
      {Object.entries(balances)
        // don't display sick leave on this page
        .filter(([type]) => type.toLowerCase() !== 'sick')
        .map(([type, amount]) => (
          <div
            key={type}
            className="bg-[#FFF3E6] border border-[#F5C28B] rounded-xl p-5"
          >
            <p className="text-sm text-gray-600 mb-1">
              {type} Leaves
            </p>
            <h2 className="text-2xl font-bold text-gray-800">
              {amount} days
            </h2>
          </div>
        ))}
    </div>
  );
}
```

File: frontend\app\documents\page.tsx

```
'use client';

import React from 'react';
import Sidebar from '@/components/Sidebar';
import TopBar from '@/components/TopBar';

export default function DocumentsPage() {
  return (
    <div className="min-h-screen bg-gray-50">
      <Sidebar />
      <TopBar />

      <div className="ml-64 pt-16">
        <main className="px-10 py-8 min-h-[calc(100vh-4rem)] overflow-auto">
          <h1 className="text-2xl font-semibold mb-4">Documents</h1>
          <p>Upload and manage HR-related documents here.</p>
        </main>
      </div>
    </div>
  );
}
```

File: frontend\app\employees\page.tsx

```
'use client';

import React from 'react';
import Sidebar from '@/components/Sidebar';
import TopBar from '@/components/TopBar';

export default function EmployeesPage() {
  return (
    <div className="min-h-screen bg-gray-50">
      <Sidebar />
      <TopBar />

      <div className="ml-64 pt-16">
        <main className="px-10 py-8 min-h-[calc(100vh-4rem)] overflow-auto">
          <h1 className="text-2xl font-semibold mb-4">Employee Management</h1>
          <p>Here you'll be able to view and manage employee records.</p>
        </main>
      </div>
    </div>
  );
}
```

File: frontend\app\leave-history\page.tsx

```
'use client';

import React, { useEffect, useState } from 'react';
import Sidebar from '@/components/Sidebar';
import TopBar from '@/components/TopBar';
import LeaveTabs from '@/components/LeaveTabs';
import LeaveResubmitModal from '@/components/LeaveResubmitModal';
import EditLeaveModal from '@/components/EditLeaveModal';
import { Search, Pencil, Trash2 } from "lucide-react";
import { API_BASE_URL, getAuthHeaders } from '../lib/api';

interface LeaveRequest {
  leave_request_id: number;
```

```
  employee_id: number;
  leave_type_id: number;
  leave_type_name?: string | null;
  start_date: string;
  end_date: string;
  total_days: number;
  half_day: boolean;
  status: 'PENDING' | 'APPROVED' | 'REJECTED' | 'REQ_INFO';
  reason?: string | null;
  attachment_urls?: string[];
  rejection_reason?: string | null;
  manager_comment?: string | null;
  approved_by?: number | null;
  approved_date?: string | null;
}


const getStatusBadgeColor = (status: string) => {
  switch (status) {
    case 'APPROVED':
      return 'bg-orange-100 text-orange-600';
    case 'PENDING':
      return 'bg-gray-200 text-gray-600';
    case 'REJECTED':
      return 'bg-red-100 text-red-600';
    case 'REQ_INFO':
      return 'bg-yellow-100 text-yellow-700';
    default:
      return 'bg-gray-100 text-gray-600';
  }
};



const leaveTypeIdMap: Record<string, string> = {
  'Annual Leave': '1',
  'Medical Leave': '2',
  'Casual Leave': '3',
  'Short Leave': '4',
};

export default function LeaveHistoryPage() {
  const [searchTerm, setSearchTerm] = useState('');
  const [leaveType, setLeaveType] = useState('All type');
  const [status, setStatus] = useState('All Status');
  const [sortBy, setSortBy] = useState('Newest first');

  const [leaveRequests, setLeaveRequests] = useState<LeaveRequest[]>([]);
  const [loading, setLoading] = useState(true);
  const [selectedRequestForInfo, setSelectedRequestForInfo] = useState<LeaveRequest | null>(null);
  const [selectedRequestForEdit, setSelectedRequestForEdit] = useState<LeaveRequest | null>(null);

  const fetchLeaveHistory = React.useCallback(async () => {
    setLoading(true);

    try {
      const params = new URLSearchParams();

      if (searchTerm.trim()) {
        params.append("search", searchTerm.trim());
      }

      if (leaveType !== "All type" && leaveTypeIdMap[leaveType]) {
        params.append("leave_type_id", leaveTypeIdMap[leaveType]);
      }

      if (status !== "All Status") {
        let backendStatus = status.toUpperCase();
        if (backendStatus === "ACTION REQUIRED") {
          backendStatus = "REQ_INFO";
        }
        params.append("status", backendStatus);
```

```
    }

    params.append("sort_by", sortBy === "Newest first" ? "newest" : "oldest");

    const url = `${API_BASE_URL}/leave/history/me?${params.toString()}`;
    console.log("Fetching leave history:", url);
    console.log("Headers:", getAuthHeaders());

    const response = await fetch(url, {
      method: "GET",
      headers: getAuthHeaders(),
      cache: "no-store",
    });

    if (!response.ok) {
      const errorText = await response.text();
      throw new Error(errorText || "Failed to fetch leave history");
    }

    const data = await response.json();
    console.log("Leave history response:", data);
    setLeaveRequests(data);
  } catch (error) {
    console.error("Error fetching leave history:", error);
    setLeaveRequests([]);
  } finally {
    setLoading(false);
  }
}, [searchTerm, leaveType, status, sortBy, leaveTypeIdMap]);

useEffect(() => {
  fetchLeaveHistory();
}, [fetchLeaveHistory]);

const handleDelete = async (requestId: number) => {
  if (window.confirm("Are you sure you want to cancel and delete this pending request?")) {
    try {
      const url = `${API_BASE_URL}/leave/requests/${requestId}`;
      const res = await fetch(url, {
        method: 'DELETE',
        headers: getAuthHeaders(),
      });
      if (!res.ok) throw new Error("Failed to delete leave request");
      setSelectedRequestForEdit(null); // safely close the modal if open
      fetchLeaveHistory();
    } catch (err) {
      console.error("Error deleting request", err);
      alert("Could not delete request. Please try again.");
    }
  }
};


const formatDate = (dateString: string) => {
  return new Date(dateString).toLocaleDateString('en-US', {
    month: 'short',
    day: 'numeric',
    year: 'numeric',
  });
};


return (
  <div className="min-h-screen bg-gray-50">
    <Sidebar />
    <TopBar />

    <div className="ml-64 pt-16">
      <main className="px-10 pt-4 pb-8 min-h-[calc(100vh-4rem)] overflow-auto">
        <LeaveTabs />

        <div className="mt-6">
```

```
            <h1 className="text-2xl font-semibold text-gray-900 mb-1">Leave History</h1>
            <p className="text-gray-600 mb-6">Review and manage pending leave requests</p>


            <div className="bg-white rounded-lg shadow-sm p-4 mb-6">
              <div className="flex flex-row gap-4 items-center">
                <div className="relative flex-[2] min-w-0">
                  <input
                    type="text"
                    placeholder="Search leave requests"
                    value={searchTerm}
                    onChange={(e) => setSearchTerm(e.target.value)}
                    className="w-full pl-10 pr-4 py-2 border border-gray-300 rounded-lg text-sm text-gray-400
placeholder-gray-400 bg-white focus:outline-none focus:ring-2 focus:ring-orange-200"
                  />
                  <span className="absolute left-3 top-1/2 -translate-y-1/2 text-gray-400">
                    <Search size={16} />
                  </span>
                </div>

                <select
                  value={leaveType}
                  onChange={(e) => setLeaveType(e.target.value)}
                  className="flex-1 min-w-[160px] px-4 py-2 border border-gray-300 rounded-lg text-sm
text-gray-400 bg-white focus:outline-none focus:ring-2 focus:ring-orange-200 cursor-pointer"
                >
                  <option>All type</option>
                  <option>Annual Leave</option>
                  <option>Casual Leave</option>
                  <option>Medical Leave</option>
                  <option>Short Leave</option>
                </select>

                <select
                  value={status}
                  onChange={(e) => setStatus(e.target.value)}
                  className="flex-1 min-w-[160px] px-4 py-2 border border-gray-300 rounded-lg text-sm
text-gray-400 bg-white focus:outline-none focus:ring-2 focus:ring-orange-200 cursor-pointer"
                >
                  <option>All Status</option>
                  <option>Approved</option>
                  <option>Pending</option>
                  <option>Rejected</option>
                  <option>Action Required</option>
                </select>
                <select
                  value={sortBy}
                  onChange={(e) => setSortBy(e.target.value)}
                  className="flex-1 min-w-[160px] px-4 py-2 border border-gray-300 rounded-lg text-sm
text-gray-400 bg-white focus:outline-none focus:ring-2 focus:ring-orange-200 cursor-pointer"
                >
                  <option>Newest first</option>
                  <option>Oldest first</option>
                </select>
              </div>
            </div>

            <div className="bg-white rounded-lg shadow-sm overflow-hidden">
              <div className="overflow-x-auto">
                <table className="w-full">
                  <thead className="bg-gray-50 border-b border-gray-200">
                    <tr>
                      <th className="px-6 py-3 text-left text-xs font-semibold text-gray-600 uppercase
tracking-wider">
                        REQUEST ID
                      </th>
                      <th className="px-6 py-3 text-left text-xs font-semibold text-gray-600 uppercase
tracking-wider">
                        LEAVE TYPE
                      </th>
                      <th className="px-6 py-3 text-left text-xs font-semibold text-gray-600 uppercase
tracking-wider">
```

```
                      DATE RANGE
                    </th>
                    <th className="px-6 py-3 text-left text-xs font-semibold text-gray-600 uppercase
tracking-wider">
                      TOTAL DAYS
                    </th>
                    <th className="px-6 py-3 text-left text-xs font-semibold text-gray-600 uppercase
tracking-wider">
                      STATUS
                    </th>
                    <th className="px-6 py-3 text-left text-xs font-semibold text-gray-600 uppercase
tracking-wider">
                      APPROVER
                    </th>
                  </tr>
                </thead>
                <tbody className="divide-y divide-gray-200">
                  {loading ? (
                    <tr>
                      <td colSpan={6} className="px-6 py-8 text-center text-sm text-gray-500">
                        Loading leave history...
                      </td>
                    </tr>
                  ) : leaveRequests.length === 0 ? (
                    <tr>
                      <td colSpan={6} className="px-6 py-8 text-center text-sm text-gray-500">
                        No leave history found
                      </td>
                    </tr>
                  ) : (
                    leaveRequests.map((request) => (
                      <tr key={request.leave_request_id} className="hover:bg-gray-50 transition-colors">
                        <td className="px-6 py-4 text-sm text-gray-900">
                          LR-{request.leave_request_id}
                        </td>
                        <td className="px-6 py-4 text-sm text-gray-700">
                          {request.leave_type_name || '-'}
                        </td>
                        <td className="px-6 py-4 text-sm text-gray-700">
                          {formatDate(request.start_date)} - {formatDate(request.end_date)}
                        </td>
                        <td className="px-6 py-4 text-sm text-gray-700">
                          {request.half_day ? '0.5 days' : `${request.total_days} days`}
                        </td>
                        <td className="px-6 py-4 text-sm">
                          {request.status === 'REQ_INFO' ? (
                            <button
                              onClick={() => setSelectedRequestForInfo(request)}
                              className="px-3 py-1 rounded-full text-xs font-semibold bg-red-100 text-red-700
hover:bg-red-200 border border-red-200 transition-colors cursor-pointer shadow-sm animate-pulse"
                            >
                              Action Req
                            </button>
                          ) : (
                            <div className="flex items-center gap-2">
                              <span className={`px-3 py-1 rounded-full text-xs font-medium
${getStatusBadgeColor(request.status)}`}>
                                {request.status}
                              </span>
                              {request.status === 'PENDING' && (
                                <div className="flex items-center gap-1 ml-1 opacity-80 hover:opacity-100
transition-opacity">
                                  <button
                                    onClick={() => setSelectedRequestForEdit(request)}
                                    title="Edit pending request"
                                    className="p-1.5 hover:bg-gray-200 rounded-md text-gray-500
hover:text-blue-600 transition-colors"
                                  >
                                    <Pencil size={14} />
                                  </button>
                                </div>
                              )}
```

```
                          </div>
                        )}
                      </td>
                      <td className="px-6 py-4 text-sm text-gray-700">
                        {request.approved_by ?? '-'}
                      </td>
                    </tr>
                  ))
                )}
              </tbody>
            </table>
          </div>
        </div>
      </div>
    </main>
  </div>

  {selectedRequestForInfo && (
    <LeaveResubmitModal
      request={selectedRequestForInfo}
      onClose={() => setSelectedRequestForInfo(null)}
      onSuccess={() => {
        setSelectedRequestForInfo(null);
        fetchLeaveHistory();
      }}
    />
  )}

  {selectedRequestForEdit && (
    <EditLeaveModal
      request={selectedRequestForEdit}
      onClose={() => setSelectedRequestForEdit(null)}
      onSuccess={() => {
        setSelectedRequestForEdit(null);
        fetchLeaveHistory();
      }}
      onDelete={() => handleDelete(selectedRequestForEdit.leave_request_id)}
    />
  )}
</div>
);
}
```

File: frontend\app\lib\api.ts

```
export const API_BASE_URL =
  process.env.NEXT_PUBLIC_API_BASE_URL || "http://localhost:8000";

export function getAuthHeaders(): HeadersInit {
  if (typeof window === "undefined") {
    return {
      "Content-Type": "application/json",
    };
  }

  const userId = localStorage.getItem("userId") || "";
  const role = localStorage.getItem("role") || "";

  console.log("Sending headers:", { userId, role });

  return {
    "Content-Type": "application/json",
    "x-user-id": userId,
    "x-user-roles": role,
  };
}
```

File: frontend\app\lib\auth.ts

```
export function getRole() {
  if (typeof window === "undefined") return null;
  return localStorage.getItem("role"); // "employee" | "hr"
}
```

File: frontend\app\recruitment\page.tsx

```
'use client';

import React from 'react';
import Sidebar from '@/components/Sidebar';
import TopBar from '@/components/TopBar';

export default function RecruitmentPage() {
  return (
    <div className="min-h-screen bg-gray-50">
      <Sidebar />
      <TopBar />

      <div className="ml-64 pt-16">
        <main className="px-10 py-8 min-h-[calc(100vh-4rem)] overflow-auto">
          <h1 className="text-2xl font-semibold mb-4">Recruitment</h1>
          <p>This page will manage job postings and applications.</p>
        </main>
      </div>
    </div>
  );
}
```

File: frontend\app\reports\data.ts

```
import {
  EmployeeDetail,
  EmployeeReportRow,
  AttendanceRecord,
  LeaveHistoryRecord,
  SummaryCardItem,
  ViolationRecord,
  ReportPeriod,
} from "./types";

export const summaryCardsByPeriod: Record<ReportPeriod, SummaryCardItem[]> = {
  weekly: [
    { label: "Total Employees", value: 5 },
    { label: "Total Leave Used", value: 9 },
    { label: "Pending Requests", value: 2 },
    { label: "Avg Remaining", value: 18 },
  ],
  monthly: [
    { label: "Total Employees", value: 5 },
    { label: "Total Leave Used", value: 46 },
    { label: "Pending Requests", value: 6 },
    { label: "Avg Remaining", value: 10 },
  ],
  annually: [
    { label: "Total Employees", value: 5 },
    { label: "Total Leave Used", value: 132 },
    { label: "Pending Requests", value: 11 },
    { label: "Avg Remaining", value: 7 },
  ],
};

export const employeeReportRowsByPeriod: Record<ReportPeriod, EmployeeReportRow[]> = {
  weekly: [
```

```
  {
    id: "EMP001",
    name: "Sarah Johnson",
    employeeCode: "EMP001",
    role: "Manager",
    department: "Engineering",
    totalLeave: 2,
    used: 1,
    pending: 0,
    remaining: 1,
    lastLeave: "Jan 5, 2025",
  },
  {
    id: "EMP002",
    name: "Mike Chen",
    employeeCode: "EMP002",
    role: "Senior Dev",
    department: "Marketing",
    totalLeave: 2,
    used: 0,
    pending: 1,
    remaining: 1,
    lastLeave: "Jan 4, 2025",
  },
  {
    id: "EMP003",
    name: "Emily Davis",
    employeeCode: "EMP003",
    role: "Senior Developer",
    department: "Engineering",
    totalLeave: 2,
    used: 1,
    pending: 0,
    remaining: 1,
    lastLeave: "Jan 3, 2025",
  },
  {
    id: "EMP004",
    name: "John Perera",
    employeeCode: "EMP004",
    role: "HR Executive",
    department: "HR",
    totalLeave: 2,
    used: 0,
    pending: 0,
    remaining: 2,
    lastLeave: "Jan 2, 2025",
  },
  {
    id: "EMP005",
    name: "Anne Silva",
    employeeCode: "EMP005",
    role: "Developer",
    department: "Engineering",
    totalLeave: 2,
    used: 1,
    pending: 1,
    remaining: 0,
    lastLeave: "Jan 6, 2025",
  },
],

monthly: [
  {
    id: "EMP001",
    name: "Sarah Johnson",
    employeeCode: "EMP001",
    role: "Manager",
    department: "Engineering",
    totalLeave: 30,
    used: 8,
    pending: 2,
```

```
    remaining: 10,
    lastLeave: "Jan 5, 2025",
  },
  {
    id: "EMP002",
    name: "Mike Chen",
    employeeCode: "EMP002",
    role: "Senior Dev",
    department: "Marketing",
    totalLeave: 30,
    used: 12,
    pending: 0,
    remaining: 6,
    lastLeave: "Jan 10, 2025",
  },
  {
    id: "EMP003",
    name: "Emily Davis",
    employeeCode: "EMP003",
    role: "Senior Developer",
    department: "Engineering",
    totalLeave: 30,
    used: 8,
    pending: 2,
    remaining: 12,
    lastLeave: "Jan 5, 2025",
  },
  {
    id: "EMP004",
    name: "John Perera",
    employeeCode: "EMP004",
    role: "HR Executive",
    department: "HR",
    totalLeave: 30,
    used: 14,
    pending: 1,
    remaining: 9,
    lastLeave: "Jan 12, 2025",
  },
  {
    id: "EMP005",
    name: "Anne Silva",
    employeeCode: "EMP005",
    role: "Developer",
    department: "Engineering",
    totalLeave: 30,
    used: 4,
    pending: 1,
    remaining: 15,
    lastLeave: "Jan 8, 2025",
  },
],

annually: [
  {
    id: "EMP001",
    name: "Sarah Johnson",
    employeeCode: "EMP001",
    role: "Manager",
    department: "Engineering",
    totalLeave: 365,
    used: 28,
    pending: 4,
    remaining: 18,
    lastLeave: "Dec 15, 2025",
  },
  {
    id: "EMP002",
    name: "Mike Chen",
    employeeCode: "EMP002",
    role: "Senior Dev",
    department: "Marketing",
```

```
      totalLeave: 365,
      used: 22,
      pending: 1,
      remaining: 24,
      lastLeave: "Nov 10, 2025",
    },
    {
      id: "EMP003",
      name: "Emily Davis",
      employeeCode: "EMP003",
      role: "Senior Developer",
      department: "Engineering",
      totalLeave: 365,
      used: 31,
      pending: 3,
      remaining: 12,
      lastLeave: "Dec 20, 2025",
    },
    {
      id: "EMP004",
      name: "John Perera",
      employeeCode: "EMP004",
      role: "HR Executive",
      department: "HR",
      totalLeave: 365,
      used: 19,
      pending: 1,
      remaining: 20,
      lastLeave: "Oct 12, 2025",
    },
    {
      id: "EMP005",
      name: "Anne Silva",
      employeeCode: "EMP005",
      role: "Developer",
      department: "Engineering",
      totalLeave: 365,
      used: 32,
      pending: 2,
      remaining: 10,
      lastLeave: "Dec 8, 2025",
    },
  ],
};

export const departments = [
  "All Departments",
  "Engineering",
  "Marketing",
  "HR",
];

// keep your existing detail data below
export const employeeDetails: Record<string, EmployeeDetail> = {
  EMP001: {
    id: "EMP001",
    name: "Sarah Johnson",
    employeeCode: "EMP001",
    department: "Engineering",
    position: "Manager",
    manager: "John Smith",
    joinDate: "Jan 15, 2023",
    attendanceRate: 86,
    absentDays: 3,
    lateArrivals: 2,
    totalViolations: 2,
  },
  EMP002: {
    id: "EMP002",
    name: "Mike Chen",
    employeeCode: "EMP002",
    department: "Marketing",
```

```
    position: "Senior Dev",
    manager: "Anne Silva",
    joinDate: "Feb 5, 2023",
    attendanceRate: 91,
    absentDays: 1,
    lateArrivals: 0,
    totalViolations: 0,
  },
  EMP003: {
    id: "EMP003",
    name: "Emily Davis",
    employeeCode: "EMP003",
    department: "Engineering",
    position: "Senior Developer",
    manager: "John Smith",
    joinDate: "Jan 15, 2023",
    attendanceRate: 68,
    absentDays: 3,
    lateArrivals: 4,
    totalViolations: 5,
  },
  EMP004: {
    id: "EMP004",
    name: "John Perera",
    employeeCode: "EMP004",
    department: "HR",
    position: "HR Executive",
    manager: "Nadeesha Silva",
    joinDate: "Mar 12, 2023",
    attendanceRate: 89,
    absentDays: 2,
    lateArrivals: 1,
    totalViolations: 1,
  },
  EMP005: {
    id: "EMP005",
    name: "Anne Silva",
    employeeCode: "EMP005",
    department: "Engineering",
    position: "Developer",
    manager: "John Smith",
    joinDate: "Apr 18, 2023",
    attendanceRate: 93,
    absentDays: 1,
    lateArrivals: 0,
    totalViolations: 0,
  },
};

export const attendanceByEmployee: Record<string, AttendanceRecord[]> = {
  EMP001: [
    {
      employeeName: "Sarah Johnson",
      employeeCode: "EMP001",
      department: "Engineering",
      present: 19,
      absent: 3,
      totalDays: 22,
      rate: 86,
      trend: "Declining",
    },
  ],
  EMP002: [
    {
      employeeName: "Mike Chen",
      employeeCode: "EMP002",
      department: "Marketing",
      present: 20,
      absent: 2,
      totalDays: 22,
      rate: 91,
      trend: "Improving",
```

```
    },
  ],
  EMP003: [
    {
      employeeName: "Emily Davis",
      employeeCode: "EMP003",
      department: "Engineering",
      present: 19,
      absent: 3,
      totalDays: 22,
      rate: 86,
      trend: "Declining",
    },
  ],
};

export const leaveHistoryByEmployee: Record<string, LeaveHistoryRecord[]> = {
  EMP003: [
    {
      type: "Annual Leave",
      startDate: "Jan 27, 2025",
      endDate: "Jan 28, 2025",
      days: 2,
      reason: "Family vacation",
      status: "APPROVED",
    },
    {
      type: "Sick Leave",
      startDate: "Jan 15, 2025",
      endDate: "Jan 17, 2025",
      days: 3,
      reason: "Flu symptoms",
      status: "APPROVED",
    },
    {
      type: "Personal Leave",
      startDate: "Dec 20, 2024",
      endDate: "Dec 20, 2024",
      days: 1,
      reason: "Personal matter",
      status: "APPROVED",
    },
    {
      type: "Sick Leave",
      startDate: "Dec 5, 2024",
      endDate: "Dec 6, 2024",
      days: 2,
      reason: "Medical checkup",
      status: "APPROVED",
    },
  ],
};

export const violationsByEmployee: Record<string, ViolationRecord[]> = {
  EMP003: [
    {
      title: "Unauthorized Absence",
      date: "Jan 13, 2025",
      description: "Absent without prior notice or approval",
      severity: "HIGH SEVERITY",
    },
    {
      title: "Unauthorized Absence",
      date: "Jan 13, 2025",
      description: "Did not report to work without notification",
      severity: "HIGH SEVERITY",
    },
    {
      title: "Late Arrival",
      date: "Jan 22, 2025",
      description: "Arrived 2 hours late without valid reason",
      severity: "MEDIUM SEVERITY",
```

```
    },
    {
      title: "Late Arrival",
      date: "Jan 22, 2025",
      description: "Arrived 1.5 hours late",
      severity: "MEDIUM SEVERITY",
    },
    {
      title: "Early Departure",
      date: "Feb 5, 2025",
      description: "Left 2 hours early without approval",
      severity: "MEDIUM SEVERITY",
    },
  ],
};
```

File: frontend\app\reports\page.tsx

```
"use client";

import React, { useEffect, useMemo, useState } from "react";
import Sidebar from "@/components/Sidebar";
import TopBar from "@/components/TopBar";
import LeaveTabs from "@/components/LeaveTabs";
import ReportSummaryCards from "@/components/reports/ReportSummaryCards";
import ReportFilterBar from "@/components/reports/ReportFilterBar";
import EmployeeReportTable from "@/components/reports/EmployeeReportTable";
import { ReportPeriod } from "./types";

type BackendLeaveRecord = {
  leave_request_id: number;
  employee_id: number;
  employee_code?: string | null;
  employee_name?: string | null;
  department?: string | null;
  designation?: string | null;
  joined_date?: string | null;
  employee_status?: string | null;
  leave_type_id: number;
  leave_type_name?: string | null;
  start_date: string;
  end_date: string;
  total_days: number;
  half_day: boolean;
  status: string;
  reason?: string | null;
  approved_by?: number | null;
  approved_date?: string | null;
  rejection_reason?: string | null;
  manager_comment?: string | null;
};

type BackendLeaveSummary = {
  total_requests: number;
  total_leave_days: number;
  approved_requests: number;
  pending_requests: number;
  rejected_requests: number;
  req_info_requests: number;
};

type BackendLeaveReportResponse = {
  summary: BackendLeaveSummary;
  records: BackendLeaveRecord[];
};

const API_BASE_URL = "http://127.0.0.1:8000";
// ■■ Helpers ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
function getPeriodDateRange(period: ReportPeriod): { start: string; end: string } {
```

```
  const today = new Date();
  const toYMD = (d: Date) => d.toISOString().split("T")[0];

  if (period === "weekly") {
    // Mon → Sun of the current week
    const day = today.getDay(); // 0 = Sun
    const diffToMon = (day + 6) % 7;
    const mon = new Date(today);
    mon.setDate(today.getDate() - diffToMon);
    const sun = new Date(mon);
    sun.setDate(mon.getDate() + 6);
    return { start: toYMD(mon), end: toYMD(sun) };
  }

  if (period === "monthly") {
    const start = new Date(today.getFullYear(), today.getMonth(), 1);
    const end = new Date(today.getFullYear(), today.getMonth() + 1, 0);
    return { start: toYMD(start), end: toYMD(end) };
  }

  // annually
  const start = new Date(today.getFullYear(), 0, 1);
  const end = new Date(today.getFullYear(), 11, 31);
  return { start: toYMD(start), end: toYMD(end) };
}

export default function ReportsPage() {
  const [period, setPeriod] = useState<ReportPeriod>("monthly");
  const [search, setSearch] = useState("");
  const [department, setDepartment] = useState("All Departments");

  const [reportData, setReportData] = useState<BackendLeaveReportResponse | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    const fetchReport = async () => {
      try {
        setLoading(true);
        setError("");

        const { start, end } = getPeriodDateRange(period);
        const url = `${API_BASE_URL}/reports/leave?start_date=${start}&end_date=${end}`;
        const response = await fetch(url);

        if (!response.ok) {
          throw new Error("Failed to fetch report data");
        }

        const data: BackendLeaveReportResponse = await response.json();
        setReportData(data);
      } catch (err) {
        console.error("Report fetch error:", err);
        setError("Failed to load report data");
      } finally {
        setLoading(false);
      }
    };

    fetchReport();
  }, [period]);

  const summaryCards = useMemo(() => {
    if (!reportData) return [];

    return [
      {
        label: "Total Leave Requests",
        value: reportData.summary.total_requests,
      },
      {
        label: "Approved Requests",
```

```
      value: reportData.summary.approved_requests,
    },
    {
      label: "Pending Requests",
      value: reportData.summary.pending_requests,
    },
    {
      label: "Rejected Requests",
      value: reportData.summary.rejected_requests,
    },
    {
      label: "Total Leave Days Taken",
      value: reportData.summary.total_leave_days,
    },
  ];
}, [reportData]);

const tableRows = useMemo(() => {
  if (!reportData) return [];

  const grouped = new Map<
    number,
    {
      id: string;
      name: string;
      employeeCode: string;
      role: string;
      department: string;
      totalLeave: number;
      used: number;
      pending: number;
      remaining: number;
      lastLeave: string;
    }
  >();

  reportData.records.forEach((record) => {
    const key = record.employee_id;

    if (!grouped.has(key)) {
      grouped.set(key, {
        id: String(record.employee_id),
        name: record.employee_name || `Employee ${record.employee_id}`,
        employeeCode:
          record.employee_code || `EMP-${String(record.employee_id).padStart(3, "0")}`,
        role: record.designation || "Staff",
        department: record.department || "N/A",
        totalLeave: 0,
        used: 0,
        pending: 0,
        remaining: 0,
        lastLeave: record.end_date,
      });
    }

    const employee = grouped.get(key)!;

    employee.totalLeave += Number(record.total_days || 0);

    if (record.status === "APPROVED") {
      employee.used += Number(record.total_days || 0);
    }

    if (record.status === "PENDING") {
      employee.pending += Number(record.total_days || 0);
    }

    if (record.end_date > employee.lastLeave) {
      employee.lastLeave = record.end_date;
    }
  });
```

```
  for (const employee of grouped.values()) {
    employee.remaining = Math.max(
      employee.totalLeave - employee.used - employee.pending,
      0
    );
  }

  return Array.from(grouped.values());
}, [reportData]);

const filteredRows = useMemo(() => {
  return tableRows.filter((row) => {
    const matchesSearch =
      row.name.toLowerCase().includes(search.toLowerCase()) ||
      row.employeeCode.toLowerCase().includes(search.toLowerCase());

    const matchesDepartment =
      department === "All Departments" || row.department === department;

    return matchesSearch && matchesDepartment;
  });
}, [tableRows, search, department]);

const handleDownloadCsv = () => {
  const { start, end } = getPeriodDateRange(period);
  window.open(
    `${API_BASE_URL}/reports/leave/export/csv?start_date=${start}&end_date=${end}`,
    "_blank"
  );
};

const handleDownloadPdf = () => {
  const { start, end } = getPeriodDateRange(period);
  window.open(
    `${API_BASE_URL}/reports/leave/pdf?start_date=${start}&end_date=${end}`,
    "_blank"
  );
};

const departmentOptions = useMemo(() => {
  const uniqueDepartments = Array.from(
    new Set(
      tableRows
        .map((row) => row.department)
        .filter((d) => d && d !== "N/A")
    )
  );

  return ["All Departments", ...uniqueDepartments];
}, [tableRows]);

const [role, setRole] = useState<string | null>(null);
const [mounted, setMounted] = useState(false);
useEffect(() => {
  setMounted(true);
  const userRole = localStorage.getItem("role");
  setRole(userRole);
}, []);

if (!mounted || role !== "hr") {
  return <div>Access Denied</div>;
}

return (
  <div className="min-h-screen bg-gray-50">
    <Sidebar />
    <TopBar />

    <div className="ml-64 pt-16">
      <main className="min-h-[calc(100vh-4rem)] overflow-auto px-10 pt-4 pb-8">
        <LeaveTabs />
```

```
          <section className="mt-6">
            <h1 className="text-2xl font-semibold text-gray-900 md:text-[24px]">
              Employee Leave Summary
            </h1>
            <p className="mt-1 md:text-[16px] text-base text-gray-500 md:text-lg">
              Comprehensive leave report for all employees
            </p>
          </section>

          {loading && (
            <div className="mt-6 rounded-2xl bg-white p-6 text-gray-500 shadow-sm">
              Loading report data...
            </div>
          )}

          {error && (
            <div className="mt-6 rounded-2xl bg-red-50 p-6 text-red-600 shadow-sm">
              {error}
            </div>
          )}

          {!loading && !error && (
            <>
              <ReportSummaryCards cards={summaryCards} />

              <div className="mt-6 rounded-3xl border border-gray-200 bg-white p-5
shadow-[0_1px_3px_rgba(16,24,40,0.06)] md:p-6">
                <ReportFilterBar
                  period={period}
                  setPeriod={setPeriod}
                  search={search}
                  setSearch={setSearch}
                  department={department}
                  setDepartment={setDepartment}
                  departments={departmentOptions}
                  onExportCsv={handleDownloadCsv}
                  onExportPdf={handleDownloadPdf}
                />

                <EmployeeReportTable rows={filteredRows} />
              </div>
            </>
          )}
        </main>
      </div>
    </div>
  );
}
```

File: frontend\app\reports\types.ts

```
export type ReportPeriod = "weekly" | "monthly" | "annually";
export type DetailTab = "attendance" | "leave" | "violations";

export interface EmployeeDetail {
  id: string;
  name: string;
  employeeCode: string;
  department: string;
  position: string;
  manager: string;
  joinDate: string;
  attendanceRate: number;
  absentDays: number;
  lateArrivals: number;
  totalViolations: number;
}

export interface EmployeeReportRow {
```

```
  id: string;
  name: string;
  employeeCode: string;
  role: string;
  department: string;
  totalLeave: number;
  used: number;
  pending: number;
  remaining: number;
  lastLeave: string;
}

export interface AttendanceRecord {
  employeeName: string;
  employeeCode: string;
  department: string;
  present: number;
  absent: number;
  totalDays: number;
  rate: number;
  trend: string;
}

export interface LeaveHistoryRecord {
  type: string;
  startDate: string;
  endDate: string;
  days: number;
  reason: string;
  status: string;
}
export interface SummaryCardItem {
  label: string;
  value: number;
}

export interface ViolationRecord {
  title: string;
  date: string;
  description: string;
  severity: string;
}
```

File: frontend\app\reports\[employeeId]\page.tsx

```
"use client";

import React, { useEffect, useMemo, useState } from "react";
import Link from "next/link";
import { useParams, notFound } from "next/navigation";

import Sidebar from "@/components/Sidebar";
import TopBar from "@/components/TopBar";
import LeaveTabs from "@/components/LeaveTabs";
import EmployeeReportHeader from "@/components/reports/EmployeeReportHeader";
import EmployeeReportStats from "@/components/reports/EmployeeReportStats";
import EmployeeDetailTabs from "@/components/reports/EmployeeDetailTabs";
import AttendanceRecordsTable from "@/components/reports/AttendanceRecordsTable";
import LeaveHistoryPanel from "@/components/reports/LeaveHistoryPanel";
import ViolationsPanel from "@/components/reports/ViolationsPanel";
import ManagerNotesCard from "@/components/reports/ManagerNotesCard";

import { DetailTab, ReportPeriod } from "../types";

export default function EmployeeReportDetailPage() {
  const params = useParams();
  const employeeId = String(params.employeeId);

  const [period, setPeriod] = useState<ReportPeriod>("monthly");
```

```
const [activeTab, setActiveTab] = useState<DetailTab>("attendance");

const [employee, setEmployee] = useState<any>(null);
const [leaveRecords, setLeaveRecords] = useState<any[]>([]);
const [loading, setLoading] = useState(true);

// ■ Fetch data from backend
useEffect(() => {
  fetch(`http://127.0.0.1:8000/reports/leave?employee_id=${employeeId}`)
    .then((res) => res.json())
    .then((data) => {
      if (!data.records || data.records.length === 0) {
        notFound();
        return;
      }

      const first = data.records[0];

      // ■ Set employee info
      setEmployee({
        id: employeeId,
        name: first.employee_name || "N/A",
        employeeCode: first.employee_code || "N/A",
        department: first.department || "N/A",
        position: first.designation || "N/A",
        manager: "N/A",
        joinDate: first.joined_date || "",
        attendanceRate: 0,
        absentDays: 0,
        lateArrivals: 0,
        totalViolations: 0,
      });

      // ■ Set leave records
      const mappedLeaves = data.records.map((r: any) => ({
        type: r.leave_type_name,
        startDate: r.start_date,
        endDate: r.end_date,
        days: r.total_days,
        reason: r.reason,
        status: r.status,
      }));

      setLeaveRecords(mappedLeaves);
      setLoading(false);
    })
    .catch(() => {
      notFound();
    });
}, [employeeId]);

// ■■ Placeholder (until backend ready)
const attendanceRecords: any[] = [];
const violationRecords: any[] = [];

const content = useMemo(() => {
  if (activeTab === "attendance") {
    return <AttendanceRecordsTable records={attendanceRecords} />;
  }

  if (activeTab === "leave") {
    return <LeaveHistoryPanel records={leaveRecords} />;
  }

  return <ViolationsPanel records={violationRecords} />;
}, [activeTab, attendanceRecords, leaveRecords, violationRecords]);

// ■ Loading state
if (loading) {
  return <div className="p-10">Loading...</div>;
}
```

```
  if (!employee) return null;

  return (
    <div className="min-h-screen bg-gray-50">
      <Sidebar />
      <TopBar />

      <div className="ml-64 pt-16">
        <main className="min-h-[calc(100vh-4rem)] overflow-auto px-10 pt-4 pb-8">
          <LeaveTabs />

          <div className="mt-2">
            <Link
              href="/reports"
              className="text-sm text-orange-400 hover:text-orange-500 mb-2"
            >
              ← Back to Summary
            </Link>

            <h1 className="mt-3 text-2xl font-semibold text-gray-900 md:text-[24px]">
              Employee Detailed Report
            </h1>
          </div>

          <EmployeeReportHeader
            employee={employee}
            period={period}
            setPeriod={setPeriod}
          />

          <EmployeeReportStats
            employee={employee}
            activeTab={activeTab}
            period={period}
            setPeriod={setPeriod}
          />

          <EmployeeDetailTabs
            activeTab={activeTab}
            setActiveTab={setActiveTab}
            violationCount={violationRecords.length}
          />

          {content}

          <ManagerNotesCard />
        </main>
      </div>
    </div>
  );
}
```

File: frontend\app\settings\page.tsx

```
'use client';

import React from 'react';
import Sidebar from '@/components/Sidebar';
import TopBar from '@/components/TopBar';

export default function SettingsPage() {
  return (
    <div className="min-h-screen bg-gray-50">
      <Sidebar />
      <TopBar />

      <div className="ml-64 pt-16">
        <main className="px-10 py-8 min-h-[calc(100vh-4rem)] overflow-auto">
          <h1 className="text-2xl font-semibold mb-4">Settings</h1>
```

```
          <p>Adjust application preferences and account settings.</p>
        </main>
      </div>
    </div>
  );
}
```

File: frontend\components\ApplyLeaveForm.tsx

```
"use client";

import React, { useEffect, useState } from "react";
import { API_BASE_URL, getAuthHeaders } from "../app/lib/api";
import LeaveDatePicker from "./LeaveDatePicker";

interface Props {
  balances: Record<string, number>;
}

interface LeaveType {
  id: number;
  name: string;
  description?: string | null;
}

const ApplyLeaveForm: React.FC<Props> = ({ balances }) => {
  const [leaveTypes, setLeaveTypes] = useState<LeaveType[]>([]);
  const [leaveTypeId, setLeaveTypeId] = useState<string>("");

  const [dateSelection, setDateSelection] = useState({
    startDate: "",
    endDate: "",
    halfDay: false,
  });
  const [reason, setReason] = useState("");
  const [attachments, setAttachments] = useState<File[]>([]);
  const [errors, setErrors] = useState<string[]>([]);
  const [success, setSuccess] = useState(false);
  const [loadingTypes, setLoadingTypes] = useState(false);
  const [submitting, setSubmitting] = useState(false);
  const [isDragging, setIsDragging] = useState(false);

  // Derived values
  const fromDate = dateSelection.startDate;
  const toDate = dateSelection.endDate;
  const halfDay = dateSelection.halfDay;
  const days = (() => {
    if (!fromDate || !toDate) return 0;
    if (halfDay) return 0.5;
    const diff =
      Math.ceil(
        (new Date(toDate).getTime() - new Date(fromDate).getTime()) /
          (1000 * 60 * 60 * 24)
      ) + 1;
    return diff > 0 ? diff : 0;
  })();


  const addFiles = (newFiles: FileList | File[]) => {
  const filesArray = Array.from(newFiles);
  setAttachments((prev) => [...prev, ...filesArray]);
  };

  const handleDragOver = (e: React.DragEvent<HTMLDivElement>) => {
  e.preventDefault();
  setIsDragging(true);
  };

  const handleDragLeave = (e: React.DragEvent<HTMLDivElement>) => {
```

```
e.preventDefault();
setIsDragging(false);
};

const handleDrop = (e: React.DragEvent<HTMLDivElement>) => {
e.preventDefault();
setIsDragging(false);

if (e.dataTransfer.files && e.dataTransfer.files.length > 0) {
  addFiles(e.dataTransfer.files);
  }
};

const removeFile = (indexToRemove: number) => {
setAttachments((prev) => prev.filter((_, index) => index !== indexToRemove));
};

useEffect(() => {
  const fetchLeaveTypes = async () => {
    setLoadingTypes(true);
    try {
      const res = await fetch(`${API_BASE_URL}/leave/types`);
      if (!res.ok) {
        throw new Error("Failed to load leave types");
      }

      const data: LeaveType[] = await res.json();
      setLeaveTypes(data);

      if (data.length > 0) {
        setLeaveTypeId(String(data[0].id));
      }
    } catch (error) {
      setErrors(["Could not load leave types"]);
    } finally {
      setLoadingTypes(false);
    }
  };
  fetchLeaveTypes();
}, []);



const validate = () => {
const errs: string[] = [];

const selectedLeaveType = leaveTypes.find(
  (type) => String(type.id) === leaveTypeId
);

if (!leaveTypeId) errs.push("Leave type is required");
if (!fromDate) errs.push("From date is required");
if (!toDate) errs.push("To date is required");
if (fromDate && toDate && new Date(toDate) < new Date(fromDate)) {
  errs.push("To date cannot be earlier than From date");
}
if (!reason.trim()) errs.push("Reason is required");

if (
  selectedLeaveType?.name.toLowerCase().includes("medical") &&
  attachments.length === 0
) {
  errs.push("Medical leave requires a supporting document");
}

setErrors(errs);
return errs.length === 0;
};
const resetForm = () => {
  setDateSelection({ startDate: "", endDate: "", halfDay: false });
  setReason("");
  setAttachments([]);
```

```
  setErrors([]);

  if (leaveTypes.length > 0) {
    setLeaveTypeId(String(leaveTypes[0].id));
  } else {
    setLeaveTypeId("");
  }
};

const handleSubmit = async (e: React.FormEvent) => {
e.preventDefault();
setSuccess(false);

if (!validate()) return;

setSubmitting(true);

try {
  let uploadedFileUrls: string[] = [];

  if (attachments.length > 0) {
    for (const file of attachments) {
      const formData = new FormData();
      formData.append("file", file);

      const headers = getAuthHeaders() as Record<string, string>;
      delete headers['Content-Type'];

      const uploadRes = await fetch(`${API_BASE_URL}/leave/upload`, {
        method: "POST",
        headers,
        body: formData,
      });

      const uploadData = await uploadRes.json();

      if (!uploadRes.ok) {
        throw new Error(uploadData.detail || `Failed to upload ${file.name}`);
      }

      uploadedFileUrls.push(uploadData.file_url);
    }
  }

  // ■ Then submit leave request with file URL
  const payload = {
    leave_type_id: Number(leaveTypeId),
    start_date: fromDate,
    end_date: toDate,
    half_day: halfDay,
    reason: reason,
    attachment_urls: uploadedFileUrls,
  };

  const res = await fetch(`${API_BASE_URL}/leave/requests`, {
    method: "POST",
    headers: {
      ...getAuthHeaders(),
      "Content-Type": "application/json",
    },
    body: JSON.stringify(payload),
  });

  const data = await res.json();

  if (!res.ok) {
    throw new Error(data.detail || "Failed to submit leave request");
  }

  setSuccess(true);
  resetForm();
```

```
  setTimeout(() => setSuccess(false), 3000);
} catch (error) {
  const message =
    error instanceof Error ? error.message : "Something went wrong";
  setErrors([message]);
} finally {
  setSubmitting(false);
}
};

return (
  <form onSubmit={handleSubmit} className="bg-white rounded-xl shadow-sm p-6 mt-6">
    {success && (
      <div className="bg-green-50 text-green-700 p-2 rounded mb-4">
        Leave Request Submitted Successfully
      </div>
    )}

    {errors.length > 0 && (
      <div className="bg-red-50 text-red-700 p-2 rounded mb-4">
        <ul className="list-disc list-inside">
          {errors.map((e, i) => (
            <li key={i}>{e}</li>
          ))}
        </ul>
      </div>
    )}

    <div className="mb-4">
      <label className="block text-sm text-gray-600 mb-1">Leave type</label>
      <select
        value={leaveTypeId}
        onChange={(e) => setLeaveTypeId(e.target.value)}
        disabled={loadingTypes}
        className="w-full border rounded-lg px-3 py-2 bg-white text-[#6C6C70]"
      >
        {leaveTypes.map((type) => (
          <option key={type.id} value={type.id}>
            {type.name}
          </option>
        ))}
      </select>
    </div>
    <div className="mb-4">
      <label className="block text-sm text-gray-600 mb-1">Leave Duration</label>
      <LeaveDatePicker
        value={dateSelection}
        onChange={(sel) => setDateSelection(sel)}
      />
    </div>

    <div className="mb-4">
      <label className="block text-sm text-gray-600 mb-1">Total Leave Days</label>
      <input
        type="number"
        value={days}
        readOnly
        className="w-full border rounded-lg px-3 py-2 bg-[#FFF3E6] text-[#6C6C70]"
      />
    </div>

    <div className="mb-4">
      <label className="block text-sm text-gray-600 mb-1">Reason for leave</label>
      <textarea
        value={reason}
        onChange={(e) => setReason(e.target.value)}
        rows={4}
        className="w-full border rounded-lg px-3 py-2 bg-white text-[#6C6C70]"
      />
    </div>

    <div className="mb-4">
```

```
        <label className="block text-sm text-gray-600 mb-1">Attachment (optional)</label>
        <div
          onDragOver={handleDragOver}
          onDragLeave={handleDragLeave}
          onDrop={handleDrop}
          className={`mt-1 flex justify-center items-center px-6 pt-5 pb-6 border-2 border-dashed rounded-xl
transition-colors ${
            isDragging
              ? "border-orange-500 bg-orange-50"
              : "border-gray-300 bg-white"
          }`}
        >
           <div className="flex flex-col items-center space-y-2">
            <div className="text-gray-400">Upload file</div>

            <div className="flex text-sm text-gray-600">
              <label
                htmlFor="file-upload"
                className="relative cursor-pointer rounded-md bg-white font-medium text-orange-600
hover:text-orange-500"
              >
                <span>Click to upload</span>
                <input
                  id="file-upload"
                  name="attachments"
                  type="file"
                  multiple
                  className="sr-only"
                  onChange={(e) => {
                    if (e.target.files) {
                      addFiles(e.target.files);
                    }
                  }}
                />
              </label>
              <p className="pl-1">or drag and drop</p>
            </div>

            <p className="text-xs text-gray-500">PDF, JPG up to 5MB</p>
          </div>
        </div>
      </div>

      <div className="flex justify-end space-x-4">
        <button
          type="button"
          onClick={resetForm}
          className="px-4 py-2 rounded-lg border border-gray-300 text-gray-700 hover:bg-gray-100
transition-colors duration-200"
        >
          Cancel Request
        </button>

        <button
          type="submit"
          disabled={submitting}
          className="px-4 py-2 rounded-lg bg-orange-500 text-white hover:bg-orange-600 transition-colors
duration-200 disabled:opacity-50"
        >
          {submitting ? "Submitting..." : "Submit Request"}
        </button>
      </div>


      {attachments.length > 0 && (
        <div className="mt-3 text-sm text-gray-700">
          <p className="font-medium mb-1">Selected documents:</p>
          <ul className="space-y-2">
            {attachments.map((file, index) => (
              <li
                key={index}
                className="flex items-center justify-between bg-gray-50 px-3 py-2 rounded"
```

```
              >
                <span>{file.name}</span>
                <button
                  type="button"
                  onClick={() => removeFile(index)}
                  className="text-red-500 hover:text-red-700 text-sm"
                >
                  Remove
                </button>
              </li>
            ))}
          </ul>
        </div>
    )}
    </form>
  );
};

export default ApplyLeaveForm;
```

File: frontend\components\ApprovalRequestCard.tsx

```
'use client';

import React from 'react';
import {
  Building2,
  CalendarDays,
  Clock3,
  Paperclip,
  CircleUserRound,
  CheckCircle2,
} from 'lucide-react';

export interface ApprovalRequest {
  id: number;
  employeeName: string;
  employeeCode: string;
  department: string;
  role: string;
  leaveType: string;
  startDate: string;
  endDate: string;
  durationText: string;
  hasAttachment: boolean;
  attachmentUrls: string[];
  appliedOn: string;
  reason: string;
  balances: {
    annual: string;
    casual: string;
  };
}

interface Props {
  request: ApprovalRequest;
  onReview: () => void;
}

export default function ApprovalRequestCard({ request, onReview }: Props) {
  return (
    <div className="rounded-[18px] border border-[#E4E7EC] bg-white p-4 shadow-sm">
      <div className="flex items-start gap-3">
        <div className="flex h-11 w-11 items-center justify-center rounded-full bg-[#F2F4F7]">
          <CircleUserRound size={22} className="text-[#667085]" />
        </div>

        <div>
          <h3 className="text-[16px] font-semibold text-[#1F2937]">
```

```
            {request.employeeName}
          </h3>
          <p className="text-[14px] text-[#98A2B3]">{request.employeeCode}</p>
        </div>
      </div>

      <div className="mt-4 space-y-3">
        <div className="flex items-start gap-2">
          <Building2 size={16} className="mt-0.5 text-[#667085]" />
          <div>
            <p className="text-[14px] text-[#344054]">{request.department}</p>
            <p className="text-[14px] text-[#667085]">{request.role}</p>
          </div>
        </div>

        <div>
          <span className="inline-flex rounded-full bg-[#F2924E] px-3 py-1 text-[13px] font-medium text-white">
            {request.leaveType}
          </span>
        </div>

        <div className="flex items-start gap-2">
          <CalendarDays size={16} className="mt-0.5 text-[#667085]" />
          <div>
            <p className="text-[14px] text-[#667085]">Date Range</p>
            <p className="text-[14px] text-[#344054]">
              {request.startDate} - {request.endDate}
            </p>
          </div>
        </div>

        <div className="flex items-start gap-2">
          <Clock3 size={16} className="mt-0.5 text-[#667085]" />
          <div>
            <p className="text-[14px] text-[#667085]">Duration</p>
            <p className="text-[14px] text-[#344054]">{request.durationText}</p>
          </div>
        </div>

        {request.hasAttachment && (
          <div className="flex items-center gap-2">
            <Paperclip size={16} className="text-[#667085]" />
            <p className="text-[14px] text-[#344054]">Has attachment</p>
          </div>
        )}
      </div>

      <button
        onClick={onReview}
        className="mt-5 flex h-[42px] w-full items-center justify-center gap-2 rounded-[12px] bg-[#667085]
text-[16px] font-medium text-white"
      >
        <CheckCircle2 size={17} />
        Review
      </button>
    </div>
  );
}
```

File: frontend\components\ApprovalReviewModal.tsx

```
'use client';

import React, { useEffect, useState } from 'react';
import { X, CircleUserRound, Building2, Check, Info, Paperclip, ExternalLink, FileText, Image } from '
lucide-react';
import { ApprovalRequest } from './ApprovalRequestCard';

export type ModalMode = 'review' | 'approve' | 'reject' | 'info';
```

```
interface Props {
  request: ApprovalRequest;
  mode: ModalMode;
  onClose: () => void;
  onApprove: () => void;
  onReject: () => void;
  onReqInfo: () => void;
  onCancelAction: () => void;
  onConfirmApprove: (comment: string) => void;
  onConfirmReject: (reason: string) => void;
  onSendInfoRequest: (message: string) => void;
  isSubmitting?: boolean;
}

export default function ApprovalReviewModal({
  request,
  mode,
  onClose,
  onApprove,
  onReject,
  onReqInfo,
  onCancelAction,
  onConfirmApprove,
  onConfirmReject,
  onSendInfoRequest,
  isSubmitting = false,
}: Props) {
  const [approveComment, setApproveComment] = useState('');
  const [rejectReason, setRejectReason] = useState('');
  const [infoMessage, setInfoMessage] = useState('');
  const [validationError, setValidationError] = useState('');

  useEffect(() => {
    setApproveComment('');
    setRejectReason('');
    setInfoMessage('');
    setValidationError('');
  }, [mode, request]);

  const handleApproveSubmit = () => {
    if (isSubmitting) return;
    onConfirmApprove(approveComment.trim());
  };

  const handleRejectSubmit = () => {
    const trimmedReason = rejectReason.trim();
    if (!trimmedReason) {
      setValidationError('Rejection reason is required.');
      return;
    }
    if (isSubmitting) return;
    setValidationError('');
    onConfirmReject(trimmedReason);
  };

  const handleInfoSubmit = () => {
    const trimmedMessage = infoMessage.trim();
    if (!trimmedMessage) {
      setValidationError('Please enter the required information message.');
      return;
    }
    if (isSubmitting) return;
    setValidationError('');
    onSendInfoRequest(trimmedMessage);
  };

  return (
    <div className="fixed inset-0 z-[100] flex items-center justify-center bg-black/35 px-4">
      <div className="relative w-full max-w-[560px] rounded-[20px] bg-white shadow-2xl flex flex-col
max-h-[90vh]">
```

```
        {/* ■■ Fixed header ■■ */}
        <div className="p-5 md:p-6 pb-4 flex-shrink-0 border-b border-[#F2F4F7]">
          <button
            onClick={() => { if (!isSubmitting) onClose(); }}
            disabled={isSubmitting}
            className="absolute right-5 top-5 text-[#98A2B3] disabled:cursor-not-allowed disabled:opacity-50"
          >
            <X size={24} />
          </button>
          <h2 className="text-[22px] font-bold text-[#1F2937]">Leave Request Review</h2>
          <p className="mt-1 text-[14px] text-[#667085]">
            Review the details and take action on this leave request.
          </p>
        </div>

        {/* ■■ Scrollable body ■■ */}
        <div className="overflow-y-auto flex-1 px-5 md:px-6 py-5 space-y-6">

          {/* Employee info */}
          <div className="flex items-start justify-between gap-4">
            <div className="flex items-start gap-3">
              <div className="flex h-12 w-12 items-center justify-center rounded-full bg-[#F2F4F7]">
                <CircleUserRound size={24} className="text-[#667085]" />
              </div>
              <div>
                <div className="flex items-center gap-1">
                  <h3 className="text-[18px] font-semibold text-[#1F2937]">{request.employeeName}</h3>
                  <span className="text-[14px] text-[#98A2B3]">{request.employeeCode}</span>
                </div>
                <div className="mt-1 flex items-start gap-2">
                  <Building2 size={14} className="mt-1 text-[#667085]" />
                  <div>
                    <p className="text-[14px] text-[#475467]">{request.department}</p>
                    <p className="text-[14px] text-[#667085]">{request.role}</p>
                  </div>
                </div>
              </div>
            </div>

            <div className="min-w-[205px]">
              <p className="mb-2 text-center text-[14px] text-[#667085]">Current Balance</p>
              <div className="flex gap-2">
                <div className="rounded-md bg-[#FFF7ED] px-3 py-2 text-center">
                  <p className="text-[14px] text-[#6B7280]">Annual Leaves</p>
                  <p className="text-[16px] font-bold text-[#1F2937]">{request.balances.annual}</p>
                </div>
                <div className="rounded-md bg-[#FFF7ED] px-3 py-2 text-center">
                  <p className="text-[14px] text-[#6B7280]">Casual Leaves</p>
                  <p className="text-[16px] font-bold text-[#1F2937]">{request.balances.casual}</p>
                </div>
              </div>
            </div>
          </div>

          {/* Leave details */}
          <div>
            <h4 className="text-[18px] font-bold text-[#1F2937] mb-4">Leave Request Details</h4>
            <div className="space-y-4">
              <DetailRow
                label="Leave Type"
                value={
                  <span className="inline-flex rounded-full bg-[#F2924E] px-4 py-1 text-[14px] font-medium
text-white">
                    {request.leaveType}
                  </span>
                }
              />
              <DetailRow
                label="Period"
                value={<span>{request.startDate} - {request.endDate}</span>}
              />
              <DetailRow
```

```
              label="Total Duration"
              value={<span className="font-semibold text-[#F2924E]">{request.durationText}</span>}
            />
            <DetailRow label="Applied On" value={<span>{request.appliedOn}</span>} />
            <DetailRow label="Reason" value={<span>{request.reason}</span>} />

            {/* Attachments */}
            {request.attachmentUrls && request.attachmentUrls.length > 0 && (
              <div className="grid grid-cols-[120px_1fr] gap-3">
                <p className="text-[16px] text-[#475467] flex items-center gap-1">
                  <Paperclip size={14} />
                  Attachments :
                </p>
                <div className="flex flex-col gap-2">
                  {request.attachmentUrls.map((url, idx) => {
                    const fullUrl = url.startsWith('http')
                      ? url
                      : `http://127.0.0.1:8000${url}`;
                    const filename = url.split('/').pop() || `file-${idx + 1}`;
                    const isImage = /\.(jpg|jpeg|png|gif|webp)$/i.test(filename);
                    const isPdf = /\.pdf$/i.test(filename);

                    return (
                      <div
                        key={idx}
                        className="rounded-[10px] border border-[#E4E7EC] bg-[#F9FAFB] p-3"
                      >
                        <div className="flex items-center justify-between gap-3">
                          <div className="flex items-center gap-2 min-w-0">
                            {isImage ? (
                              <Image size={16} className="text-[#F2924E] flex-shrink-0" />
                            ) : isPdf ? (
                              <FileText size={16} className="text-red-500 flex-shrink-0" />
                            ) : (
                              <Paperclip size={16} className="text-[#667085] flex-shrink-0" />
                            )}
                            <span className="text-[13px] text-[#344054] truncate">{filename}</span>
                          </div>
                          <a
                            href={fullUrl}
                            target="_blank"
                            rel="noopener noreferrer"
                            className="inline-flex items-center gap-1 flex-shrink-0 rounded-md bg-[#F2924E]
px-2.5 py-1 text-[12px] font-medium text-white hover:bg-orange-600 transition-colors"
                          >
                            <ExternalLink size={12} />
                            Open
                          </a>
                        </div>

                        {isImage && (
                          <div className="mt-2 rounded-md overflow-hidden border border-[#E4E7EC]">
                            <img
                              src={fullUrl}
                              alt={filename}
                              className="w-full max-h-48 object-contain bg-white"
                            />
                          </div>
                        )}
                      </div>
                    );
                  })}
                </div>
              </div>
            )}
          </div>
        </div>

        {/* Conditional action textareas */}
        {mode === 'approve' && (
          <div>
            <label className="mb-3 block text-[16px] text-[#1F2937]">Comments (Optional)</label>
```

```
              <textarea
                value={approveComment}
                onChange={(e) => setApproveComment(e.target.value)}
                placeholder="Add any comment about this approval."
                disabled={isSubmitting}
                className="h-[112px] w-full resize-none rounded-[14px] border border-[#D0D5DD] bg-[#F9FAFB]
px-4 py-3 text-[15px] text-[#344054] placeholder:text-[#98A2B3] focus:outline-none disabled:cursor-not-allowed
disabled:opacity-70"
              />
            </div>
          )}

          {mode === 'reject' && (
            <div>
              <label className="mb-3 block text-[16px] text-[#1F2937]">Rejection Reason</label>
              <textarea
                value={rejectReason}
                onChange={(e) => {
                  setRejectReason(e.target.value);
                  if (validationError) setValidationError('');
                }}
                placeholder="Please provide a clear reason for rejection."
                disabled={isSubmitting}
                className="h-[112px] w-full resize-none rounded-[14px] border border-[#D0D5DD] bg-[#F9FAFB]
px-4 py-3 text-[15px] text-[#344054] placeholder:text-[#98A2B3] focus:outline-none disabled:cursor-not-allowed
disabled:opacity-70"
              />
            </div>
          )}

          {mode === 'info' && (
            <div>
              <label className="mb-3 block text-[16px] text-[#1F2937]">What information do you need?</label>
              <textarea
                value={infoMessage}
                onChange={(e) => {
                  setInfoMessage(e.target.value);
                  if (validationError) setValidationError('');
                }}
                placeholder="Specify what additional information you need."
                disabled={isSubmitting}
                className="h-[112px] w-full resize-none rounded-[14px] border border-[#D0D5DD] bg-[#F9FAFB]
px-4 py-3 text-[15px] text-[#344054] placeholder:text-[#98A2B3] focus:outline-none disabled:cursor-not-allowed
disabled:opacity-70"
              />
            </div>
          )}

          {validationError && (mode === 'reject' || mode === 'info') && (
            <p className="text-[14px] font-medium text-red-600">{validationError}</p>
          )}

        </div>{/* end scrollable body */}

        {/* ■■ Fixed footer: action buttons ■■ */}
        <div className="px-5 md:px-6 py-4 border-t border-[#F2F4F7] flex-shrink-0">
          {mode === 'review' && (
            <div className="grid grid-cols-3 gap-3">
              <button
                onClick={onApprove}
                disabled={isSubmitting}
                className="flex h-[48px] items-center justify-center gap-2 rounded-[12px] bg-[#F2924E]
text-[16px] font-medium text-white disabled:cursor-not-allowed disabled:opacity-70"
              >
                <Check size={18} />
                Approve
              </button>
              <button
                onClick={onReject}
                disabled={isSubmitting}
                className="flex h-[48px] items-center justify-center gap-2 rounded-[12px] bg-[#98A2B3]
text-[16px] font-medium text-white disabled:cursor-not-allowed disabled:opacity-70"
```

```
              >
                <X size={18} />
                Reject
              </button>
              <button
                onClick={onReqInfo}
                disabled={isSubmitting}
                className="flex h-[48px] items-center justify-center gap-2 rounded-[12px] border
border-[#D0D5DD] bg-white text-[16px] font-medium text-[#344054] disabled:cursor-not-allowed
disabled:opacity-70"
              >
                <Info size={18} />
                Req Info
              </button>
            </div>
          )}

          {mode === 'approve' && (
            <div className="grid grid-cols-2 gap-3">
              <button
                onClick={handleApproveSubmit}
                disabled={isSubmitting}
                className="h-[48px] rounded-[12px] bg-[#F2924E] text-[16px] font-medium text-white
disabled:cursor-not-allowed disabled:opacity-70"
              >
                {isSubmitting ? 'Processing...' : 'Confirm Approval'}
              </button>
              <button
                onClick={onCancelAction}
                disabled={isSubmitting}
                className="h-[48px] rounded-[12px] bg-[#98A2B3] text-[16px] font-medium text-white
disabled:cursor-not-allowed disabled:opacity-70"
              >
                Cancel
              </button>
            </div>
          )}

          {mode === 'reject' && (
            <div className="grid grid-cols-2 gap-3">
              <button
                onClick={handleRejectSubmit}
                disabled={isSubmitting}
                className="h-[48px] rounded-[12px] bg-[#F2924E] text-[16px] font-medium text-white
disabled:cursor-not-allowed disabled:opacity-70"
              >
                {isSubmitting ? 'Processing...' : 'Confirm Rejection'}
              </button>
              <button
                onClick={onCancelAction}
                disabled={isSubmitting}
                className="h-[48px] rounded-[12px] bg-[#98A2B3] text-[16px] font-medium text-white
disabled:cursor-not-allowed disabled:opacity-70"
              >
                Cancel
              </button>
            </div>
          )}

          {mode === 'info' && (
            <div className="grid grid-cols-2 gap-3">
              <button
                onClick={handleInfoSubmit}
                disabled={isSubmitting}
                className="h-[48px] rounded-[12px] bg-[#F2924E] text-[16px] font-medium text-white
disabled:cursor-not-allowed disabled:opacity-70"
              >
                {isSubmitting ? 'Processing...' : 'Send Request'}
              </button>
              <button
                onClick={onCancelAction}
                disabled={isSubmitting}
```

```
                className="h-[48px] rounded-[12px] bg-[#98A2B3] text-[16px] font-medium text-white
disabled:cursor-not-allowed disabled:opacity-70"
              >
                Cancel
              </button>
            </div>
          )}
        </div>{/* end footer */}

      </div>
    </div>
  );
}

function DetailRow({
  label,
  value,
}: {
  label: string;
  value: React.ReactNode;
}) {
  return (
    <div className="grid grid-cols-[120px_1fr] gap-3">
      <p className="text-[16px] text-[#475467]">{label} :</p>
      <div className="text-[16px] text-[#1F2937]">{value}</div>
    </div>
  );
}
```

File: frontend\components\BalanceCard.tsx

```
import React from 'react';

interface BalanceCardProps {
  type: string;
  remaining: number;
  total: number;
  colorClass?: string; // tailwind background color
}

const BalanceCard: React.FC<BalanceCardProps> = ({ type, remaining, total, colorClass = 'bg-[#FFF3E6]
border-[#F5C28B]' }) => {
  return (
    <div className={`rounded-xl p-5 shadow-sm border ${colorClass} flex flex-col`}>
      <span className="text-sm text-gray-600 mb-1">{type}</span>
      <span className="text-2xl font-bold text-gray-800">
        {remaining} / {total}
      </span>
    </div>
  );
};

export default BalanceCard;
```

File: frontend\components\EditLeaveModal.tsx

```
'use client';

import React, { useEffect, useState } from 'react';
import { X, UploadCloud, File as FileIcon, Loader2, Trash2 } from 'lucide-react';
import { API_BASE_URL, getAuthHeaders } from '../app/lib/api';
import LeaveDatePicker from './LeaveDatePicker';

interface LeaveRequest {
  leave_request_id: number;
  employee_id: number;
  leave_type_id: number;
```

```
  leave_type_name?: string | null;
  start_date: string;
  end_date: string;
  total_days: number;
  half_day: boolean;
  status: string;
  reason?: string | null;
  attachment_urls?: string[];
}

interface LeaveType {
  id: number;
  name: string;
}

interface EditLeaveModalProps {
  request: LeaveRequest;
  onClose: () => void;
  onSuccess: () => void;
  onDelete: () => void;
}

export default function EditLeaveModal({
  request,
  onClose,
  onSuccess,
  onDelete,
}: EditLeaveModalProps) {
  const [leaveTypes, setLeaveTypes] = useState<LeaveType[]>([]);
  const [leaveTypeId, setLeaveTypeId] = useState<string>(String(request.leave_type_id));
  const [dateSelection, setDateSelection] = useState({
    startDate: request.start_date,
    endDate: request.end_date,
    halfDay: request.half_day,
  });
  const [reason, setReason] = useState(request.reason || '');
  const [existingUrls, setExistingUrls] = useState<string[]>(request.attachment_urls || []);
  const [newAttachments, setNewAttachments] = useState<File[]>([]);

  const [errors, setErrors] = useState<string[]>([]);
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [isDragging, setIsDragging] = useState(false);

  // Derived values
  const fromDate = dateSelection.startDate;
  const toDate = dateSelection.endDate;
  const halfDay = dateSelection.halfDay;

  useEffect(() => {
    const fetchLeaveTypes = async () => {
      try {
        const res = await fetch(`${API_BASE_URL}/leave/types`);
        if (res.ok) {
          const data = await res.json();
          setLeaveTypes(data);
        }
      } catch (err) {
        console.error("Failed to fetch leave types", err);
      }
    };
    fetchLeaveTypes();
  }, []);

  const handleFileChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    if (e.target.files && e.target.files.length > 0) {
      setNewAttachments(prev => [...prev, ...Array.from(e.target.files!)]);
      setErrors([]);
    }
  };

  const removeNewFile = (index: number) => {
    setNewAttachments(prev => prev.filter((_, i) => i !== index));
```

```
};

const removeExistingFile = (index: number) => {
  setExistingUrls(prev => prev.filter((_, i) => i !== index));
};

const validate = () => {
  const errs: string[] = [];
  const selectedLeaveType = leaveTypes.find(t => String(t.id) === leaveTypeId);

  if (!leaveTypeId) errs.push("Leave type is required");
  if (!fromDate) errs.push("From date is required");
  if (!toDate) errs.push("To date is required");
  if (fromDate && toDate && new Date(toDate) < new Date(fromDate)) {
    errs.push("To date cannot be earlier than From date");
  }
  if (!reason.trim()) errs.push("Reason is required");

  if (
    selectedLeaveType?.name.toLowerCase().includes("medical") &&
    existingUrls.length === 0 && newAttachments.length === 0
  ) {
    errs.push("Medical leave requires a supporting document");
  }

  setErrors(errs);
  return errs.length === 0;
};

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setErrors([]);

  if (!validate()) return;

  setIsSubmitting(true);

  try {
    let uploadedFileUrls: string[] = [...existingUrls];

    if (newAttachments.length > 0) {
      for (const file of newAttachments) {
        const formData = new FormData();
        formData.append('file', file);

        const headers = getAuthHeaders() as Record<string, string>;
        delete headers['Content-Type'];

        const uploadRes = await fetch(`${API_BASE_URL}/leave/upload`, {
          method: 'POST',
          headers,
          body: formData,
        });

        if (!uploadRes.ok) {
          throw new Error(`Failed to upload ${file.name}`);
        }
        const uploadData = await uploadRes.json();
        uploadedFileUrls.push(uploadData.file_url);
      }
    }

    const payload = {
      leave_type_id: parseInt(leaveTypeId),
      start_date: fromDate,
      end_date: toDate,
      half_day: halfDay,
      reason: reason.trim(),
      attachment_urls: uploadedFileUrls,
    };

    const updateRes = await fetch(`${API_BASE_URL}/leave/requests/${request.leave_request_id}`, {
```

```
        method: 'PUT',
        headers: getAuthHeaders(),
        body: JSON.stringify(payload),
      });

      if (!updateRes.ok) {
        const errorData = await updateRes.json();
        throw new Error(errorData.detail || 'Failed to update request');
      }

      onSuccess();
    } catch (err: any) {
      console.error(err);
      setErrors([err.message || 'Something went wrong. Please try again.']);
      setIsSubmitting(false);
    }
  };

  return (
    <div className="fixed inset-0 z-[110] flex items-center justify-center bg-black/40 px-4 py-8
backdrop-blur-sm overflow-y-auto">
      <div className="relative w-full max-w-[600px] rounded-2xl bg-white shadow-xl p-6 flex flex-col my-auto">

        <div className="flex items-center justify-between mb-6">
          <h2 className="text-xl font-bold text-slate-800">Edit Leave Request</h2>
          <button
            type="button"
            onClick={() => !isSubmitting && onClose()}
            disabled={isSubmitting}
            className="text-slate-400 hover:text-slate-600 transition-colors disabled:opacity-50"
          >
            <X size={22} />
          </button>
        </div>

        <form onSubmit={handleSubmit} className="flex flex-col flex-1 overflow-hidden max-h-[75vh]">

          <div className="flex-1 pr-2 custom-scrollbar overflow-y-auto space-y-5 pb-2">
            <div>
              <label className="block text-sm text-slate-500 mb-2">Leave Type</label>
              <select
                value={leaveTypeId}
                onChange={(e) => setLeaveTypeId(e.target.value)}
                className="w-full px-4 py-2.5 border border-gray-200 shadow-sm rounded-lg bg-white
text-slate-700 text-sm focus:outline-none focus:ring-2 focus:ring-orange-50 focus:border-[#F2924E]"
              >
                <option value="" disabled>Select Leave Type</option>
                {leaveTypes.map((type) => (
                  <option key={type.id} value={type.id}>
                    {type.name}
                  </option>
                ))}
              </select>
            </div>

            <div>
              <label className="block text-sm text-slate-500 mb-2">Date Range</label>
              <LeaveDatePicker
                value={dateSelection}
                onChange={setDateSelection}
              />
            </div>

            <div>
              <label className="block text-sm text-slate-500 mb-2">Reason</label>
              <textarea
                value={reason}
                onChange={(e) => setReason(e.target.value)}
                className="w-full px-4 py-3 border border-gray-200 shadow-sm rounded-lg text-slate-700 text-sm
focus:outline-none focus:ring-2 focus:ring-orange-50 focus:border-[#F2924E] resize-none h-24"
                placeholder="Why do you need this leave?"
              />
```

```
            </div>

            <div>
              <label className="block text-sm text-slate-500 mb-2">Attachments</label>

              <div
                className={`border border-dashed rounded-md transition-colors cursor-pointer group relative
mb-3 ${isDragging ? 'border-[#F2924E] bg-orange-50/50' : 'border-slate-300 bg-slate-50/50
hover:bg-slate-100/50'}`}
                onDragOver={(e) => { e.preventDefault(); setIsDragging(true); }}
                onDragLeave={() => setIsDragging(false)}
                onDrop={(e) => {
                  e.preventDefault();
                  setIsDragging(false);
                  if (e.dataTransfer.files) {
                    setNewAttachments(prev => [...prev, ...Array.from(e.dataTransfer.files)]);
                  }
                }}
              >
                <input
                  type="file"
                  multiple
                  onChange={handleFileChange}
                  className="absolute inset-0 w-full h-full opacity-0 cursor-pointer z-10"
                />
                <div className="py-8 px-4 text-center flex flex-col items-center">
                  <div className="w-10 h-10 bg-white border border-slate-100 rounded-full shadow-sm flex
items-center justify-center mb-3 group-hover:scale-105 transition-transform">
                    <UploadCloud className="text-[#F2924E]" size={20} />
                  </div>
                  <p className="text-sm text-slate-600">Click or drag files to upload</p>
                </div>
              </div>

              {(existingUrls.length > 0 || newAttachments.length > 0) && (
                <div className="space-y-2">
                  {existingUrls.map((url, i) => (
                    <div key={`old-${i}`} className="flex items-center justify-between p-3 bg-white border
border-gray-200 rounded-lg shadow-sm">
                      <div className="flex items-center gap-3 overflow-hidden">
                        <FileIcon size={16} className="text-orange-400 flex-shrink-0" />
                        <span className="text-sm text-gray-600 truncate">{url.split('/').pop()} (previously
uploaded)</span>
                      </div>
                      <button type="button" onClick={() => removeExistingFile(i)} className="text-gray-400
hover:text-red-500 p-1">
                        <X size={16} />
                      </button>
                    </div>
                  ))}
                  {newAttachments.map((f, i) => (
                    <div key={`new-${i}`} className="flex items-center justify-between p-3 bg-white border
border-gray-200 rounded-lg shadow-sm">
                      <div className="flex items-center gap-3 overflow-hidden">
                        <FileIcon size={16} className="text-orange-500 flex-shrink-0" />
                        <span className="text-sm text-gray-900 truncate">{f.name}</span>
                      </div>
                      <button type="button" onClick={() => removeNewFile(i)} className="text-gray-400
hover:text-red-500 p-1">
                        <X size={16} />
                      </button>
                    </div>
                  ))}
                </div>
              )}
            </div>
          </div>

          {errors.length > 0 && (
            <div className="bg-red-50 border border-red-100 p-3 rounded-lg mb-4">
              <ul className="list-disc list-inside space-y-1">
                {errors.map((err, i) => (
```

```
                <li key={i} className="text-sm text-red-600 font-medium">{err}</li>
              ))}
            </ul>
          </div>
        )}

        <div className="pt-4 mt-2 shrink-0 flex flex-col gap-3 bg-white border-t border-slate-100">
          <div className="grid grid-cols-2 gap-3">
            <button
              type="button"
              onClick={onClose}
              disabled={isSubmitting}
              className="py-2.5 px-4 bg-slate-100 hover:bg-slate-200 text-slate-700 font-medium rounded-md
transition-colors disabled:opacity-50"
            >
              Cancel
            </button>
            <button
              type="submit"
              disabled={isSubmitting}
              className="flex items-center justify-center gap-2 py-2.5 px-4 bg-[#F2924E] hover:bg-[#e07b34]
text-white font-medium rounded-md transition-colors shadow-sm disabled:opacity-70"
            >
              {isSubmitting ? (
                <><Loader2 size={18} className="animate-spin" /> Saving</>
              ) : (
                'Save Changes'
              )}
            </button>
          </div>

          <button
            type="button"
            onClick={onDelete}
            disabled={isSubmitting}
            className="w-full py-2.5 px-4 bg-white hover:bg-red-50 text-red-600 font-medium rounded-md border
border-red-200 transition-colors disabled:opacity-50 flex items-center justify-center gap-2"
          >
            <Trash2 size={18} />
            Cancel & Delete Request
          </button>
        </div>
      </form>
    </div>
  </div>
  );
}
```

File: frontend\components\LeaveBalanceCards.tsx

```
import React from 'react';

interface Props {
  balances: Record<string, number>;
}

export default function LeaveBalanceCards({ balances }: Props) {
  return (
    <div className="grid grid-cols-2 gap-6 mb-6">
      {Object.entries(balances)
        // don't display sick leave on this page
        .filter(([type]) => type.toLowerCase() !== 'sick')
        .map(([type, amount]) => (
          <div
            key={type}
            className="bg-[#FFF3E6] border border-[#F5C28B] rounded-xl p-5"
          >
            <p className="text-sm text-gray-600 mb-1">
              {type} Leaves
```

```
            </p>
            <h2 className="text-2xl font-bold text-gray-800">
              {amount} days
            </h2>
          </div>
        ))}
    </div>
  );
}
```

File: frontend\components\LeaveDatePicker.tsx

```
"use client";

import React, { useEffect, useRef, useState } from "react";

interface DateSelection {
  startDate: string; // YYYY-MM-DD
  endDate: string;   // YYYY-MM-DD
  halfDay: boolean;
}

interface Props {
  onChange: (selection: DateSelection) => void;
  value: DateSelection;
}

const DAYS_OF_WEEK = ["Mo", "Tu", "We", "Th", "Fr", "Sa", "Su"];
const MONTHS = [
  "Jan", "Feb", "Mar", "Apr", "May", "Jun",
  "Jul", "Aug", "Sep", "Oct", "Nov", "Dec",
];

function toYMD(date: Date): string {
  return date.toISOString().split("T")[0];
}

function parseYMD(str: string): Date {
  const [y, m, d] = str.split("-").map(Number);
  return new Date(y, m - 1, d);
}

function formatDisplay(start: string, end: string, halfDay: boolean): string {
  if (!start) return "";
  const s = parseYMD(start);
  const fmt = (d: Date) =>
    `${d.getDate()} ${MONTHS[d.getMonth()]} ${d.getFullYear()}`;

  if (halfDay || start === end) return `${fmt(s)} (${halfDay ? "Half Day" : "1 day"})`;
  const e = parseYMD(end);
  const diff =
    Math.ceil((e.getTime() - s.getTime()) / (1000 * 60 * 60 * 24)) + 1;
  return `${fmt(s)} → ${fmt(e)} (${diff} days)`;
}

export default function LeaveDatePicker({ onChange, value }: Props) {
  const [open, setOpen] = useState(false);
  const [viewYear, setViewYear] = useState(() => new Date().getFullYear());
  const [viewMonth, setViewMonth] = useState(() => new Date().getMonth());
  const [selecting, setSelecting] = useState<"start" | "end">("start");
  const [hoverDate, setHoverDate] = useState<string | null>(null);
  const wrapperRef = useRef<HTMLDivElement>(null);
  const today = toYMD(new Date());

  // Close on outside click
  useEffect(() => {
    const handler = (e: MouseEvent) => {
      if (wrapperRef.current && !wrapperRef.current.contains(e.target as Node)) {
        setOpen(false);
```

```
    }
  };
  document.addEventListener("mousedown", handler);
  return () => document.removeEventListener("mousedown", handler);
}, []);

// Build grid for current month view
const firstDay = new Date(viewYear, viewMonth, 1);
// Monday-based: Mon=0 … Sun=6
const startOffset = (firstDay.getDay() + 6) % 7;
const daysInMonth = new Date(viewYear, viewMonth + 1, 0).getDate();

const cells: (number | null)[] = [
  ...Array(startOffset).fill(null),
  ...Array.from({ length: daysInMonth }, (_, i) => i + 1),
];
// Pad to full weeks
while (cells.length % 7 !== 0) cells.push(null);

const prevMonth = () => {
  if (viewMonth === 0) { setViewMonth(11); setViewYear((y) => y - 1); }
  else setViewMonth((m) => m - 1);
};
const nextMonth = () => {
  if (viewMonth === 11) { setViewMonth(0); setViewYear((y) => y + 1); }
  else setViewMonth((m) => m + 1);
};

const handleDayClick = (day: number) => {
  const clicked = toYMD(new Date(viewYear, viewMonth, day));
  if (clicked < today) return; // past — blocked

  if (value.halfDay) {
    // Half day: always single date
    onChange({ startDate: clicked, endDate: clicked, halfDay: true });
    setOpen(false);
    return;
  }

  if (selecting === "start") {
    onChange({ startDate: clicked, endDate: clicked, halfDay: false });
    setSelecting("end");
  } else {
    if (clicked < value.startDate) {
      // Clicked before start → reset as new start
      onChange({ startDate: clicked, endDate: clicked, halfDay: false });
      setSelecting("end");
    } else {
      onChange({ startDate: value.startDate, endDate: clicked, halfDay: false });
      setSelecting("start");
      setOpen(false);
    }
  }
};

const handleHalfDayToggle = () => {
  const next = !value.halfDay;
  if (next && value.startDate) {
    onChange({ startDate: value.startDate, endDate: value.startDate, halfDay: true });
  } else {
    onChange({ ...value, halfDay: next });
  }
  setSelecting("start");
};

const getCellStyle = (day: number): string => {
  const d = toYMD(new Date(viewYear, viewMonth, day));
  const isPast = d < today;
  const isStart = d === value.startDate;
  const isEnd = d === value.endDate;
  const effectiveEnd = hoverDate && selecting === "end" ? hoverDate : value.endDate;
  const inRange =
```

```
      value.startDate &&
      effectiveEnd &&
      d > value.startDate &&
      d < effectiveEnd;

    if (isPast)
      return "text-gray-300 cursor-not-allowed";
    if (isStart || isEnd)
      return "bg-[#F2924E] text-white rounded-full font-semibold cursor-pointer";
    if (inRange)
      return "bg-orange-100 text-orange-800 rounded-full cursor-pointer";
    return "text-gray-700 hover:bg-orange-50 rounded-full cursor-pointer";
  };

  const displayText = value.startDate
    ? formatDisplay(value.startDate, value.endDate, value.halfDay)
    : "";

  return (
    <div ref={wrapperRef} className="relative w-full">
      {/* Trigger */}
      <div
        onClick={() => setOpen((o) => !o)}
        className={`flex items-center justify-between w-full border border-gray-200 shadow-sm rounded-lg px-4
py-2.5 bg-white cursor-pointer hover:border-[#F2924E] transition-colors ${open ? 'ring-2 ring-orange-50
border-[#F2924E]' : ''}`}
      >
        <span className={`text-sm ${displayText ? "text-slate-700" : "text-slate-400"}`}>
          {displayText || "Pick the date :"}
        </span>
        {/* Calendar icon */}
        <svg
          className="w-5 h-5 text-gray-400 flex-shrink-0"
          fill="none"
          stroke="currentColor"
          viewBox="0 0 24 24"
        >
          <rect x="3" y="4" width="18" height="18" rx="2" ry="2" strokeWidth="2" />
          <line x1="16" y1="2" x2="16" y2="6" strokeWidth="2" strokeLinecap="round" />
          <line x1="8" y1="2" x2="8" y2="6" strokeWidth="2" strokeLinecap="round" />
          <line x1="3" y1="10" x2="21" y2="10" strokeWidth="2" />
        </svg>
      </div>

      {/* Calendar popup */}
      {open && (
        <div className="absolute z-50 top-full mt-2 left-0 bg-white rounded-xl shadow-xl border border-gray-100
w-72 p-4 select-none">

          {/* Month navigation */}
          <div className="flex items-center justify-between mb-4">
            <div>
              <p className="font-bold text-sm text-gray-800">{MONTHS[viewMonth]}</p>
              <p className="font-bold text-sm text-gray-800">{viewYear}</p>
            </div>
            <div className="flex gap-2">
              <button
                onClick={prevMonth}
                className="text-[#F2924E] hover:text-orange-600 p-1 rounded transition-colors"
              >
                <svg className="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                  <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M15 19l-7-7 7-7" />
                </svg>
              </button>
              <button
                onClick={nextMonth}
                className="text-[#F2924E] hover:text-orange-600 p-1 rounded transition-colors"
              >
                <svg className="w-4 h-4" fill="none" stroke="currentColor" viewBox="0 0 24 24">
                  <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={2} d="M9 5l7 7-7 7" />
                </svg>
              </button>
```

```
            </div>
          </div>

          {/* Day-of-week headers */}
          <div className="grid grid-cols-7 mb-1">
            {DAYS_OF_WEEK.map((d) => (
              <div key={d} className="text-center text-xs font-medium text-gray-400 py-1">
                {d}
              </div>
            ))}
          </div>

          {/* Date cells */}
          <div className="grid grid-cols-7">
            {cells.map((day, i) =>
              day === null ? (
                <div key={i} />
              ) : (
                <div
                  key={i}
                  onClick={() => handleDayClick(day)}
                  onMouseEnter={() => {
                    const d = toYMD(new Date(viewYear, viewMonth, day));
                    if (selecting === "end") setHoverDate(d);
                  }}
                  onMouseLeave={() => setHoverDate(null)}
                  className={`flex items-center justify-center text-sm h-8 w-8 mx-auto transition-colors
${getCellStyle(day)}`}
                >
                  {day}
                </div>
              )
            )}
          </div>

          {/* Divider */}
          <div className="border-t border-gray-100 mt-3 pt-3">
            {/* Half Day toggle */}
            <div className="flex items-center justify-between">
              <span className="text-sm text-gray-600">Half Day</span>
              <button
                type="button"
                onClick={handleHalfDayToggle}
                className={`relative inline-flex h-6 w-11 items-center rounded-full transition-colors
duration-300 focus:outline-none ${
                  value.halfDay ? "bg-[#F2924E]" : "bg-gray-200"
                }`}
              >
                <span
                  className={`inline-block h-4 w-4 transform rounded-full bg-white shadow transition-transform
duration-300 ${
                    value.halfDay ? "translate-x-6" : "translate-x-1"
                  }`}
                />
              </button>
            </div>
          </div>
        </div>
      )}
    </div>
  );
}
```

File: frontend\components\LeaveForm.tsx

```
"use client";

import { useState } from "react";
```

```
export default function LeaveForm() {
  const [leaveType, setLeaveType] = useState("");
  const [fromDate, setFromDate] = useState("");
  const [toDate, setToDate] = useState("");
  const [reason, setReason] = useState("");

  const handleSubmit = () => {
    const data = {
      leaveType,
      fromDate,
      toDate,
      reason,
    };

    console.log("Leave Request:", data);
  };

  return (
    <div className="bg-white p-6 rounded-lg shadow space-y-4">
      {/* Leave Type */}
      {/*<div>
        <label className="block text-sm mb-1">Leave Type</label>
        <select
          value={leaveType}
          onChange={(e) => setLeaveType(e.target.value)}
          className="w-full border p-2 rounded"
        >
          <option value="">Select Type</option>
          <option value="annual">Annual</option>
          <option value="casual">Casual</option>
          <option value="medical">Medical</option>

        </select>
      </div> */}

      {/* Date Range */}
      <div className="grid grid-cols-2 gap-4">
        <div>
          <label className="block text-sm mb-1">From</label>
          <input
            type="date"
            value={fromDate}
            onChange={(e) => setFromDate(e.target.value)}
            className="w-full border p-2 rounded"
          />
        </div>

        <div>
          <label className="block text-sm mb-1">To</label>
          <input
            type="date"
            value={toDate}
            onChange={(e) => setToDate(e.target.value)}
            className="w-full border p-2 rounded"
          />
        </div>
      </div>

      {/* Reason */}
      <div>
        <label className="block text-sm mb-1">Reason</label>
        <textarea
          value={reason}
          onChange={(e) => setReason(e.target.value)}
          className="w-full border p-2 rounded"
          rows={4}
        />
      </div>

      {/* Buttons */}
      <div className="flex justify-end gap-3">
        <button className="px-4 py-2 border rounded">
```

```
          Cancel
        </button>

        <button
          onClick={handleSubmit}
          className="px-4 py-2 bg-orange-500 text-white rounded"
        >
          Submit Request
        </button>
      </div>
    </div>
  );
}
```

File: frontend\components\LeaveResubmitModal.tsx

```
'use client';

import React, { useState } from 'react';
import { X, UploadCloud, File as FileIcon, Loader2 } from 'lucide-react';
import { API_BASE_URL, getAuthHeaders } from '../app/lib/api';

interface LeaveRequest {
  leave_request_id: number;
  leave_type_name?: string | null;
  start_date: string;
  end_date: string;
  total_days: number;
  half_day: boolean;
  reason?: string | null;
  manager_comment?: string | null;
}

interface LeaveResubmitModalProps {
  request: LeaveRequest;
  onClose: () => void;
  onSuccess: () => void;
}

export default function LeaveResubmitModal({
  request,
  onClose,
  onSuccess,
}: LeaveResubmitModalProps) {
  const [attachments, setAttachments] = useState<File[]>([]);
  const [reason, setReason] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [error, setError] = useState('');

  const requestId = request.leave_request_id;
  const managerComment = request.manager_comment;

  const handleFileChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    if (e.target.files) {
      setAttachments(Array.from(e.target.files));
      setError('');
    }
  };

  const removeFile = (index: number) => {
    setAttachments(prev => prev.filter((_, i) => i !== index));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError('');

    if (attachments.length === 0 && !reason.trim()) {
      setError('Please provide a new document or a comment to resubmit.');
```

```
      return;
    }

    setIsSubmitting(true);

    try {
      let uploadedFileUrls: string[] = [];

      // Step 1: Upload new attachments if any
      if (attachments.length > 0) {
        for (const file of attachments) {
          const formData = new FormData();
          formData.append('file', file);

          const headers = getAuthHeaders() as Record<string, string>;
          delete headers['Content-Type'];

          const uploadRes = await fetch(`${API_BASE_URL}/leave/upload`, {
            method: 'POST',
            headers,
            body: formData,
          });

          if (!uploadRes.ok) {
            throw new Error(`Failed to upload ${file.name}`);
          }
          const uploadData = await uploadRes.json();
          uploadedFileUrls.push(uploadData.file_url);
        }
      }

      // Step 2: Call the resubmit patch endpoint
      const payload = {
        attachment_urls: uploadedFileUrls,
        reason: reason.trim() || undefined,
      };

      const resubmitRes = await fetch(`${API_BASE_URL}/leave/requests/${requestId}/resubmit`, {
        method: 'PATCH',
        headers: {
          ...getAuthHeaders(),
          'Content-Type': 'application/json',
        },
        body: JSON.stringify(payload),
      });

      if (!resubmitRes.ok) {
        const errorText = await resubmitRes.text();
        throw new Error(errorText || 'Failed to resubmit request');
      }

      onSuccess();
    } catch (err: any) {
      console.error(err);
      setError(err.message || 'Something went wrong. Please try again.');
      setIsSubmitting(false);
    }
  };

  return (
    <div className="fixed inset-0 z-[100] flex items-center justify-center bg-black/40 px-4 backdrop-blur-sm">
      <div className="relative w-full max-w-[500px] rounded-[24px] bg-white shadow-2xl p-6 md:p-8 flex flex-col
max-h-[90vh]">

        {/* Header */}
        <div className="flex items-center justify-between mb-2">
          <h2 className="text-xl font-bold text-gray-900">Action Required</h2>
          <button
            onClick={() => !isSubmitting && onClose()}
            disabled={isSubmitting}
            className="text-gray-400 hover:text-gray-600 transition-colors disabled:opacity-50"
          >
```

```
            <X size={24} />
          </button>
        </div>
        <p className="text-sm text-gray-500 mb-4">
          HR has requested additional information for Leave Request LR-{requestId}.
        </p>

        {/* Request Details Summary Table */}
        <div className="mb-6 overflow-hidden rounded-xl border border-gray-200 bg-gray-50/30">
          <table className="w-full text-left text-xs">
            <tbody className="divide-y divide-gray-100">
              <tr>
                <th className="px-4 py-2 font-semibold text-gray-500 w-1/3">Type</th>
                <td className="px-4 py-2 font-medium text-gray-900">{request.leave_type_name || "-"}</td>
              </tr>
              <tr>
                <th className="px-4 py-2 font-semibold text-gray-500">Dates</th>
                <td className="px-4 py-2 font-medium text-gray-900">
                  {new Date(request.start_date).toLocaleDateString()} - {new
Date(request.end_date).toLocaleDateString()}
                </td>
              </tr>
              <tr>
                <th className="px-4 py-2 font-semibold text-gray-500">Total Days</th>
                <td className="px-4 py-2 font-medium text-gray-900">
                  {request.half_day ? "0.5 days" : `${request.total_days} days`}
                </td>
              </tr>
              {request.reason && (
                <tr>
                  <th className="px-4 py-2 font-semibold text-gray-500 align-top">Original Reason</th>
                  <td className="px-4 py-2 font-medium text-gray-900 italic break-words">"{request.reason}"</td>
                </tr>
              )}
            </tbody>
          </table>
        </div>

        <form onSubmit={handleSubmit} className="flex-1 overflow-y-auto pr-2 custom-scrollbar space-y-6">

          {/* HR Comment Callout */}
          <div className="bg-orange-50 border border-orange-100 rounded-xl p-4">
            <span className="text-xs font-semibold text-orange-600 uppercase tracking-wider block mb-1">
              HR message
            </span>
            <p className="text-sm text-orange-900 leading-relaxed font-medium">
              "{managerComment || 'Please provide correct documents or clarify your request.'}"
            </p>
          </div>

          {/* Upload Section */}
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-2">
              Upload New Documents (Optional)
            </label>
            <div className="border-2 border-dashed border-gray-200 rounded-xl bg-gray-50 hover:bg-gray-100
transition-colors cursor-pointer group relative">
              <input
                type="file"
                multiple
                onChange={handleFileChange}
                className="absolute inset-0 w-full h-full opacity-0 cursor-pointer z-10"
              />
              <div className="py-8 px-4 text-center flex flex-col items-center">
                <div className="w-12 h-12 bg-white rounded-full shadow-sm flex items-center justify-center mb-3
group-hover:scale-105 transition-transform">
                  <UploadCloud className="text-orange-500" size={24} />
                </div>
                <p className="text-sm font-medium text-gray-700">Click to upload new files</p>
                <p className="text-xs text-gray-500 mt-1">PDF, JPG, PNG (Max 5MB)</p>
              </div>
            </div>
```

```
            {attachments.length > 0 && (
              <div className="mt-3 space-y-2">
                {attachments.map((file, index) => (
                  <div key={index} className="flex items-center justify-between p-3 bg-white border
border-gray-200 rounded-lg shadow-sm">
                    <div className="flex items-center gap-3 overflow-hidden">
                      <FileIcon size={16} className="text-orange-500 flex-shrink-0" />
                      <span className="text-sm text-gray-700 truncate">{file.name}</span>
                    </div>
                    <button
                      type="button"
                      onClick={() => removeFile(index)}
                      className="text-gray-400 hover:text-red-500 p-1"
                    >
                      <X size={16} />
                    </button>
                  </div>
                ))}
              </div>
            )}
          </div>

          {/* Reason Section */}
          <div>
            <label className="block text-sm font-medium text-gray-700 mb-2">
              Additional Comments (Optional)
            </label>
            <textarea
              className="w-full px-4 py-3 border border-gray-300 rounded-xl text-sm focus:outline-none
focus:ring-2 focus:ring-orange-200 resize-none"
              rows={3}
              placeholder="Clarify your leave request here..."
              value={reason}
              onChange={(e) => {
                setReason(e.target.value);
                if (error) setError('');
              }}
            />
          </div>

          {error && (
            <p className="text-sm text-red-600 font-medium bg-red-50 p-3 rounded-lg border border-red-100">
              {error}
            </p>
          )}

          {/* Action Buttons */}
          <div className="pt-4 grid grid-cols-2 gap-3 pb-2">
            <button
              type="button"
              onClick={onClose}
              disabled={isSubmitting}
              className="py-3 px-4 bg-gray-100 hover:bg-gray-200 text-gray-700 font-medium rounded-xl
transition-colors disabled:opacity-50"
            >
              Cancel
            </button>
            <button
              type="submit"
              disabled={isSubmitting}
              className="flex items-center justify-center gap-2 py-3 px-4 bg-[#F2924E] hover:bg-orange-500
text-white font-medium rounded-xl transition-colors disabled:opacity-70 shadow-sm"
            >
              {isSubmitting ? (
                <>
                  <Loader2 size={18} className="animate-spin" />
                  Submitting
                </>
              ) : (
                'Resubmit Request'
              )}
```

```
            </button>
          </div>
        </form>
      </div>
    </div>
  );
}
```

File: frontend\components\LeaveTable.tsx

```
import React from 'react';

export interface LeaveRecord {
  id: number;
  type: string;
  from: string; // yyyy-mm-dd
  to: string;
  days: number;
  status: 'Pending' | 'Approved' | 'Rejected';
}

interface LeaveTableProps {
  records: LeaveRecord[];
}

const statusColor = (status: string) => {
  switch (status) {
    case 'Pending':
      return 'bg-yellow-100 text-yellow-800';
    case 'Approved':
      return 'bg-green-100 text-green-800';
    case 'Rejected':
      return 'bg-red-100 text-red-800';
    default:
      return '';
  }
};

const LeaveTable: React.FC<LeaveTableProps> = ({ records }) => {
  return (
    <div className="overflow-x-auto">
      <table className="w-full table-auto border-collapse">
        <thead>
          <tr className="bg-gray-100">
            <th className="px-4 py-2 text-left text-xs font-medium text-gray-800">Type</th>
            <th className="px-4 py-2 text-left text-xs font-medium text-gray-800">From</th>
            <th className="px-4 py-2 text-left text-xs font-medium text-gray-800">To</th>
            <th className="px-4 py-2 text-left text-xs font-medium text-gray-800">Days</th>
            <th className="px-4 py-2 text-left text-xs font-medium text-gray-800">Status</th>
          </tr>
        </thead>
        <tbody>
          {records.map((r) => (
            <tr key={r.id} className="border-b">
              <td className="px-4 py-2 text-sm text-gray-800">{r.type}</td>
              <td className="px-4 py-2 text-sm text-gray-800">{r.from}</td>
              <td className="px-4 py-2 text-sm text-gray-800">{r.to}</td>
              <td className="px-4 py-2 text-sm text-gray-800">{r.days}</td>
              <td className="px-4 py-2 text-sm">
                <span className={`inline-block px-2 py-1 rounded-full text-xs font-semibold
${statusColor(r.status)}`}>{r.status}</span>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
};
```

```
export default LeaveTable;
```

File: frontend\components\LeaveTabs.tsx

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import { useEffect, useState } from "react";
import { motion } from "framer-motion";

export default function LeaveTabs() {
  const pathname = usePathname();
  const [activePath, setActivePath] = useState(pathname);

  // Sync internal state if URL changes externally (e.g. browser back button)
  useEffect(() => {
    setActivePath(pathname);
  }, [pathname]);

  // Sync read: ensures it doesn't drop to null during page transitions
  const role = typeof window !== "undefined" ? localStorage.getItem("role") : null;

  const tabs = [
    { name: "Request Leave", path: "/apply-leave" },
    { name: "Leave History", path: "/leave-history" },
    ...(role === "hr"
      ? [
          { name: "Approval panel", path: "/approval" },
          { name: "Get Reports", path: "/reports" },
        ]
      : []),
  ];

  return (
    <div
      suppressHydrationWarning
      className="inline-flex w-max bg-white p-1 rounded-full shadow-lg relative mt-1 mb-6"
    >
      {tabs.map((tab) => {
        let active = false;

        if (tab.path === "/apply-leave") {
          active = activePath === tab.path;
        } else {
          active = activePath === tab.path || activePath.startsWith(`${tab.path}/`);
        }

        return (
          <Link
            key={tab.path}
            href={tab.path}
            onClick={() => setActivePath(tab.path)}
            className={`relative px-6 py-2 rounded-full text-sm font-medium transition-colors duration-300 z-10
${
              active ? "text-white" : "text-gray-600 hover:text-gray-800"
            }`}
          >
            {active && (
              <motion.div
                layoutId="active-leave-tab"
                className="absolute inset-0 bg-[#F2924E] rounded-full -z-10"
                transition={{
                  type: "spring",
                  stiffness: 500,
                  damping: 30,
                  mass: 0.5,
                }}
```

```
              />
            )}
            {tab.name}
          </Link>
        );
      })}
    </div>
  );
}
```

File: frontend\components\Logo.tsx

```
import React from 'react';

interface LogoProps {
  withText?: boolean;
}

const Logo: React.FC<LogoProps> = ({ withText = true }) => {
  return (
    <div className="flex items-center space-x-3">
      <div className="w-10 h-10 rounded-lg bg-orange-400 flex items-center justify-center flex-shrink-0">
        <svg viewBox="0 0 48 48" className="w-6 h-6" xmlns="http://www.w3.org/2000/svg" aria-hidden>
          <rect width="48" height="48" rx="8" fill="#fb923c" />
          <text x="50%" y="55%" textAnchor="middle" dominantBaseline="middle" fontWeight={800} fontSize={16}
fill="#fff">HR</text>
        </svg>
      </div>
      {withText && (
        <div className="leading-tight">
          <div className="text-lg font-semibold text-gray-900">HRMS</div>
          <div className="text-xs text-gray-500">Management System</div>
        </div>
      )}
    </div>
  );
};

export default Logo;
```

File: frontend\components\Navbar.tsx

```
import React from "react";
import { Bell, MessageSquare, User } from "lucide-react";

export default function Navbar() {
  return (
    <div className="h-16 bg-white border-b flex items-center justify-between px-12">

      <input
        type="text"
        placeholder="Search"
        className="bg-gray-100 px-4 py-2 rounded-lg w-96 outline-none text-sm"
      />

      <div className="flex items-center gap-6 text-gray-600">
        <Bell size={20} />
        <MessageSquare size={20} />
        <User size={20} />
      </div>

    </div>
  );
}
```

File: frontend\components\Sidebar.tsx

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";
import {
  LayoutGrid,
  Shield,
  UserPlus,
  CalendarDays,
  FileText,
  Settings,
} from "lucide-react";

const navItems = [
  { name: "Dashboard", href: "/", icon: LayoutGrid },
  { name: "Employee Management", href: "/employees", icon: Shield },
  { name: "Recruitment", href: "/recruitment", icon: UserPlus },
  { name: "Leave", href: "/apply-leave", icon: CalendarDays },
  { name: "Document", href: "/document", icon: FileText },
  { name: "Settings", href: "/settings", icon: Settings },
];

export default function Sidebar() {
  const pathname = usePathname() || "/";

  return (
    <aside className="fixed top-0 left-0 bottom-0 w-64 bg-[#F3F4F6] border-r border-gray-200 z-50">
      {/* Logo section */}
      <div className="px-6 pt-8 pb-6">
        <div className="flex items-center gap-3">
          <div className="w-14 h-14 rounded-2xl bg-orange-500 flex items-center justify-center text-white
font-bold">
            HR
          </div>

          <div className="leading-tight">
            <div className="text-lg font-semibold text-gray-900">HRMS</div>
            <div className="text-xs text-gray-500">Management System</div>
          </div>
        </div>
      </div>

      {/* MAIN MENU title */}
      <div className="px-6 text-xs font-semibold text-gray-400 mb-3">
        MAIN MENU
      </div>

      {/* Navigation */}
      <nav className="px-4 space-y-2">
        {navItems.map((item) => {
          const Icon = item.icon;
          const isActive =
            item.name === "Leave"
              ? pathname === "/apply-leave" ||
              pathname === "/leave-history" ||
              pathname === "/approval" ||
              pathname === "/reports"
              : pathname === item.href || pathname.startsWith(item.href + "/");
          return (
            <Link
              key={item.name}
              href={item.href}
              className={`flex items-center gap-3 px-3 py-2 rounded-lg text-sm transition ${isActive ? "
text-orange-500" : "text-gray-600 hover:bg-white/70"
                }`}
            >
              <Icon
                size={20}
                className={isActive ? "text-orange-500" : "text-gray-500"}
```

```
              />
              <span className={isActive ? "font-medium" : ""}>{item.name}</span>
            </Link>
          );
        })}
      </nav>

      {/* Footer */}
      <div className="absolute bottom-4 left-0 w-full px-6 text-xs text-gray-400">
        © 2026 HRSM
      </div>
    </aside>
  );

  const role = localStorage.getItem("role");

  <ul>
    <li><Link href="/apply-leave">Apply Leave</Link></li>
    <li><Link href="/leave-history">Leave History</Link></li>

    {role !== "employee" && (
      <li><Link href="/approval">Approval</Link></li>
    )}

    {role === "hr" && (
      <li><Link href="/reports">Reports</Link></li>
    )}
  </ul>
}
```

File: frontend\components\TopBar.tsx

```
"use client";

import { Search, Bell, MessageSquare, User } from "lucide-react";

export default function TopBar() {
  return (
    <header className="fixed top-0 left-0 right-0 z-40 h-16 border-b border-gray-200">
      <div className="h-full flex items-center">
        {/* ■ left area same as sidebar */}
        <div className="w-64 h-full bg-[#F3F4F6] border-r border-gray-200" />

        {/* ■ right area white */}
        <div className="flex-1 h-full bg-white flex items-center justify-between px-10">
          <div className="flex items-center bg-gray-100 px-5 py-2 rounded-full w-[320px]">
            <Search size={18} className="text-gray-400 mr-3" />
            <input
              type="text"
              placeholder="Search"
              className="bg-transparent outline-none w-full text-sm text-gray-600 placeholder-gray-400"
            />
          </div>

          <div className="flex items-center gap-6 text-gray-600">
            <Bell size={20} className="cursor-pointer hover:text-black" />
            <MessageSquare size={20} className="cursor-pointer hover:text-black" />
            <User size={20} className="cursor-pointer hover:text-black" />
          </div>
        </div>
      </div>
    </header>
  );
}
```

File: frontend\components\UploadIcon.tsx

```tsx
import React from 'react';

interface Props {
  className?: string;
}

const UploadIcon: React.FC<Props> = ({ className = 'h-12 w-12 text-gray-400' }) => (
  <svg
    className={className}
    viewBox="0 0 24 24"
    fill="none"
    stroke="currentColor"
    strokeWidth={2}
    strokeLinecap="round"
    strokeLinejoin="round"
    xmlns="http://www.w3.org/2000/svg"
  >
    <path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4" />
    <polyline points="7 10 12 5 17 10" />
    <line x1="12" y1="5" x2="12" y2="19" />
  </svg>
);

export default UploadIcon;
```

File: frontend\components\reports\AnalyticsSection.tsx

```tsx
"use client";

import React, { useState, useMemo } from "react";

interface LeaveRecord {
  employee_id: number;
  employee_name?: string | null;
  start_date: string;
  end_date: string;
  status: string;
}

interface AnalyticsSectionProps {
  records: LeaveRecord[];
}

export default function AnalyticsSection({ records }: AnalyticsSectionProps) {
  const [activeTab, setActiveTab] = useState<"weekly" | "monthly" | "yearly">("monthly");

  // Filter only approved leaves
  const approvedRecords = useMemo(() => {
    return records.filter((record) => record.status === "APPROVED");
  }, [records]);

  const weeklyData = useMemo(() => {
    const weekMap = new Map<string, Set<number>>();
    const now = new Date();

    approvedRecords.forEach((record) => {
      const startDate = new Date(record.start_date);
      const endDate = new Date(record.end_date);

      // Only count leaves from the last 12 weeks
      const twoMonthsAgo = new Date(now);
      twoMonthsAgo.setDate(twoMonthsAgo.getDate() - 84);

      if (endDate < twoMonthsAgo) return;

      let currentDate = new Date(startDate);
      while (currentDate <= endDate) {
        const weekStart = new Date(currentDate);
        weekStart.setDate(weekStart.getDate() - weekStart.getDay());
```

```
      const weekKey = weekStart.toISOString().split("T")[0];

      if (!weekMap.has(weekKey)) {
        weekMap.set(weekKey, new Set());
      }
      weekMap.get(weekKey)!.add(record.employee_id);

      currentDate.setDate(currentDate.getDate() + 1);
    }
  });

  const data = Array.from(weekMap)
    .sort((a, b) => a[0].localeCompare(b[0]))
    .map(([week, employees]) => ({
      week: new Date(week).toLocaleDateString("en-US", { month: "short", day: "numeric" }),
      employees: employees.size,
    }))
    .slice(-12);

  return data;
}, [approvedRecords]);

const monthlyData = useMemo(() => {
  const monthMap = new Map<string, Set<number>>();
  const now = new Date();
  const twelveMonthsAgo = new Date(now);
  twelveMonthsAgo.setFullYear(twelveMonthsAgo.getFullYear() - 1);

  approvedRecords.forEach((record) => {
    const startDate = new Date(record.start_date);
    const endDate = new Date(record.end_date);

    if (endDate < twelveMonthsAgo) return;

    let currentDate = new Date(startDate);
    while (currentDate <= endDate) {
      const monthKey = currentDate.toISOString().slice(0, 7);

      if (!monthMap.has(monthKey)) {
        monthMap.set(monthKey, new Set());
      }
      monthMap.get(monthKey)!.add(record.employee_id);

      currentDate.setDate(currentDate.getDate() + 1);
    }
  });

  const data = Array.from(monthMap)
    .sort((a, b) => a[0].localeCompare(b[0]))
    .map(([month, employees]) => ({
      month: new Date(month + "-01").toLocaleDateString("en-US", { month: "short", year: "2-digit" }),
      employees: employees.size,
    }));

  return data;
}, [approvedRecords]);

const yearlyData = useMemo(() => {
  const yearMap = new Map<string, Set<number>>();

  approvedRecords.forEach((record) => {
    const startDate = new Date(record.start_date);
    const endDate = new Date(record.end_date);

    let currentDate = new Date(startDate);
    while (currentDate <= endDate) {
      const year = currentDate.getFullYear().toString();

      if (!yearMap.has(year)) {
        yearMap.set(year, new Set());
      }
      yearMap.get(year)!.add(record.employee_id);
```

```
        currentDate.setDate(currentDate.getDate() + 1);
      }
    });

    const data = Array.from(yearMap)
      .sort((a, b) => a[0].localeCompare(b[0]))
      .map(([year, employees]) => ({
        year,
        employees: employees.size,
      }));

    return data;
  }, [approvedRecords]);

  const getChartData = () => {
    switch (activeTab) {
      case "weekly":
        return weeklyData;
      case "monthly":
        return monthlyData;
      case "yearly":
        return yearlyData;
      default:
        return monthlyData;
    }
  };

  const getAxisLabel = () => {
    switch (activeTab) {
      case "weekly":
        return "Week";
      case "monthly":
        return "Month";
      case "yearly":
        return "Year";
      default:
        return "Month";
    }
  };

  const chartData = getChartData();
  const maxEmployees = Math.max(...chartData.map((d) => d.employees), 1);

  return (
    <div className="mt-8 rounded-3xl border border-gray-200 bg-white p-5 shadow-[0_1px_3px_rgba(16,24,40,0.06)]
md:p-6">
      <div className="mb-6">
        <h2 className="text-xl font-semibold text-gray-900 mb-4">Employee Leave Analytics</h2>

        {/* Tabs */}
        <div className="flex gap-2 border-b border-gray-200">
          <button
            onClick={() => setActiveTab("weekly")}
            className={`px-4 py-2 font-medium transition ${
              activeTab === "weekly"
                ? "border-b-2 border-orange-500 text-orange-500"
                : "text-gray-600 hover:text-gray-900"
            }`}
          >
            Weekly View
          </button>
          <button
            onClick={() => setActiveTab("monthly")}
            className={`px-4 py-2 font-medium transition ${
              activeTab === "monthly"
                ? "border-b-2 border-orange-500 text-orange-500"
                : "text-gray-600 hover:text-gray-900"
            }`}
          >
            Monthly View
          </button>
```

```
          <button
            onClick={() => setActiveTab("yearly")}
            className={`px-4 py-2 font-medium transition ${
              activeTab === "yearly"
                ? "border-b-2 border-orange-500 text-orange-500"
                : "text-gray-600 hover:text-gray-900"
            }`}
          >
            Annual View
          </button>
        </div>
      </div>

      {/* Simple Bar Chart */}
      <div className="mt-8">
        {chartData.length === 0 ? (
          <div className="text-center py-12 text-gray-500">
            No leave data available for this period
          </div>
        ) : (
          <div className="flex flex-col gap-6">
            {/* Chart */}
            <div className="flex items-end justify-start gap-4 h-64 overflow-x-auto pb-4">
              {chartData.map((data, index) => {
                const height = (data.employees / maxEmployees) * 200;
                const label = 'week' in data ? data.week : 'month' in data ? data.month : 'year' in data ?
data.year : '';
                return (
                  <div key={index} className="flex flex-col items-center gap-2 min-w-fit">
                    <div className="relative h-48 w-12">
                      <div
                        className="absolute bottom-0 left-0 right-0 bg-orange-500 rounded-t-lg transition-all
hover:bg-orange-600 cursor-pointer"
                        style={{ height: `${height}px` }}
                        title={`${data.employees} employees`}
                      ></div>
                      <div className="absolute -top-6 left-0 right-0 text-center text-sm font-semibold
text-gray-900">
                        {data.employees}
                      </div>
                    </div>
                    <div className="text-xs text-gray-600 text-center max-w-fit">{label}</div>
                  </div>
                );
              })}
            </div>

            {/* Legend */}
            <div className="flex items-center justify-between pt-4 border-t border-gray-200">
              <div className="flex items-center gap-2">
                <div className="w-3 h-3 bg-orange-500 rounded-sm"></div>
                <span className="text-sm text-gray-600">Employees on Leave</span>
              </div>
              <div className="text-sm text-gray-500">
                Y-axis: Number of Employees | X-axis: {getAxisLabel()}
              </div>
            </div>
          </div>
        )}
      </div>
    </div>
  );
}
```

File: frontend\components\reports\AttendanceRecordsTable.tsx

```
import React from "react";
import { AttendanceRecord } from "@/app/reports/types";
```

```
interface Props {
  records: AttendanceRecord[];
}

export default function AttendanceRecordsTable({ records }: Props) {
  return (
    <div className="mt-4 overflow-x-auto rounded-2xl border border-gray-200 bg-white p-4 md:p-6">
      <table className="w-full min-w-[900px]">
        <thead>
          <tr className="text-left text-xs uppercase text-gray-400">
            <th className="py-3 font-medium">Employee</th>
            <th className="py-3 font-medium">Department</th>
            <th className="py-3 font-medium">Present</th>
            <th className="py-3 font-medium">Absent</th>
            <th className="py-3 font-medium">Total Days</th>
            <th className="py-3 font-medium">Rate</th>
            <th className="py-3 font-medium">Trend</th>
          </tr>
        </thead>

        <tbody>
          {records.map((record) => (
            <tr key={record.employeeCode} className="border-t border-gray-100">
              <td className="py-4">
                <div className="text-sm font-medium text-gray-900">
                  {record.employeeName}
                </div>
                <div className="text-sm text-gray-500">{record.employeeCode}</div>
              </td>
              <td className="py-4 text-sm text-gray-700">{record.department}</td>
              <td className="py-4 text-sm text-green-600">{record.present} days</td>
              <td className="py-4 text-sm text-red-400">{record.absent} days</td>
              <td className="py-4 text-sm text-gray-700">{record.totalDays} days</td>
              <td className="py-4 text-sm text-gray-700">
                <div className="flex items-center gap-3">
                  <div className="h-1.5 w-16 rounded-full bg-orange-100">
                    <div
                      className="h-1.5 rounded-full bg-orange-500"
                      style={{ width: `${record.rate}%` }}
                    />
                  </div>
                  <span>{record.rate}%</span>
                </div>
              </td>
              <td className="py-4 text-sm text-red-400">{record.trend}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

File: frontend\components\reports\EmployeeDetailTabs.tsx

```
"use client";

import React from "react";
import { DetailTab } from "@/app/reports/types";

interface Props {
  activeTab: DetailTab;
  setActiveTab: (tab: DetailTab) => void;
  violationCount?: number;
}

export default function EmployeeDetailTabs({
  activeTab,
  setActiveTab,
```

```
  violationCount = 0,
}: Props) {
  const buttonClass = (tab: DetailTab) =>
    `rounded-xl px-4 py-2 text-sm ${activeTab === tab
      ? "bg-orange-500 text-white"
      : "text-gray-500 hover:bg-gray-100"
    }`;

  return (
    <div className="mt-6 flex items-center gap-3">
      <button
        type="button"
        onClick={() => setActiveTab("attendance")}
        className={buttonClass("attendance")}
      >
        Attendance Records
      </button>

      <button
        type="button"
        onClick={() => setActiveTab("leave")}
        className={buttonClass("leave")}
      >
        Leave History
      </button>

      <div className="relative">
        <button
          type="button"
          onClick={() => setActiveTab("violations")}
          className={buttonClass("violations")}
        >
          Violations
        </button>

        {violationCount > 0 && (
          <span className="absolute -right-2 -top-2 inline-flex h-5 min-w-5 items-center justify-center
rounded-full bg-red-500 px-1 text-[10px] font-semibold text-white">
            {violationCount}
          </span>
        )}
      </div>
    </div>
  );
}
```

File: frontend\components\reports\EmployeeReportHeader.tsx

```
"use client";

import React from "react";
import { Download, Calendar, ChevronDown } from "lucide-react";
import { EmployeeDetail, ReportPeriod } from "@/app/reports/types";

interface Props {
  employee: EmployeeDetail;
  period: ReportPeriod;
  setPeriod: (value: ReportPeriod) => void;
}

export default function EmployeeReportHeader({
  employee,
  period,
  setPeriod,
}: Props) {
  const handleDownload = () => {
    const today = new Date();
    const startDate = new Date();
```

```
   if (period === "weekly") {
     startDate.setDate(today.getDate() - 7);
   } else if (period === "monthly") {
     startDate.setMonth(today.getMonth() - 1);
   } else if (period === "annually") {
     startDate.setFullYear(today.getFullYear() - 1);
   }

   const startStr = startDate.toISOString().split("T")[0];
   const endStr = today.toISOString().split("T")[0];

   // Construct the URL with filters and dates
   const url = `http://127.0.0.1:8000/reports/leave/pdf?employee_id=${employee.id}&start_date=${startStr}&end_d
ate=${endStr}`;

   // Trigger download
   window.location.href = url;
 };
 const getInitials = (name: string) => {
   if (!name) return "";
   return name
     .split(" ")
     .map((n) => n[0])
     .join("")
     .substring(0, 2)
     .toUpperCase();
 };

 return (
   <div className="mt-4 overflow-hidden rounded-2xl border border-gray-200 bg-white">
     {/* TOP HEADER SECTION */}
     <div
       className="flex items-center justify-between px-5 py-4"
       style={{ backgroundColor: "#ffffffff" }}
     >
       {/* LEFT SIDE */}
       <div className="flex items-center gap-5">
         <div
           className="flex h-14 w-14 shrink-0 items-center justify-center rounded-full"
           style={{ backgroundColor: "#F9D2BB" }}
         >
           <span
             className="text-[18px] font-medium leading-none"
             style={{ color: "#8D4A27" }}
           >
             {getInitials(employee.name)}
           </span>
         </div>

         <div className="min-w-[160px]">
           <h2
             className="text-[18px] font-semibold leading-tight"
             style={{ color: "#2D3748" }}
           >
             {employee.name}
           </h2>
           <p
             className="mt-1 text-[14px] font-medium"
             style={{ color: "#667085" }}
           >
             {employee.employeeCode}
           </p>
         </div>

         <div className="min-w-[110px]">
           <p className="text-[14px] text-gray-500">Department</p>
           <p className="text-[15px] font-medium text-gray-800">
             {employee.department}
           </p>
         </div>

         <div className="min-w-[140px]">
```

```
            <p className="text-[14px] text-gray-500">Position</p>
            <p className="text-[15px] font-medium text-gray-800">
              Senior Developer
            </p>
          </div>

          <div className="min-w-[110px]">
            <p className="text-[14px] text-gray-500">Manager</p>
            <p className="text-[15px] font-medium text-gray-800">
              {employee.manager}
            </p>
          </div>

          <div className="min-w-[120px]">
            <p className="text-[14px] text-gray-500">Join Date</p>
            <p className="text-[15px] font-medium text-gray-800">
              {employee.joinDate}
            </p>
          </div>
        </div>

        {/* RIGHT SIDE */}
        <div className="flex items-end gap-2">

          <button
            onClick={handleDownload}
            className="mt-2 inline-flex h-12 items-center justify-center gap-2 rounded-xl bg-[#F2924E] px-4
text-sm font-medium text-white shadow-sm transition hover:bg-[#d87c3b]"
          >
            <Download className="h-5 w-5" />
            Download Full Report(PDF)
          </button>
        </div>
      </div>
    </div>
  );
}
```

File: frontend\components\reports\EmployeeReportStats.tsx

```
import React from "react";
import { ArrowDownRight, ArrowUpRight } from "lucide-react";
import { EmployeeDetail } from "@/app/reports/types";

interface Props {
  employee: EmployeeDetail;
  activeTab: "attendance" | "leave" | "violations";
  period: "weekly" | "monthly" | "annually";
  setPeriod: (value: "weekly" | "monthly" | "annually") => void;
}

export default function EmployeeReportStats({ employee, period, setPeriod }: Props) {
  const cards = [
    {
      label: "Attendance Rate",
      value: `${employee.attendanceRate}%`,
      note: "Critical - Below Standards",
      noteColor: "text-red-500",
      icon: <ArrowDownRight className="h-4 w-4 text-red-500" />,
    },
    {
      label: "Absent Days",
      value: employee.absentDays,
      note: "Out of 22 working days",
      noteColor: "text-gray-500",
      icon: null,
    },
    {
      label: "Late Arrivals",
```

```
    value: employee.lateArrivals,
    note: "Exceeded limit",
    noteColor: "text-red-500",
    icon: null,
  },
  {
    label: "Total Violations",
    value: employee.totalViolations,
    note:
      employee.totalViolations > 0
        ? `${employee.totalViolations} High-severity`
        : "No violations",
    noteColor: employee.totalViolations > 0 ? "text-red-500" : "text-gray-500",
    icon: employee.totalViolations > 0 ? (
      <ArrowUpRight className="h-4 w-4 text-red-500" />
    ) : null,
  },
];

return (
  <div className="mt-4">
    {/* Report Tabs */}
    <div className="flex gap-6 border-b border-gray-200 pb-0">
      <button
        onClick={() => setPeriod("weekly")}
        className={`pb-3 text-sm font-medium transition-colors ${
          period === "weekly"
            ? "border-b-2 border-[#F2924E] text-gray-900"
            : "text-gray-600 hover:text-gray-900"
        }`}
      >
        Weekly Report
      </button>
      <button
        onClick={() => setPeriod("monthly")}
        className={`pb-3 text-sm font-medium transition-colors ${
          period === "monthly"
            ? "border-b-2 border-[#F2924E] text-gray-900"
            : "text-gray-600 hover:text-gray-900"
        }`}
      >
        Monthly Report
      </button>
      <button
        onClick={() => setPeriod("annually")}
        className={`pb-3 text-sm font-medium transition-colors ${
          period === "annually"
            ? "border-b-2 border-[#F2924E] text-gray-900"
            : "text-gray-600 hover:text-gray-900"
        }`}
      >
        Annual Report
      </button>
    </div>

    {/* Stats Cards */}
    <div className="mt-6 grid grid-cols-1 gap-4 md:grid-cols-2 xl:grid-cols-4">
    {cards.map((card) => (
      <div
        key={card.label}
        className="rounded-[20px] border border-gray-200 bg-white p-5 shadow-sm"
      >
        <div className="flex items-start justify-between gap-3">
          <p className="text-sm font-semibold text-gray-600">
            {card.label}
          </p>
          {card.icon ? (
            <span className="flex h-8 w-8 items-center justify-center rounded-full bg-red-50">
              {card.icon}
            </span>
          ) : (
```

```
            <span className="h-8 w-8" />
          )}
        </div>
        <h3 className="mt-4 text-3xl font-bold text-gray-900">{card.value}</h3>
        <p className={`mt-3 text-sm ${card.noteColor}`}>{card.note}</p>
      </div>
    ))}
    </div>
  </div>
 );
}
```

File: frontend\components\reports\EmployeeReportTable.tsx

```
import React from "react";
import Link from "next/link";
import { EmployeeReportRow } from "@/app/reports/types";

interface Props {
  rows: EmployeeReportRow[];
}

export default function EmployeeReportTable({ rows }: Props) {
  return (
    <div className="mt-6 overflow-x-auto">
      <table className="w-full min-w-[1000px] border-separate border-spacing-0">
        <thead>
          <tr className="text-left text-xs uppercase text-gray-400">
            <th className="px-4 py-4 font-medium">Employee Name</th>
            <th className="px-4 py-4 font-medium">Department</th>
            <th className="px-4 py-4 font-medium">Allocated Leave Days</th>
            <th className="px-4 py-4 font-medium">Used Leave Days</th>
            <th className="px-4 py-4 font-medium">Pending Leave Days</th>
            <th className="px-4 py-4 font-medium">Remaining Leave Balance</th>
            <th className="px-4 py-4 font-medium">Last Leave Date</th>
            <th className="px-4 py-4 font-medium">Actions</th>
          </tr>
        </thead>

        <tbody>
          {rows.map((row) => (
            <tr key={row.id} className="border-t border-gray-100">
              <td className="px-4 py-4 align-top">
                <div className="text-sm font-medium text-gray-900">{row.name}</div>
                <div className="text-sm text-gray-500">{row.role}</div>
              </td>

              <td className="px-4 py-4 text-sm text-gray-700">{row.department}</td>
              <td className="px-4 py-4 text-sm text-gray-700">{row.totalLeave} days</td>
              <td className="px-4 py-4 text-sm text-orange-500">{row.used} days</td>
              <td className="px-4 py-4 text-sm text-orange-500">{row.pending} days</td>
              <td className="px-4 py-4 text-sm text-green-600">{row.remaining} days</td>
              <td className="px-4 py-4 text-sm text-gray-700">
                {new Date(row.lastLeave).toLocaleDateString("en-US", {
                  month: "short",
                  day: "numeric",
                  year: "numeric",
                })}
              </td>
              <td className="px-4 py-4 text-sm">
                <Link
                  href={`/reports/${row.id}`}
                  className="font-medium text-orange-500 hover:text-orange-600"
                >
                  View Details
                </Link>
              </td>
            </tr>
          ))}
```

```
        </tbody>
      </table>
    </div>
  );
}
```

File: frontend\components\reports\LeaveHistoryPanel.tsx

```
import React from "react";
import { LeaveHistoryRecord } from "@/app/reports/types";

interface Props {
  records: LeaveHistoryRecord[];
}

export default function LeaveHistoryPanel({ records }: Props) {
  const totalAllowed = 20;
  const leaveUsed = 8;
  const leaveRemaining = 12;

  const statusClasses = (status: LeaveHistoryRecord["status"]) => {
    if (status === "APPROVED") {
      return "bg-green-100 text-green-600";
    }
    if (status === "PENDING") {
      return "bg-yellow-100 text-yellow-700";
    }
    return "bg-red-100 text-red-600";
  };

  return (
    <div className="mt-4 rounded-2xl border border-gray-200 bg-white p-4 md:p-6">
      <div className="mb-6 grid grid-cols-1 gap-4 md:grid-cols-3">
        <div className="rounded-xl border border-gray-200 bg-gray-50 p-4">
          <p className="text-xs text-gray-400">Total Leave Allowed</p>
          <h3 className="mt-2 text-3xl font-semibold text-gray-900">20 days</h3>
        </div>

        <div className="rounded-xl border border-orange-100 bg-orange-50 p-4">
          <p className="text-xs text-gray-400">Leave Used</p>
          <h3 className="mt-2 text-3xl font-semibold text-orange-500">
            {leaveUsed} days
          </h3>
        </div>

        <div className="rounded-xl border border-green-100 bg-green-50 p-4">
          <p className="text-xs text-gray-400">Leave Remaining</p>
          <h3 className="mt-2 text-3xl font-semibold text-green-600">
            {leaveRemaining} days
          </h3>
        </div>
      </div>

      <div className="overflow-x-auto">
        <table className="w-full min-w-[900px]">
          <thead>
            <tr className="text-left text-xs uppercase text-gray-400">
              <th className="py-3 font-medium">Type</th>
              <th className="py-3 font-medium">Start Date</th>
              <th className="py-3 font-medium">End Date</th>
              <th className="py-3 font-medium">Days</th>
              <th className="py-3 font-medium">Reason</th>
              <th className="py-3 font-medium">Status</th>
            </tr>
          </thead>

          <tbody>
            {records.map((record, index) => (
              <tr key={`${record.type}-${index}`} className="border-t border-gray-100">
```

```
                <td className="py-4 text-sm text-gray-700">{record.type}</td>
                <td className="py-4 text-sm text-gray-700">{record.startDate}</td>
                <td className="py-4 text-sm text-gray-700">{record.endDate}</td>
                <td className="py-4 text-sm text-gray-700">{record.days}</td>
                <td className="py-4 text-sm text-gray-700">{record.reason}</td>
                <td className="py-4 text-sm">
                  <span
                    className={`inline-flex rounded-full px-3 py-1 text-xs font-semibold ${statusClasses(
                      record.status
                    )}`}
                  >
                    {record.status}
                  </span>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}
```

File: frontend\components\reports\ManagerNotesCard.tsx

```
"use client";

import React, { useState } from "react";

export default function ManagerNotesCard() {
  const [notes, setNotes] = useState("");

  return (
    <div className="mt-4 rounded-2xl border border-gray-200 bg-white p-4 md:p-6">
      <h3 className="text-sm font-semibold text-gray-800">
        Manager Notes & Recommendations
      </h3>

      <textarea
        value={notes}
        onChange={(e) => setNotes(e.target.value)}
        rows={5}
        placeholder="Add notes about this employee's performance, disciplinary actions taken, or
recommendations for HR..."
        className="mt-4 w-full rounded-xl border border-gray-200 px-4 py-3 text-sm text-gray-700 outline-none"
      />

      <div className="mt-3 flex items-center justify-between">
        <p className="text-xs text-gray-400">
          These notes will be included in the comprehensive PDF report
        </p>

        <button className="rounded-xl border border-gray-200 bg-gray-50 px-4 py-2 text-sm text-gray-600
hover:bg-gray-100">
          Save Notes
        </button>
      </div>
    </div>
  );
}
```

File: frontend\components\reports\ReportFilterBar.tsx

```
"use client";

import React from "react";
import { Search, FileText, Download } from "lucide-react";
```

```
import { ReportPeriod } from "@/app/reports/types";

interface Props {
  period: ReportPeriod;
  setPeriod: (value: ReportPeriod) => void;
  search: string;
  setSearch: (value: string) => void;
  department: string;
  setDepartment: (value: string) => void;
  departments: string[];
  onExportPdf?: () => void;
  onExportCsv?: () => void;
}

export default function ReportFilterBar({
  period,
  setPeriod,
  search,
  setSearch,
  department,
  setDepartment,
  departments,
  onExportPdf,
  onExportCsv,
}: Props) {
  const periodButton = (value: ReportPeriod) =>
    `relative text-sm px-1 pb-2 font-medium transition-colors duration-200 ${
      period === value
        ? "text-[#F2924E] after:absolute after:bottom-0 after:left-0 after:w-full after:h-0.5
after:bg-[#F2924E] after:rounded-full"
        : "text-gray-500 hover:text-gray-700"
    }`;

  return (
    <div className="flex flex-col gap-4">
      {/* Top row: period tabs + export buttons */}
      <div className="flex flex-wrap items-center justify-between gap-4 border-b border-gray-100 pb-3">
        <div className="flex items-center gap-8">
          <button onClick={() => setPeriod("weekly")} className={periodButton("weekly")}>
            Weekly
          </button>
          <button onClick={() => setPeriod("monthly")} className={periodButton("monthly")}>
            Monthly
          </button>
          <button onClick={() => setPeriod("annually")} className={periodButton("annually")}>
            Annually
          </button>
        </div>

        <div className="flex flex-wrap items-center gap-3">
          <button
            onClick={onExportPdf}
            className="inline-flex items-center gap-2 rounded-xl bg-[#F2924E] px-4 py-2 text-sm font-medium
text-white hover:bg-orange-600 transition-colors"
          >
            <FileText size={16} />
            Export PDF
          </button>

          <button
            onClick={onExportCsv}
            className="inline-flex items-center gap-2 rounded-xl border border-gray-300 bg-white px-4 py-2
text-sm font-medium text-gray-700 hover:bg-gray-50 transition-colors"
          >
            <Download size={16} />
            Export CSV
          </button>
        </div>
      </div>

      {/* Bottom row: search + department filter */}
      <div className="flex flex-col md:flex-row gap-3">
```

```
        <div className="relative w-full md:max-w-sm">
          <Search
            size={18}
            className="absolute left-4 top-1/2 -translate-y-1/2 text-gray-400"
          />
          <input
            type="text"
            placeholder="Search for name or ID..."
            value={search}
            onChange={(e) => setSearch(e.target.value)}
            className="w-full rounded-xl border border-gray-200 bg-white py-3 pl-11 pr-4 text-sm text-gray-700
outline-none focus:border-orange-300 transition-colors"
          />
        </div>

        <select
          value={department}
          onChange={(e) => setDepartment(e.target.value)}
          className="rounded-xl border border-gray-200 bg-white px-4 py-3 text-sm text-gray-700 outline-none
md:min-w-[200px] focus:border-orange-300 transition-colors"
        >
          {departments.map((item) => (
            <option key={item} value={item}>
              {item}
            </option>
          ))}
        </select>
      </div>
    </div>
  );
}
```

File: frontend\components\reports\ReportSummaryCards.tsx

```
import React from "react";
import { SummaryCardItem } from "@/app/reports/types";

interface Props {
  cards: SummaryCardItem[];
}

export default function ReportSummaryCards({ cards }: Props) {
  return (
    <div className="mt-6 grid grid-cols-1 gap-6 md:grid-cols-2 xl:grid-cols-4">
      {cards.map((card) => (
        <div
          key={card.label}
          className="bg-[#FFF3E6] border border-[#F5C28B] rounded-xl p-5"
        >
          <p className="text-sm text-gray-600 mb-1">{card.label}</p>
          <h2 className="text-2xl font-bold text-gray-800">{card.value}</h2>
        </div>
      ))}
    </div>
  );
}
```

File: frontend\components\reports\ViolationsPanel.tsx

```
import React from "react";
import { AlertTriangle } from "lucide-react";
import { ViolationRecord } from "@/app/reports/types";

interface Props {
  records: ViolationRecord[];
}
```

```
export default function ViolationsPanel({ records }: Props) {
  const severityClasses = (severity: ViolationRecord["severity"]) => {
    if (severity === "HIGH SEVERITY") {
      return {
        card: "border-red-200 bg-red-50",
        badge: "bg-red-500 text-white",
      };
    }

    return {
      card: "border-orange-200 bg-orange-50",
      badge: "bg-orange-400 text-white",
    };
  };

  return (
    <div className="mt-4 rounded-2xl border border-gray-200 bg-white p-4 md:p-6">
      <div className="mb-4 rounded-xl border border-red-200 bg-red-50 p-4">
        <div className="flex items-start gap-3">
          <AlertTriangle size={18} className="mt-0.5 text-red-500" />
          <div>
            <h4 className="text-sm font-semibold text-red-600">
              Attendance Violation Summary
            </h4>
            <p className="mt-1 text-xs text-red-500">
              This employee has {records.length} documented violations, including
              high-severity incidents. This documentation can be used for
              disciplinary actions.
            </p>
          </div>
        </div>
      </div>

      <div className="space-y-3">
        {records.map((record, index) => {
          const styles = severityClasses(record.severity);

          return (
            <div
              key={`${record.title}-${index}`}
              className={`rounded-xl border p-4 ${styles.card}`}
            >
              <div className="flex items-start justify-between gap-4">
                <div className="flex items-start gap-3">
                  <AlertTriangle size={18} className="mt-0.5 text-orange-500" />
                  <div>
                    <h5 className="text-sm font-semibold text-gray-800">
                      {record.title}
                    </h5>
                    <p className="mt-1 text-xs text-gray-500">{record.date}</p>
                    <p className="mt-2 text-sm text-gray-700">
                      {record.description}
                    </p>
                  </div>
                </div>

                <span
                  className={`inline-flex rounded-full px-3 py-1 text-[10px] font-semibold ${styles.badge}`}
                >
                  {record.severity}
                </span>
              </div>
            </div>
          );
        })}
      </div>
    </div>
  );
}
```

File: vite-project\eslint.config.js

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
      parserOptions: {
        ecmaVersion: 'latest',
        ecmaFeatures: { jsx: true },
        sourceType: 'module',
      },
    },
    rules: {
      'no-unused-vars': ['error', { varsIgnorePattern: '^[A-Z_]' }],
    },
  },
])
```

File: vite-project\index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>vite-project</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

File: vite-project\package.json

```
{
  "name": "vite-project",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.0",
    "react-dom": "^19.2.0"
  },
  "devDependencies": {
```

```
    "@eslint/js": "^9.39.1",
    "@types/react": "^19.2.7",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^5.1.1",
    "eslint": "^9.39.1",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "vite": "^7.3.1"
  }
}
```

File: vite-project\README.md

```
# React + Vite

This template provides a minimal setup to get React working in Vite with HMR and some ESLint rules.

Currently, two official plugins are available:

- [@vitejs/plugin-react](https://github.com/vitejs/vite-plugin-react/blob/main/packages/plugin-react) uses
[Babel](https://babeljs.io/) (or [oxc](https://oxc.rs) when used in
[rolldown-vite](https://vite.dev/guide/rolldown)) for Fast Refresh
- [@vitejs/plugin-react-swc](https://github.com/vitejs/vite-plugin-react/blob/main/packages/plugin-react-swc)
uses [SWC](https://swc.rs/) for Fast Refresh

## React Compiler

The React Compiler is not enabled on this template because of its impact on dev & build performances. To add
it, see [this documentation](https://react.dev/learn/react-compiler/installation).

## Expanding the ESLint configuration

If you are developing a production application, we recommend using TypeScript with type-aware lint rules
enabled. Check out the [TS template](https://github.com/vitejs/vite/tree/main/packages/create-vite/template-reac
t-ts) for information on how to integrate TypeScript and [`typescript-eslint`](https://typescript-eslint.io) in
your project.
```

File: vite-project\vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

File: vite-project\src\App.css

```
#root {
  max-width: 1280px;
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

.logo {
  height: 6em;
  padding: 1.5em;
  will-change: filter;
  transition: filter 300ms;
}
.logo:hover {
```

```
  filter: drop-shadow(0 0 2em #646cffaa);
}
.logo.react:hover {
  filter: drop-shadow(0 0 2em #61dafbaa);
}

@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}

.card {
  padding: 2em;
}

.read-the-docs {
  color: #888;
}
```

File: vite-project\src\App.jsx

```
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from '/vite.svg'
import './App.css'

function App() {
  const [count, setCount] = useState(0)

  return (
    <>
      <div>
        <a href="https://vite.dev" target="_blank">
          <img src={viteLogo} className="logo" alt="Vite logo" />
        </a>
        <a href="https://react.dev" target="_blank">
          <img src={reactLogo} className="logo react" alt="React logo" />
        </a>
      </div>
      <h1>Vite + React</h1>
      <div className="card">
        <button onClick={() => setCount((count) => count + 1)}>
          count is {count}
        </button>
        <p>
          Edit <code>src/App.jsx</code> and save to test HMR
        </p>
      </div>
      <p className="read-the-docs">
        Click on the Vite and React logos to learn more
      </p>
    </>
  )
}

export default App
```

File: vite-project\src\index.css

```
:root {
  font-family: system-ui, Avenir, Helvetica, Arial, sans-serif;
  line-height: 1.5;
  font-weight: 400;

  color-scheme: light dark;
  color: rgba(255, 255, 255, 0.87);
  background-color: #242424;

  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

a {
  font-weight: 500;
  color: #646cff;
  text-decoration: inherit;
}
a:hover {
  color: #535bf2;
}

body {
  margin: 0;
  display: flex;
  place-items: center;
  min-width: 320px;
  min-height: 100vh;
}

h1 {
  font-size: 3.2em;
  line-height: 1.1;
}

button {
  border-radius: 8px;
  border: 1px solid transparent;
  padding: 0.6em 1.2em;
  font-size: 1em;
  font-weight: 500;
  font-family: inherit;
  background-color: #1a1a1a;
  cursor: pointer;
  transition: border-color 0.25s;
}
button:hover {
  border-color: #646cff;
}
button:focus,
button:focus-visible {
  outline: 4px auto -webkit-focus-ring-color;
}

@media (prefers-color-scheme: light) {
  :root {
    color: #213547;
    background-color: #ffffff;
  }
  a:hover {
    color: #747bff;
  }
  button {
    background-color: #f9f9f9;
  }
}
```

File: vite-project\src\main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```